ДИПЛОМНА РАБОТА

по професия код 523050 „Техник на компютърни системи“
специалност код 5230502 „Компютърни мрежи“

Тема:
Подсигуряване на динамичното обвързване MAC-IP и IP-MAC
на ниво комуникационен протокол

Дипломант:

Никола Емилов Алексов

Дипломен ръководител:

доц. д-р А. Цокев

Консултант:

д-р А. Иванов

СОФИЯ

2 0 2 6

ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

Дата на заданието: 17.10.2025 г. Утвърждавам:.....
Дата на предаване: 17.01.2026 г. /проф. д-р инж. П. Якимов/

ЗАДАНИЕ
за дипломна работа

ДЪРЖАВЕН ИЗПИТ ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ
по професия 523050 „Техник на компютърни системи“
специалност 5230502 „Компютърни мрежи“

на ученика Никола Емилов Алексов от 12Г клас

1. Тема: Подсигуряване на динамичното обвързване MAC-IP и IP-MAC ниво комуникационен протокол

2. Изисквания:
   2.1. Да се проектира и разработи подсигурен и надежден протокол за обвързване на MAC-IP и IP-MAC в комуникационна инфраструктура.
   2.2. Разработката да се верифицира в емулирана комуникационна среда, като се използват предварително дефинирани сценарии.

3. Съдържание 3.1 Теоретична част
   3.2 Практическа част
   3.3 Приложение

Дипломант :.....
/ Никола Алексов /

Ръководител:.....
/ доц. д-р А. Цокев /

Консултант:.....
/ д-р А. Иванов /

ВРИД Директор:.....
/ ст. пр. д-р Веселка Христова /

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ СОФИЯ

ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

С Т А Н О В И Щ Е

КЪМ ДИПЛОМНА РАБОТА

Тема на дипломната работа: Подсигуряване на динамичното обвързване MAC-IP и IP-MAC на ниво комуникационен протокол

Ученик: Никола Емилов Алексов

Клас: 12^{Г}

Професия: код 523050 „Техник на компютърни системи“

Специалност: код 5230502 „Компютърни мрежи“

Дипломен ръководител: доц. д-р Александър Цокев

Дипломантът Никола Алексов се справи отлично с поставената му дипломна работа. По време на разработването ѝ той работи отговорно, последователно и професионално, като демонстрира задълбочени знания и практически умения. Самостоятелно идентифицира и решава възникналите проблеми, проявявайки аналитично мислене и инициативност. В процеса на реализация не само изпълни поставените изисквания, но и ги надхвърли, като предложи допълнителни функционалности и подобрения.

По мое мнение Никола притежава необходимите качества, мотивация и потенциал да се реализира като изключителен специалист в областта на комуникацията и киберсигурността.

Предлагам за рецензент: Петър Недялков Кирков, експерт по киберсигурност, email: pkirkov@gmail.com, телефон: 0886066744.

Всичко изложено дотук дава основание ученикът **Никола Алексов** да бъде допуснат до защита пред комисията за провеждане на държавен изпит за придобиване на професионална квалификация по теория и практика на специалността и комисията да оцени дипломната работа **отлично**.

28.01.2025 г. Дипломен ръководител:.....…….....

(доц. д-р А. Цокев)

Увод 8
Първа глава: Слабости и проблеми при ARP 11
1.1 Описание на ARP протокола 11
1.1.1 Функция на ARP 11
1.1.2. Видове ARP съобщения 12
1.1.3. Структура на ARP съобщението 13
1.1.4. Принцип на работа на ARP 14
1.1.5. Ограничения на ARP 17
1.2 Опасности и рискове за сигурността при ARP Spoofing 18
1.2.1 Атаки от тип Man-in-the-Middle (MITM) 19
1.2.2 Атаки от тип отказ от услуга (DoS) 20
1.2.3 Превземане на сесия (Session Hijacking) 21
1.2.4 Анализ на рисковете и последствията 22
1.2.5 Заключение 22
1.3 Съществуващи методи за подсигуряване на ARP и принципи на действие 23
1.3.1 Статични ARP записи (Static ARP Entries) 23
1.3.2 Динамична инспекция на ARP (Dynamic ARP Inspection/DAI) 23
1.3.3 DHCP Snooping 24
1.3.4 ARP Monitoring и IDS/IPS системи 25
1.3.5 VLAN сегментация 25
1.4 Заключение 26
Втора глава: Проектиране на T-ARP 28
2.1 T-ARP 28
2.2 AES-GCM и приложението му в T-ARP [4][5][6] 28
2.2.1 Основни понятия 28
2.2.2 Входни и изходни параметри 29
2.2.3 Диаграма на AES-GCM 30
2.2.4 Алгоритмичен процес 31

2.2.4.1. AES-GCM Encryption 31
2.2.4.2. AES-GCM Decryption 31
2.2.5. Формални дефиниции и операции в GF(2^{128}) 32
2.2.5.1. Крайно поле GF(2^{128}) 32
2.2.5.2. Операции в полето 33
2.2.5.3. Формална дефиниция на GHASH 33
2.2.6. Защо AES-GCM е избран за протокола T-ARP 34
2.2.6.1. AEAD свойства 34
2.2.6.2. Висока производителност 34
2.2.6.3. Поддръжка на AAD 34
2.3 Симетричен Message Authentication Code (MAC) 34
2.3.1. Въведение 34
2.3.2. Основна концепция и определение 35
2.3.3. Какви свойства осигурява симетричният MAC 36
2.3.4. Типичен процес на използване 36
2.3.5. Разлика между MAC и други криптографски механизми 37
2.3.6. Популярни реализации на симетричен MAC 37
2.3.7. Защита срещу атаки и ограничения 38
2.4 Принцип на работа на T-ARP 38
2.4.1 Структура на пакетите 38
2.4.1.1 Общ Ethernet формат на кадрите 39
2.4.1.2 Криптирана комуникация между крайно устройство и шлюз 39
2.4.1.2.1. Формат на AES-GCM съобщението 40
2.4.1.2.2. Authenticated Additional Data (AAD) 40
2.4.1.3 Пакет „Заявка за ключ“/”GET Request” (Type 1) 41
2.4.1.4. Пакет „Отговор с ключ“/”GET Reply” (Type 2) 42
2.4.1.5 T-ARP пакети между крайните устройства 42
2.4.1.5.1. T-ARP Broadcast Discover (Type 3) 43

2.4.1.5.2. T-ARP Unicast Reply (Type 4) 43
2.4.1.6. Обобщение на типовете пакети 44
2.5 T-ARP комуникация 44
2.6 Осъществяване на първоначална комуникация на устройство с централния шлюз (gateway, GW) 52
2.6.1 Принцип на работа 53
2.6.2 Стъпков процес 53
2.6.2.1 Първоначална конфигурация на шлюза 54
2.6.2.2 Добавяне на ново устройство към мрежата 54
2.6.2.3 Първоначален обмен на данни между крайното устройство и шлюза 54
2.7 T-ARP GW 56
2.7.1 VyOS 57
2.7.2 Конфигурация на протокола 57
2.7.3 Таблици с данни в T-ARP 57
2.7.3.1 Таблица с данни за устройствата и ключовете за комуникация с тях 58
2.7.3.2 Таблица със симетрични ключове за комуникация между две устройства 58
2.7.3.3 Таблица с метаданни 59
2.8 Заключение 59
Трета глава: Софтуерна разработка на T-ARP 61
3.1 T-ARP GW 61
3.1.1 Изграждане на Debian пакет 61
3.1.1.1 Структура на Debian пакет 61
3.1.1.2 Компилиране на Debian пакета 67
3.1.1.3 Подготовка на .deb файла за компилация с VyOS 67
3.1.2 Основен изпълним файл, отговарящ за функционалността на пакета 67
3.1.2.1 Използвани функции и библиотеки 67

3.1.2.2 Слушане за пакети 75
3.1.2.3 Декриптиране и верификация на пакета 76
3.1.2.4 Обработка на декриптирания пакет и изпращане на отговор 79
3.1.3 Компилиране на VyOS [7] 82
3.2 T-ARP симулационен скрипт на крайните устройства 83
3.2.1 Използвани библиотеки и зависимости 84
3.2.3 Конфигурационен файл 85
3.2.4 Помощни функции 86
3.2.5 Криптографска логика 87
3.2.6 Sniffer нишка 90
3.2.7 Заявяване на ключ от шлюза 92
3.2.8 Изпращане на T-ARP заявки (broadcast) 94
3.2.9 Обработка на входящи T-ARP заявки 95
3.2.10 Основна функция main() 97
Четвърта глава: Използвани ресурси за верификация на T-ARP в емулирана мрежова инфраструктура 99
4.1 Цел и обхват на верификацията 99
4.2 Избор на подход за тестване 100
4.2.1 Предимства на емулираната инфраструктура 100
4.2.2 GNS3 100
4.2.3 Избор на VyOS като платформа за интеграция 101
4.2.4 Използване на VMware за емулация на крайни устройства 101
4.2.5 Използване на Wireshark за анализ на мрежовия трафик 102
4.3 Архитектура на тестовата инфраструктура 102
4.3.1 Основни компоненти 102
4.3.1.1 T-ARP Gateway (VyOS) 102
4.3.1.2 Легитимни устройства 103
4.3.1.3 Виртуален суич 103
4.3.2 Логическа топология 103

Пета глава: Верифициране на работоспособността 104
5.1 Инициализация и конфигурация на GNS3 104
5.1.1 Инсталация на GNS3 [8] 104
5.1.2 Свързване на VMware виртуални машини към GNS3 107
5.2 Създаване и конфигурация на VMware виртуалните машини 112
5.2.1 Виртуални машини, използвани за крайни устройства в симулацията 112
5.2.2 Виртуална машина на VyOS 113
5.2.3 Конфигурация на vmnets (виртуални мрежи) във VMware 114
5.3 Наблюдаване на мрежовия трафик с Wireshark 117
5.4 Симулация на T-ARP 118
5.4.1 Създаване на GNS3 симулацията 118
5.4.2 Конфигурация на виртуалните машини след включване 120
5.4.2.1 Конфигурация на крайните устройства 120
5.4.2.2 Конфигурация на VyOS рутера 121
5.4.3 Тестване на работоспособността 123
5.4.4 Обобщение на получените резултати 130
Шеста глава: Заключение 131
Седма глава: Бъдещо развитие 134
7.1 Разширяване към IPv6 среда 134
7.2 Усъвършенстване на управлението на криптографските ключове 134
7.3 Имплементиране на механизъм за осъществяване на първоначална връзка с шлюза (GW) 135
7.4 Интеграция с хардуерни механизми за защита 135
7.5 Оптимизация на производителността 136
7.6 Разширяване на функционалността и интеграция в цялостна стратегия за сигурност 136
7.7 Потенциал за стандартизация 137
7.8 Обобщение 137

Източници 138

Увод

Съвременните компютърни мрежи разчитат на множество основополагащи протоколи, които осигуряват коректната комуникация между мрежовите и крайните устройства. Един от тях е протоколът за обвързване (резолюция) на адреси (ARP - Address Resolution Protocol), чиято основна функция е да осъществява връзката между логическия IP адрес и физическия MAC адрес в локалните мрежи. Въпреки своята ключова роля и широко разпространение, ARP е проектиран в ранните етапи на развитието на компютърните мрежи, когато сигурността не е била водещ приоритет. В резултат на това протоколът не включва механизми за автентикация и защита на обменяната информация, което го прави уязвим на различни типове атаки.

Една от най-сериозните слабости на ARP е възможността за ARP spoofing (ARP poisoning) - атака, при която злонамерен участник изпраща фалшиви ARP съобщения в мрежата с цел да асоциира своя MAC адрес с IP адреса на легитимно устройство, например шлюз (gateway) или сървър. Това позволява прихващане, подмяна или блокиране на трафик и често служи като основа за по-сложни атаки, като например “човек по средата” (Man-In-The-Middle - MITM), отказ от услуга (Denial of Service - DoS) или кражба на чувствителна информация, обменяна по мрежата. Проблемът се задълбочава от факта, че ARP работи на доверителен принцип, т.е. всяко устройство в мрежата приема ARP отговорите без да проверява техния произход или автентичност.

Съществуват различни подходи за ограничаване и предотвратяване на ARP атаките, като статични ARP таблици, Dynamic ARP Inspection (DAI), използване на VLAN сегментация или внедряване на допълнителни защитни

механизми в мрежовото оборудване. Въпреки това тези решения често са зависими от конкретни производители, изискват специализирано хардуерно оборудване или са трудни за мащабиране и поддръжка. Освен това те не решават фундаменталния проблем - липсата на вградена сигурност в самия ARP протокол.

В научната литература и изследователската общност са предложени редица теоретични модели и концепции за подсигурени версии на ARP, базирани на криптографски методи, цифрови подписи или централизирани удостоверителни услуги. Въпреки това, към настоящия момент липсват широко разпространени и практически приложими реализации, които да бъдат интегрирани лесно в реални или тестови мрежови среди. Това създава необходимост от разработване и експериментално валидиране на работещо решение, което да съчетава сигурност, надеждност и приложимост.

Настоящата дипломна работа е насочена именно към този проблем.

Целта на дипломната работа е да се проектира, разработи и верифицира нов мрежови протокол с име T-ARP, който представлява подсигурен и надежден протокол за обвързване на MAC-IP и IP-MAC (подсигурена версия на протокола ARP), базиран на надеждни криптографски алгоритми и системи. Работоспособността на разработения протокол се проверява в емулирана мрежова инфраструктура.

За постигането на тази цел в дипломната работа се разглеждат съществуващите слабости на стандартния ARP протокол, анализират се възможностите за интегриране на криптографски механизми за защита и се предлага архитектура на подсигурен протокол. Разработеното решение се тества и анализира в контролирана емулирана среда, което позволява да се

оцени неговата ефективност, надеждност и потенциал за бъдещо развитие и практическо приложение. Важно е да се отбележи, че **разработката е ограничена единствено за IPv4.**

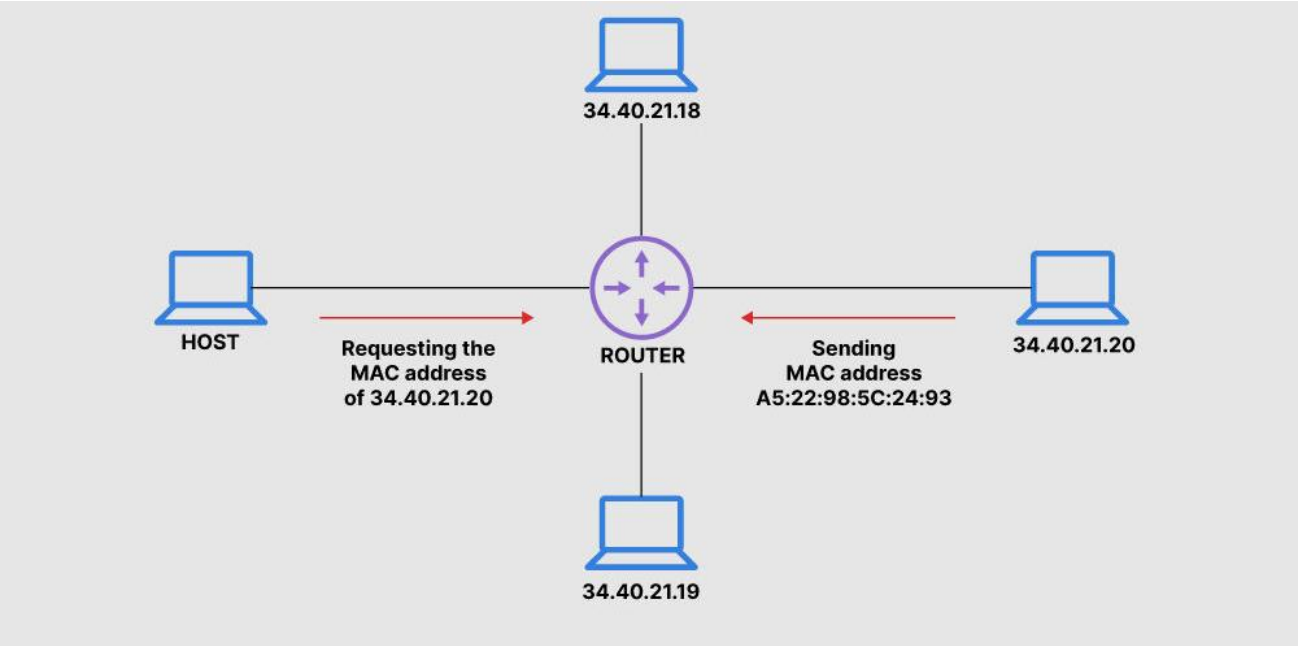
Първа глава: Слабости и проблеми при ARP

1.1 Описание на ARP протокола

Address Resolution Protocol (ARP) е мрежов протокол от слой 2/3 (канален и мрежов слой) на модела OSI, който се използва за обвързване на IP адреси към MAC адреси в локални мрежи. ARP е ключов компонент на IPv4 мрежите, тъй като позволява на устройствата да откриват физическите адреси на други устройства в същата мрежа и да осъществяват коректна доставка на данните на ниво Ethernet.

1.1.1 Функция на ARP

Основната функция на ARP е да осигури обвързването между логически (IP) и физически (MAC) адреси. Когато дадено устройство иска да изпрати IP пакет към друго устройство в локалната мрежа, то проверява своята ARP таблица (ARP cache), за да установи дали вече има запис за желания IP адрес. Ако такъв запис съществува, устройството използва своята MAC адресна таблица, за да изпрати пакета на правилния адрес. Ако записът не съществува, устройството изпраща ARP заявка (ARP Request) като broadcast в мрежата (фиг. 1.1), за да открие MAC адреса, който съответства на дадения IP адрес. Устройството, което притежава IP адреса, отговаря с ARP отговор (ARP Reply), който съдържа неговия MAC адрес. Този запис се кешира за определено време.



Фиг. 1.1 Работа на ARP [1]

1.1.2. Видове ARP съобщения

ARP протоколът използва два основни типа съобщения:

1. ARP Request (Заявка) - изпраща се до всички устройства в локалната мрежа (broadcast), с цел да се открие MAC адреса, който съответства на даден IP адрес.
2. ARP Reply (Отговор) - изпраща се като unicast към устройството, което е изпратило заявката, и съдържа необходимия MAC адрес.

Допълнително се използват и gratuitous ARP съобщения, при които устройство изпраща ARP отговор без той да е поискан, за да информира мрежата за промяна на собствените си адреси или за предотвратяване на конфликти на IP адреси. Именно този тип съобщения се използват при атака от тип ARP Spoofing.

1.1.3. Структура на ARP съобщението

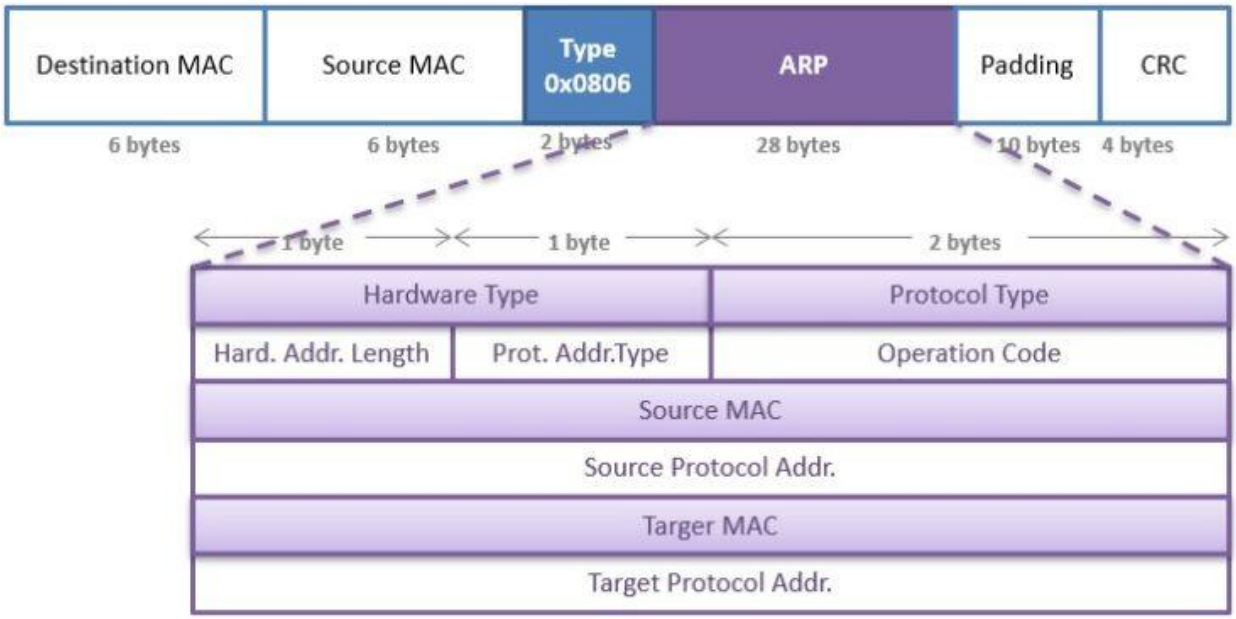
ARP пакетът се състои от Ethernet Frame Header с дължина 14 байта и ARP Header с дължина 28 байта, като в последния се намира и информацията за ARP (Address Resolution Protocol).

В ARP пакета EtherType на Ethernet пакета е 0x0806. Останалата част от заглавната част на Ethernet рамката е подобна на другите Ethernet пакети.

От своя страна ARP Header съдържа различни полета, които са описани в детайли по-долу. [2]

Основните полета в ARP пакета са (фиг. 1.2):

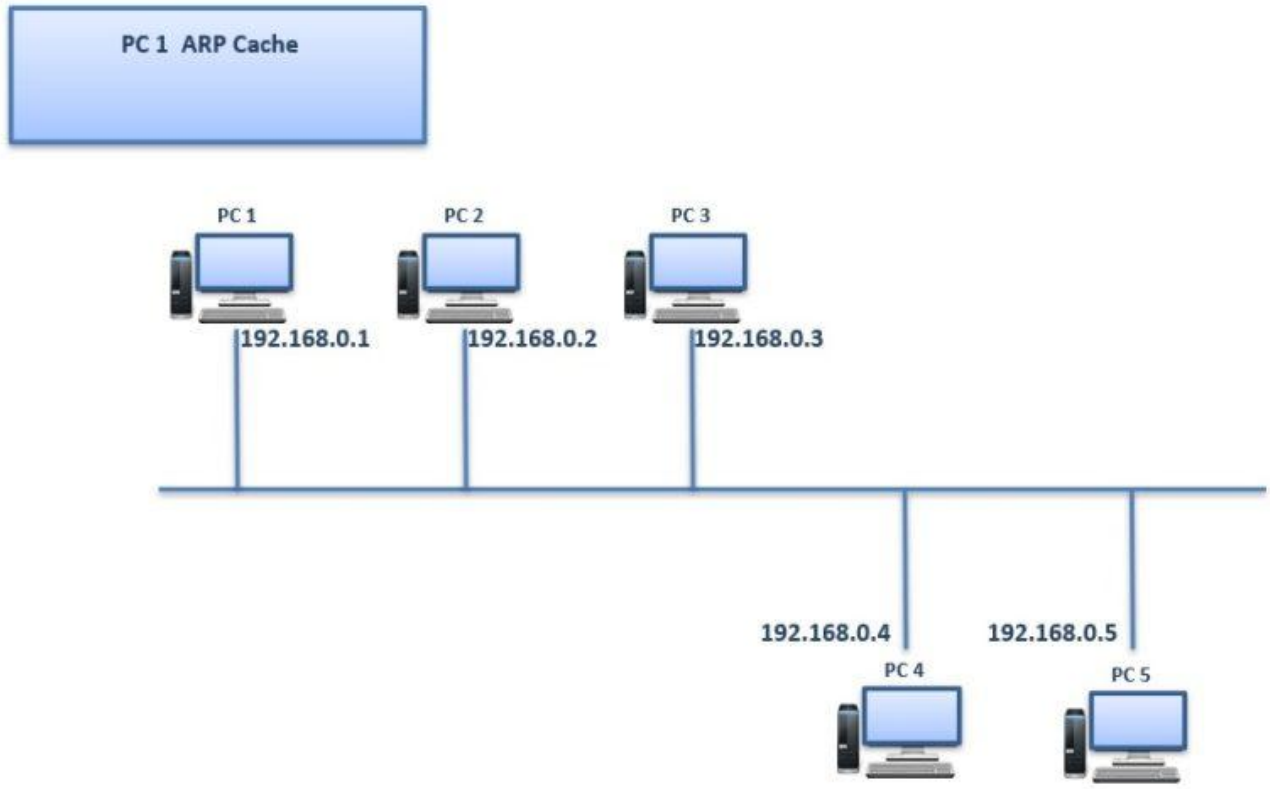
- Hardware Type - тип на мрежовото устройство (например Ethernet).
- Protocol Type - тип на протокола на мрежовия слой (IPv4). За ARP е 0x0806.
- Hardware Address Length - дължина на MAC адреса.
- Protocol Address Length - дължина на IP адреса.
- Operation - вид на съобщението (заявка или отговор).
- Sender Hardware Address - MAC адрес на подателя.
- Sender Protocol Address - IP адрес на подателя.
- Target Hardware Address - MAC адрес на получателя (празно при заявка).
- Target Protocol Address - IP адрес на получателя.



Фиг. 1.2 Структура на ARP Header [2]

1.1.4. Принцип на работа на ARP

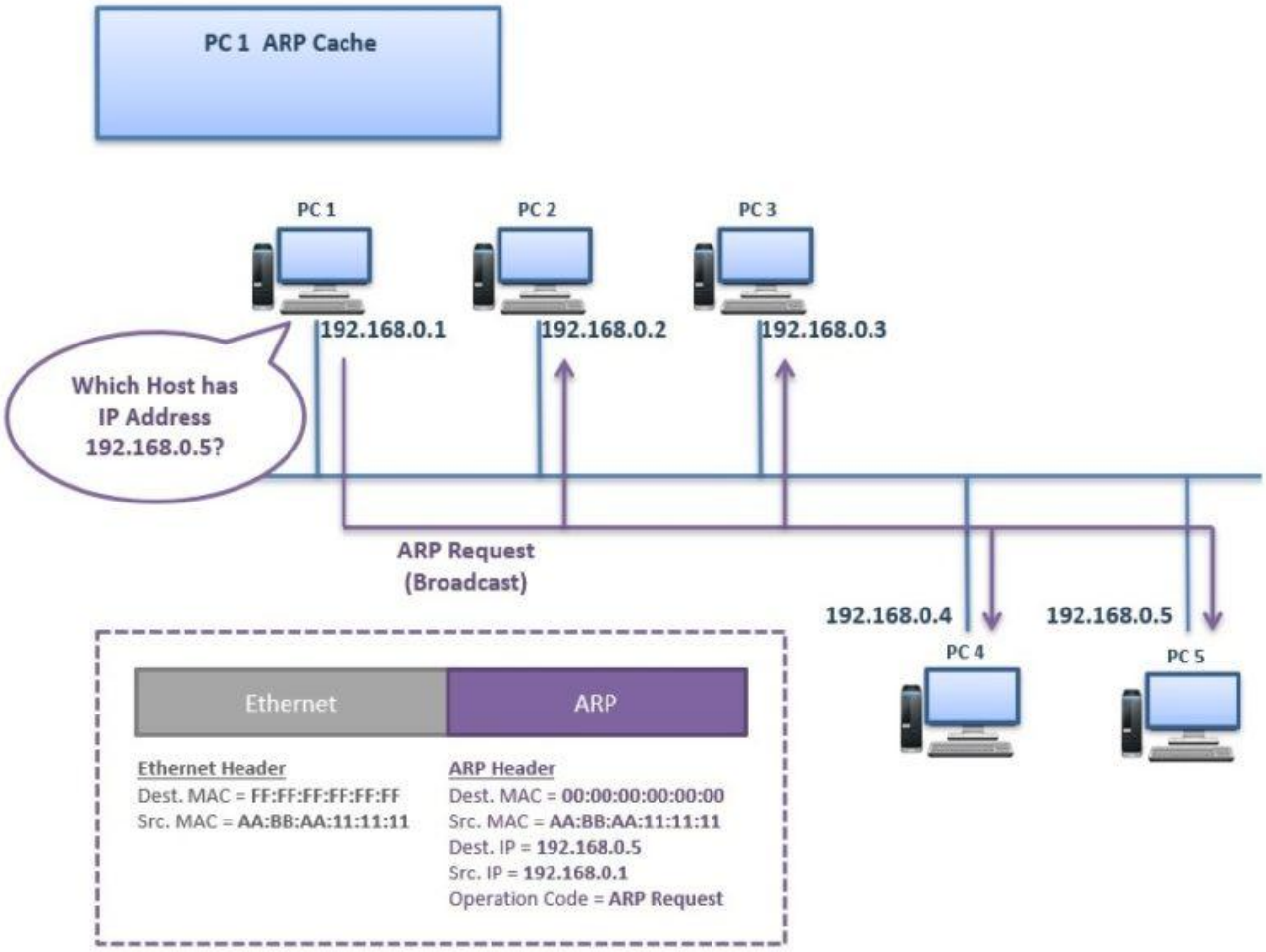
За обяснението ще се използва примерна топология, представена на фигура 1.3.



Фиг. 1.3 Примерна топология [2]

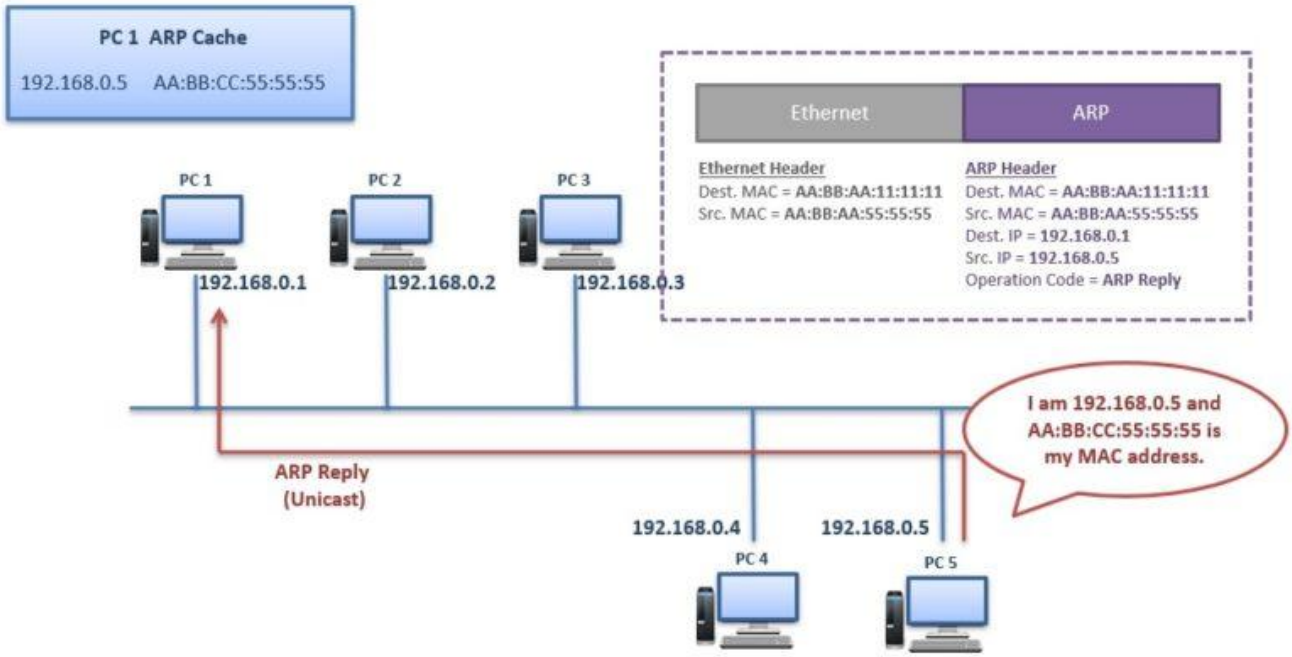
PC 1 иска да изпрати пакет до PC 5 и проверява локалната ARP таблица за съответствие между търсения IP адрес и MAC адрес.

Тъй като няма валиден запис за търсения IP адрес, PC 1 изпраща ARP заявка. (фиг. 1.4)



Фиг. 1.4 PC 1 изпраща широковещателна (broadcast) заявка [2]

Целевото устройство отговаря с ARP отговор, съдържащ неговия MAC адрес (фиг. 1.5).



Фиг. 1.5 PC 5 изпраща едноадресен (unicast) пакет към PC 1

PC 1 кешира получената информация за определен период от време (обикновено няколко минути) (фиг. 1.5).

Пакетът се предава на целевия MAC адрес.

1.1.5. Ограничения на ARP

Няколко от основните ограничения на ARP протокола са:

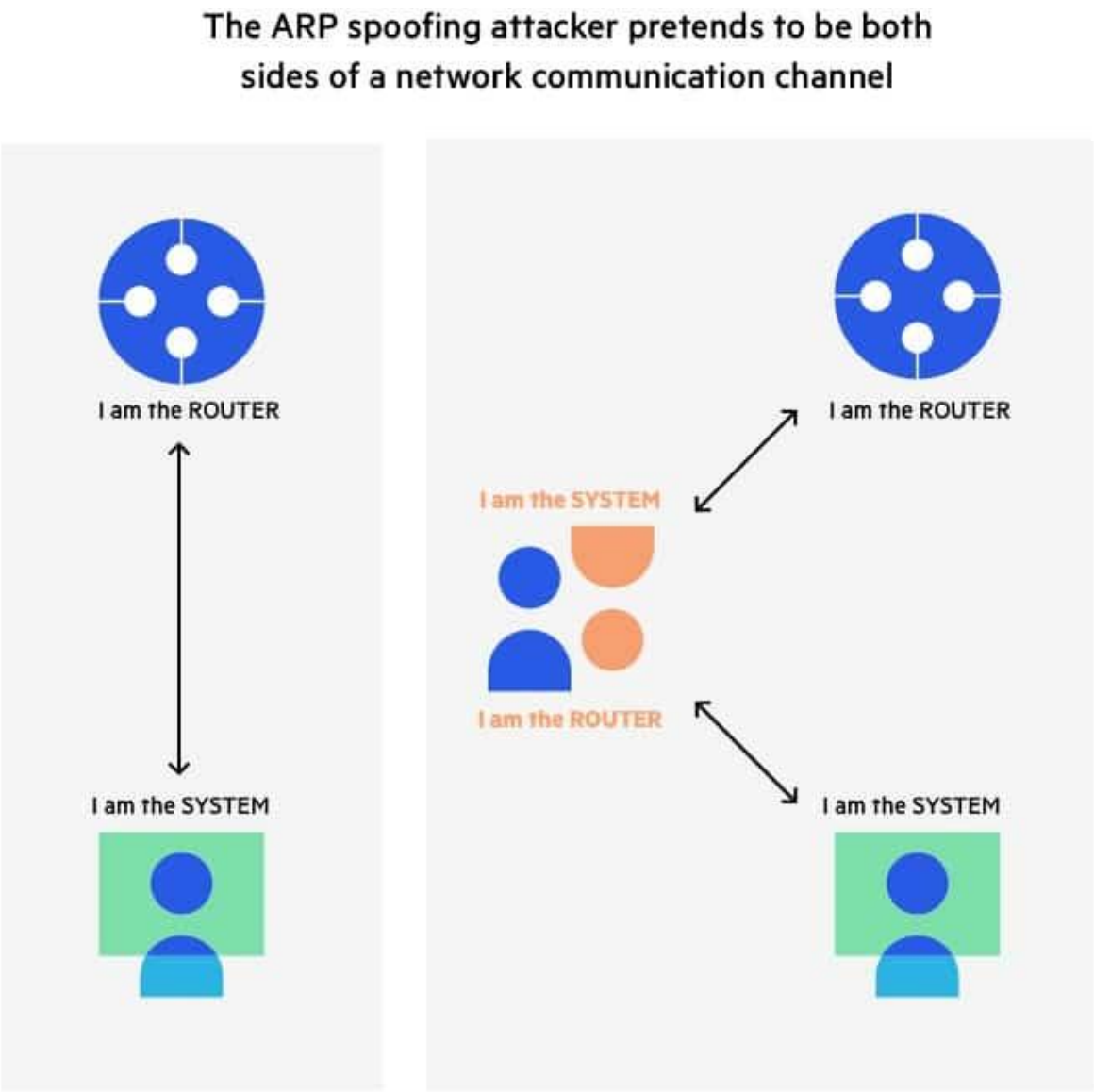
- Липса на автентикация - всяко устройство може да изпраща ARP отговори.
- Липса на криптиране и защита на целостта на данните.
- Податливост на ARP spoofing и ARP poisoning атаки.
- Ограничена приложимост в големи и критични мрежи без допълнителни механизми за защита.

1.2 Опасности и рискове за сигурността при ARP Spoofing

ARP spoofing, наричан още ARP poisoning или подмяна на ARP кеша, представлява атака на ниво локална мрежа на слой 2/3 (канален и мрежов слой), при която атакуващото устройство изпраща фалшиви ARP съобщения с цел манипулиране на ARP таблиците на останалите устройства в мрежата. Основният принцип на атаката се базира на факта, че стандартният ARP протокол не извършва проверка за автентичност и приема всички ARP отговори като легитимни.

В нормален режим на работа ARP таблицата съдържа временни записи, които асоциират IP адреси с MAC адреси. При ARP spoofing атакуващият периодично изпраща фалшиви ARP Reply пакети, в които декларира, че IP адресът на легитимно устройство (например default gateway) принадлежи на неговия MAC адрес. Поради липсата на механизми за валидация, жертвите автоматично актуализират своите ARP кешове с тази подменена информация.

В резултат на това целият мрежов трафик, предназначен за легитимното устройство, се пренасочва към атакуващия (фиг. 1.6), който може да го анализира, модифицира или изцяло да го блокира, което често служи като основа за реализиране на други, по-сложни кибератаки.



Фиг. 1.6 Пример за резултат от ARP spoofing атака [3]

1.2.1 Атаки от тип Man-in-the-Middle (MITM)

Man-in-the-Middle (MITM) атаката е една от най-опасните последици от ARP spoofing. При този сценарий атакуващият се позиционира логически

между две комуникиращи страни (например клиент и шлюз), без те да са наясно с това. ARP spoofing се използва като механизъм за пренасочване на трафика, след което атакуващият функционира като прозрачен посредник.

Технически погледнато, атакуващият приема входящите пакети от жертвата, обработва ги (например чрез packet sniffing или packet modification) и след това ги препраща към реалния получател. Това позволява:

- Пасивно прихващане на трафик (sniffing);
- Активна подмяна или инжектиране на пакети;
- Манипулиране на TCP сесии и приложни данни.

Дори когато се използват защитени протоколи като HTTPS, атакуващият може да прилага техники като SSL stripping, при които криптираната връзка се понижава до некриптирана, без потребителят да забележи това. MITM атаките са изключително трудни за откриване, тъй като комуникацията между страните продължава да функционира нормално.

1.2.2 Атаки от тип отказ от услуга (DoS)

ARP spoofing може да бъде използван и като средство за реализиране на Denial-of-Service (DoS) злонамерени действия. В този случай атакуващият не препраща получените пакети към реалния получател, а умишлено ги изтрива. Това води до прекъсване на комуникацията между устройствата в локалната мрежа.

Технически, DoS атака чрез ARP spoofing може да се осъществи чрез:

- Подмяна на ARP записите за default gateway;
- Пренасочване на трафика към несъществуващ MAC адрес;
- Генериране на голям обем фалшиви ARP пакети, което води до нестабилност на ARP кеша на системите.

В по-големи мрежи подобни атаки могат да доведат до значително увеличаване на broadcast трафика и натоварване на мрежовите устройства, което допълнително влошава качеството на услугите.

1.2.3 Превземане на сесия (Session Hijacking)

Session hijacking е атака, при която атакуващият използва ARP spoofing като средство за прихващане на идентификатори на активни потребителски сесии. След като трафикът бъде пренасочен, атакуващият може да анализира HTTP/HTTPS пакети и да извлече session ID стойности, които обикновено се съхраняват в cookies.

След получаване на session ID, атакуващият може да:

- Се представи за легитимен потребител;
- Извършва операции от името на жертвата;
- Получи достъп до защитени ресурси без необходимост от повторно удостоверяване.

Този тип атаки са особено опасни в уеб-базирани приложения, корпоративни портали и системи за управление, където сесиите имат продължителен живот и високо ниво на доверие.

1.2.4 Анализ на рисковете и последствията

ARP spoofing представлява сериозна заплаха за сигурността на локалните мрежи, тъй като атаките се извършват на ниско ниво от мрежовия модел и не изискват експлоатиране на уязвимости в операционни системи или приложения. Достатъчно е наличието на един компрометирано устройство в мрежата, за да се наруши цялостната комуникация.

Основните рискове включват:

- Компрометиране на конфиденциалността на данните.
- Нарушаване на целостта на комуникацията.
- Загуба на наличността на мрежовите услуги.
- Пълна загуба на доверие в локалната мрежова инфраструктура.

1.2.5 Заключение

ARP spoofing ясно демонстрира фундаменталните архитектурни слабости на стандартния ARP протокол. Липсата на автентикация, криптографска защита и механизми за контрол на достъпа прави ARP не напълно подходящ за използване в среди с високи изисквания за сигурност. Това налага необходимостта от разработване на подсигурени механизми за

обвързване на MAC и IP адреси, които да елиминират възможността за подобни атаки и да гарантират надеждна мрежова комуникация.

1.3 Съществуващи методи за подсигуряване на ARP и принципи на действие

1.3.1 Статични ARP записи (Static ARP Entries)

Статичните ARP записи представляват ръчно конфигурирани записи в ARP таблицата, които фиксират обвързването между IP и MAC адрес. По своята същност те предотвратяват подмяната на ARP записи, тъй като статичните записи не могат да бъдат автоматично променяни.

Статичните ARP записи работят ефективно в малки и статични мрежи с ограничен брой устройства.

Недостатъци:

- Трудни за администриране и поддръжка.
- Непрактични за динамични и мащабируеми мрежи.
- Висок риск от грешки при ръчна конфигурация.

1.3.2 Динамична инспекция на ARP (Dynamic ARP Inspection/DAI)

Dynamic ARP Inspection е механизъм за защита, внедрен в управляеми мрежови комутатори.

Работи чрез инспекция на всички ARP пакети в мрежата, като сравнява съдържанието на ARP съобщенията с доверена база от IP адрес - MAC адрес асоциации, обикновено генерирана от DHCP snooping.

Ако ARP пакетът не съответства на записите в базата, той се блокира.

Предимства:

- Ефективна защита срещу ARP spoofing.
- Автоматично управление на IP адрес - MAC адрес съответствията.

Недостатъци:

- Изисква специфично мрежово оборудване.
- Зависимост от коректно конфигуриран DHCP сървър.
- Ограничена гъвкавост при нестандартни сценарии.

1.3.3 DHCP Snooping

DHCP Snooping е механизъм за контрол и мониторинг на DHCP трафика в локална мрежа.

Този подход разделя портовете на комутатора на доверени и недоверени.

Той изгражда база данни с валидни IP адрес - MAC адрес асоциации въз основа на DHCP процеса.

Използва се като основа за други защитни механизми, като например DAI.

Ограничения:

- Работи само в DHCP-базирани мрежи.
- Не защитава директно ARP, а индиректно подпомага защитата.

1.3.4 ARP Monitoring и IDS/IPS системи

Използват се системи за мониторинг и откриване на аномалии в ARP трафика.

Анализират честотата, съдържанието и поведението на ARP съобщенията.

Засичат подозрителни събития като:

- Честа промяна на ARP записи.
- Дублиране на IP адреси.
- Неочаквани gratuitous ARP пакети.

Недостатъци:

- Реактивен подход - атаката вече е започнала.
- Висок брой фалшиви аларми/сигнали при динамични мрежи.

1.3.5 VLAN сегментация

VLAN сегментацията разделя мрежата на логически сегменти, като ограничава обхвата на ARP broadcast трафика и намалява броя на потенциалните жертви на ARP атаки.

Недостатъци:

- Не предотвратява ARP атаки в рамките на един VLAN.
- Представлява мярка за ограничаване на ARP атаките, а не реална защита от тях.

1.4 Заключение

В първа глава бяха разгледани основните принципи на работа на протокола ARP, неговата роля в IPv4 мрежите и критичните архитектурни слабости, произтичащи от липсата на вградени механизми за сигурност. Анализът показа, че ARP разчита изцяло на доверителен модел на комуникация, без автентикация и защита на целостта на данните, което го прави изключително уязвим на атаки от тип ARP spoofing.

Бяха представени основните рискове и последствия от подобни атаки, включително реализирането на Man-in-the-Middle атаки, отказ от услуга и превземане на потребителски сесии, които могат да доведат до сериозни компромиси в конфиденциалността, целостта и наличността на мрежовите услуги. Стана ясно, че дори една единствена компрометирана крайна точка може да застраши сигурността на цялата локална мрежа.

Разгледаните съществуващи методи за защита - като статични ARP записи, Dynamic ARP Inspection, DHCP snooping, мониторинг на ARP трафика и VLAN сегментация - предлагат частични решения на проблема, но не елиминират фундаменталната уязвимост на протокола. Повечето от тях са ограничени по мащабируемост, зависими са от конкретно оборудване или представляват по-скоро реактивни, отколкото превантивни мерки.

Направеният анализ обосновава необходимостта от разработване на подсигурен протокол за обвързване на MAC и IP адреси, който да интегрира криптографски механизми и да осигури надеждна защита срещу подмяна и манипулация на ARP информацията. Тези изводи служат като теоретична основа за следващите глави, в които е представена концепцията,

архитектурата и практическата реализация на предложеното подсигурено решение.

Втора глава: Проектиране на T-ARP

2.1 T-ARP

T-ARP (Trusted Address Resolution Protocol) представлява защитено разширение на класическия Address Resolution Protocol (ARP) и е проектиран с цел елиминиране на описаните в Глава 1 уязвимости, като ARP spoofing и Man-in-the-Middle атаки, които бяха описани в Глава 1.

T-ARP работи на слой 2 / 2.5 от OSI модела и използва сурови Ethernet кадри за пренос на данни, което го прави независим от IP стека.

Използват се **AES-GCM** и **симетричен MAC** като основни методи за подсигуряване на интегритета и достоверността на данните в пакетите.

2.2 AES-GCM и приложението му в T-ARP [4][5][6]

В тази част от Глава 2 се описва AES-GCM с математически дефиниции и се аргументира избора на AES-GCM в протокола T-ARP.

2.2.1 Основни понятия

AES (Advanced Encryption Standard) е блоков шифър с фиксиран размер на блока от 128 бита и дължини на ключа 128, 192 или 256 бита. AES сам по себе си реализира само конфиденциалност, но не гарантира интегритет или автентичност на данните.

GCM (Galois/Counter Mode) е режим на работа, който комбинира:

- **CTR (Counter) режим** за криптиране - превръща блоковия шифър в поточен шифър.
- **GHASH** - универсална хеш функция върху полето $GF(2^{128})$ за изчисляване на аутентификационна стойност (tag).

Това позволява на AES-GCM да реализира AEAD (Authenticated Encryption with Associated Data) - аутентифицирано шифроване с допълнително аутентикирани данни.

2.2.2 Входни и изходни параметри

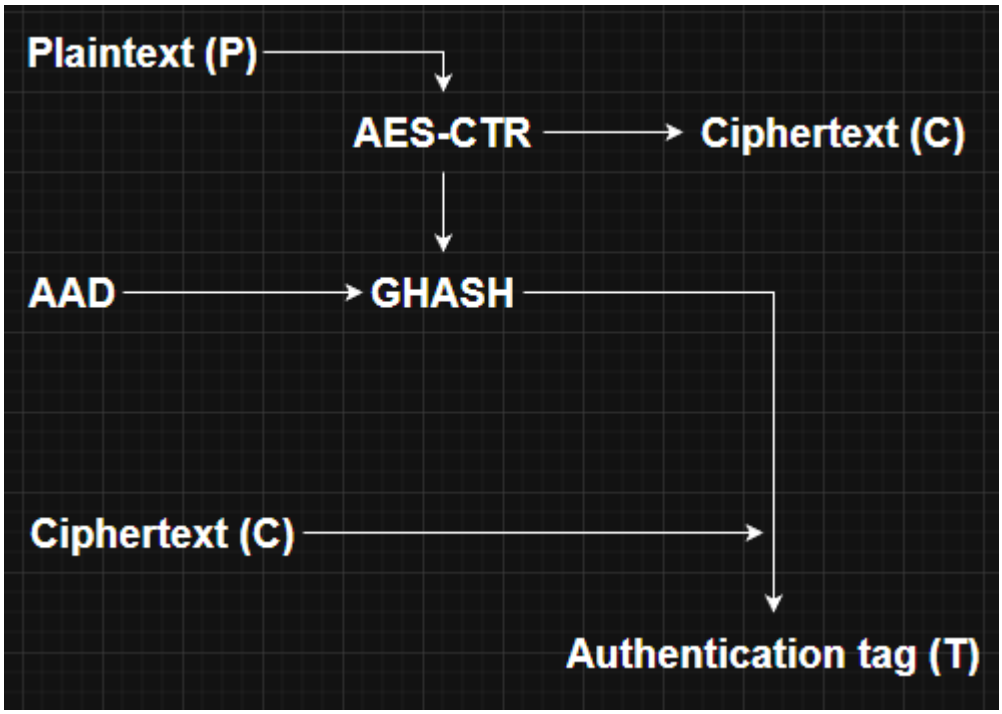
AES-GCM има следните входни и изходни параметри:

Входни параметри	
Параметър	**Описание**
Key (K)	Симетричен таен ключ (128/192/256 бита)
Nonce/IV (N)	Инициализиращ вектор (96 бита)
Plaintext (P)	Данни за криптиране/явен текст (променлива дължина)
AAD	Допълнителни автентикационни данни

Изходни параметри	
Параметър	**Описание**
Ciphertext (C)	Криптиран текст (криптограма) със същата дължина като P
Authentication tag (T)	128-битова стойност за проверка на интегритет/автентичност

Чрез тези параметри получателят може да провери интегритета и автентичността на данните преди да разшифрова самия криптиран текст.

2.2.3 Диаграма на AES-GCM



- GCM използва AES-CTR за генериране на keystream (потоци от 128 бита).

- Тези стойности се XOR-ват с plaintext, за да се получи ciphertext.
- GHASH функцията изчислява tag чрез умножения в полето $GF(2^{128})$ върху AAD и C.

2.2.4 Алгоритмичен процес

2.2.4.1. AES-GCM Encryption

1. Input: Key K, Nonce N, Plaintext P, AAD
2. Compute Hash subkey H = AES_K(0^128)
3. Derive initial Counter block = Inc(N || 0^31 || 1)
4. For each block i of P:
   keystream_i = AES_K(Counter_i)
   Ciphertext_i = Plaintext_i XOR keystream_i
   Counter_i = Counter_i + 1
5. Compute Tag via GHASH(H, AAD, Ciphertext)
6. Output: Ciphertext, Tag

- Хеш ключ H се генерира от AES върху всички нули - използва се за GHASH.
- Counter се инкрементира за всеки блок - това гарантира уникален keystream.
- GHASH обединява AAD и ciphertext, за да генерира таг, който защитава целия пакет от промени.

2.2.4.2. AES-GCM Decryption

1. Input: Key K, Nonce N, Ciphertext C, AAD, Tag
2. Recompute Hash subkey H = AES_K(0^128)

3. Perform same Counter generation as Encryption
4. For each block i of C:
   keystream_i = AES_K(Counter_i)
   Plaintext_i = Ciphertext_i XOR keystream_i
5. Compute expected Tag via GHASH(H, AAD, Ciphertext)
6. Compare expected Tag == Tag?
   If equal: accept plaintext
   Else: reject (authenticity failure)

Алгоритъмът гарантира, че никакво дешифриране не се извършва преди тагът да бъде проверен успешно, тъй като при несъответствие се сигнализира за нарушена автентичност.

2.2.5. Формални дефиниции и операции в $GF(2^{128})$

2.2.5.1. Крайно поле $GF(2^{128})$

AES-GCM използва крайното поле $GF(2^{128})$, дефинирано като:

$$\mathbf{GF(2^{128})=GF(2)[x]/(x^{128}+x^7+x^2+x+1)}$$

Всеки елемент в полето се представя като 128-битов вектор:

$$\mathbf{a=(a_{127},a_{126},\ldots,a_0),\ a_i\in\{0,1\}}$$

2.2.5.2. Операции в полето

Събиране - побитово XOR:

$$a+b=a \oplus b$$

Умножение - полиномиално умножение по модул на неприводимия полином:

$$(x^{128}+x^{7}+x^{2}+x+1)$$

Тези операции се използват в алгоритъма GHASH за автентикация.

2.2.5.3. Формална дефиниция на GHASH

Нека:

- ***H*** - хеш подключ (Hash Subkey).
- $X_1, X_2, \ldots, X_n$ - 128-битови блокове от AAD и ciphertext.

Тогава GHASH се дефинира рекурсивно:

$$Y_0=0^{128}$$

$$Y_i=(Y_{i-1} \oplus X_i)\cdot H \text{ в } GF(2^{128})$$

Крайната стойност Y_n се използва при формирането на автентикационния таг.

2.2.6. Защо AES-GCM е избран за протокола T-ARP

Изборът на AES-GCM като криптографски механизъм в T-ARP е резултат от изброените по-долу инженерни и криптографски съображения.

2.2.6.1. AEAD свойства

AES-GCM е AEAD алгоритъм, което означава, че осигурява едновременно конфиденциалност, интегритет и автентичност, което е критично за T-ARP, тъй като протоколът работи на слой 2 и няма допълнителни защитни механизми от по-горните слоеве.

2.2.6.2. Висока производителност

AES-GCM позволява паралелна обработка, има хардуерно ускорение (AES-NI) и не изисква padding, което го прави подходящ за чест обмен на кратки пакети, каквито са T-ARP съобщенията.

2.2.6.3. Поддръжка на AAD

В T-ARP AAD се използва за автентикация на Ethernet заглавието (MAC адреси + EtherType), което се използва за предотвратяване на MAC spoofing и допълнително криптографски обвързва съдържанието с L2 контекста.

2.3 Симетричен Message Authentication Code (MAC)

2.3.1. Въведение

Симетричният Message Authentication Code (MAC) представлява криптографски механизъм, използван за осигуряване на целостта и автентичността на съобщения, предавани по несигурен комуникационен

канал. За разлика от криптографските алгоритми за шифроване, MAC не предоставя конфиденциалност, а се фокусира върху защитата срещу подмяна и неоторизирано генериране на съобщения.

MAC алгоритмите се използват широко в мрежови протоколи, системи за удостоверяване и защитени комуникации, където е необходимо получателят да бъде сигурен, че съобщението е изпратено от легитимен източник и не е било модифицирано по време на преноса.

2.3.2. Основна концепция и определение

Симетричният MAC се базира на споделен таен ключ, който е известен единствено на легитимния подател и получател. При изпращане на съобщение подателят изчислява MAC стойност върху съобщението, използвайки този ключ, и я добавя към предаваните данни. Получателят извършва същото изчисление и сравнява резултата с получения MAC.

Формално, MAC може да бъде дефиниран като функция:

$$\mathbf{MAC : \{0,1\}^k \times \{0,1\}^* \rightarrow \{0,1\}^n}$$

където:

- k е дължината на симетричния ключ
- входът е произволно дълго съобщение
- изходът е MAC стойност с фиксирана дължина n

Сигурността на MAC се основава на предположението, че атакуващият не разполага със споделения ключ и следователно не може да изчисли валидна MAC стойност за ново или модифицирано съобщение.

2.3.3. Какви свойства осигурява симетричният MAC

Използването на MAC предоставя следните основни криптографски свойства:

- **Цялост** (Integrity) - всяка промяна в съдържанието на съобщението води до различна MAC стойност.
- **Автентичност** (Authenticity) - само участник, който притежава тайния ключ, може да генерира валиден MAC.
- **Неотказуемост** в рамките на симетричен модел - получателят може да бъде сигурен в източника на съобщението, макар че това не осигурява юридическа неотказуемост (както при цифровите подписи).

Важно е да се подчертае, че MAC не осигурява поверителност - съдържанието на съобщението остава четимо за всеки, който има достъп до комуникационния канал.

2.3.4. Типичен процес на използване

Процесът на използване на симетричен MAC може да бъде описан в следните логически стъпки:

1. Подателят и получателят споделят предварително установен таен ключ.
2. Подателят изчислява MAC стойност върху съобщението.
3. Съобщението и MAC стойността се изпращат заедно.

4. Получателят пресмята MAC върху полученото съобщение, използвайки същия таен ключ.
5. Ако двете MAC стойности съвпадат, съобщението се приема като автентично и непроменено.
6. В противен случай съобщението се счита за компрометирано и се отхвърля.

2.3.5. Разлика между MAC и други криптографски механизми

Симетричният MAC често се бърка с други понятия, поради сходството в терминологията. Основните разграничения са следните:

- **MAC** срещу **хеш функция**: хеш функциите не използват таен ключ и не осигуряват автентичност.
- **MAC** срещу **цифров подпис**: цифровите подписи използват асиметрична криптография и осигуряват неотказуемост.
- **MAC** срещу **шифроване**: шифроването защитава съдържанието, докато MAC защитава целостта и произхода.

В много съвременни протоколи MAC и шифроване се комбинират чрез схеми за автентифицирано шифроване (AEAD), като AES-GCM.

2.3.6. Популярни реализации на симетричен MAC

На практика симетричният MAC се реализира чрез различни конструкции, сред които най-разпространени са:

- **HMAC** - базиран на криптографски хеш функции (SHA-256, SHA-3).
- **CMAC** - базиран на блокови шифри, най-често AES.
- **GMAC** - автентикационният компонент на AES-GCM.

- **CBC-MAC** - историческа конструкция с ограничена приложимост.

Тези алгоритми се различават по производителност, сигурност и приложна област, но всички следват основния принцип на симетричния MAC.

2.3.7. Защита срещу атаки и ограничения

Симетричният MAC защитава ефективно срещу:

- Подмяна на съдържанието на съобщения.
- Инжектиране на фалшиви съобщения.
- Имперсонация на легитимни участници.

Въпреки това, съществуват важни ограничения:

- Сигурността зависи изцяло от опазването на тайния ключ.
- Компрометиране на ключа компрометира всички съобщения.

2.4 Принцип на работа на T-ARP

2.4.1 Структура на пакетите

T-ARP протокола изолзва 4 вида пакети за осъществяване на комуникацията. За комуникация между крайни устройства и GW се използват пакети от тип T-ARP GET Request/Reply, а за комуникацията между две крайни устройства се използват пакети от тип T-ARP DISCOVER Request/Reply.

2.4.1.1 Общ Ethernet формат на кадрите

Всички пакети в T-ARP се енкапсулират в стандартни Ethernet кадри. Общата структура на Ethernet рамката е еднаква за всички типове съобщения и е показана на фигура 2.1.

Destination MAC (6 bytes)	Source MAC (6 bytes)	EtherType (2 bytes)
Payload (variable)		

Фиг. 2.1 Общ формат на Ethernet кадър в T-ARP

Полето EtherType има фиксирана стойност 0x88B5, която служи за еднозначна идентификация на T-ARP трафика. Използван е EtherType 0x88b5, тъй катое заделен от IEEE като локален експериментален тип и стойността му не съвпада с тази на вече съществуващи протоколи. MAC адресите на подателя и получателя винаги се предават в некриптиран вид, тъй като те са необходими за маршрутизацията на кадрите на канално ниво.

Важно проектно решение е използването на MAC адресите като част от криптографската автентикация (чрез AAD), без те да бъдат криптирани. По този начин се постига баланс между съвместимост с Ethernet и повишена сигурност.

2.4.1.2 Криптирана комуникация между крайно устройство и шлюз

Комуникацията между крайните устройства и доверения шлюз (Gateway) използва симетрично криптиране с алгоритъма AES-GCM (Advanced Encryption Standard - Galois/Counter Mode). Този режим осигурява едновременно:

- конфиденциалност на данните;
- целостта на съобщенията;
- автентикация на подателя.

2.4.1.2.1. Формат на AES-GCM съобщението

Всеки криптиран пакет съдържа структурата на AES-GCM (AES-GCM message format), показана на фигура 2.2.

Nonce (12 bytes)	Type (1 byte)	Ciphertext (variable)	Auth Tag (16 bytes)

Фиг. 2.2 AES-GCM плик в T-ARP

Nonce стойността е случайна и уникална за всеки пакет, което е критично изискване за сигурността на GCM режима. Автентикационният таг гарантира, че съдържанието на пакета не е било модифицирано.

2.4.1.2.2. Authenticated Additional Data (AAD)

Като допълнителни автентикирани данни (AAD) се използват следните полета от Ethernet заглавието (фиг. 2.3):

Поле	**Дължина (bytes)**
Source MAC	6
Destination MAC	6
EtherType	2
Общо	**14**

Фиг. 2.3 Състав на AAD в T-ARP

Използването на AAD криптографски обвързва съдържанието на пакета с конкретните MAC адреси и протокола, като по този начин се предотвратява подмяна на Ethernet хедъра.

2.4.1.3 Пакет „Заявка за ключ“/”GET Request” (Type 1)

Пакетите от тип 1 се изпращат от дадено устройство към шлюза с цел заявяване на симетричен ключ за комуникация с друго устройство.

Съдържанието на T-ARP пакет от тип 1 спазва следната структура, показана на фигура 2.4:

Type (1 byte)	Sender IP (4 bytes)	Target IP (4 bytes)	Timestamp (4 bytes)

Фиг. 2.4 Ciphertext на пакет Type 1

Sender IP е адреса на устройството, което изпраща заявката, а Target IP е адреса на другото устройство, с който се цели стартиране на комуникация.

Времевият печат (timestamp) е Unix време в секунди и се използва за защита от обработване на стари пакети. Пакетът се криптира изцяло с AES-GCM преди предаване по мрежата.

2.4.1.4. Пакет „Отговор с ключ“/”GET Reply” (Type 2)

Шлюзът отговаря със съобщение от тип 2, съдържащо генериран симетричен ключ, използван за подсигуряване на host-to-host комуникацията (фиг. 2.5).

Type (1 byte)	Sender IP (4 bytes)	Target IP (4 bytes)	Timestamp (4 bytes)	Pairwise Key (16 bytes)

Фиг. 2.5 Ciphertext на пакет Type 2

Както при пакетите от тип 1, Sender IP е адреса на устройството, което изпраща заявката, а Target IP е адреса на другото устройство, с който се цели комуникация.

Ключът се използва единствено за директната T-ARP комуникация между две устройства и не е достъпен за други участници в мрежата.

2.4.1.5 T-ARP пакети между крайните устройства

След разпределението на ключовете от шлюза, устройствата комуникират директно помежду си чрез T-ARP пакети, които не се криптират, но се защитават чрез криптографски MAC.

2.4.1.5.1. T-ARP Broadcast Discover (Type 3)

T-ARP Discover пакетите спазват структурата на полезния товар, показана на фигура 2.6.

Type	Sender IP	Target IP	Nonce	Timestamp	Signature/MAC
1 byte	4 bytes	4 bytes	4 bytes	4 bytes	16 bytes

Фиг. 2.6 Формат на T-ARP Broadcast Discover пакет

Пакетът се изпраща като broadcast и позволява откриване на MAC адреса на целевото устройство.

2.4.1.5.2. T-ARP Unicast Reply (Type 4)

Форматът е идентичен с този на Type 3 (фиг. 2.6), но пакетът се изпраща като unicast директно до заявителя.

2.4.1.6. Обобщение на типовете пакети

Типове пакети в T-ARP				
Тип	Посока	Назначение	Криптиране	Метод за защита
1	HostA -> Gateway	Заявка за ключ	Да	AES-GCM + AAD
2	Gateway -> HostA	Отговор с ключ	Да	AES-GCM + AAD
3	HostA -> Broadcast	T-ARP DISCOVER Request	Не	Symmetric MAC
4	HostB -> HostA	T-ARP DISCOVER Reply	Не	Symmetric MAC

Фиг. 2.7 Обобщение на T-ARP пакетите

В T-ARP се използват 4 вида пакети, от които само тези между крайните устройства и шлюза са криптирани, а пакетите между две крайни устройства са защитени чрез симетричен MAC.

2.5 T-ARP комуникация

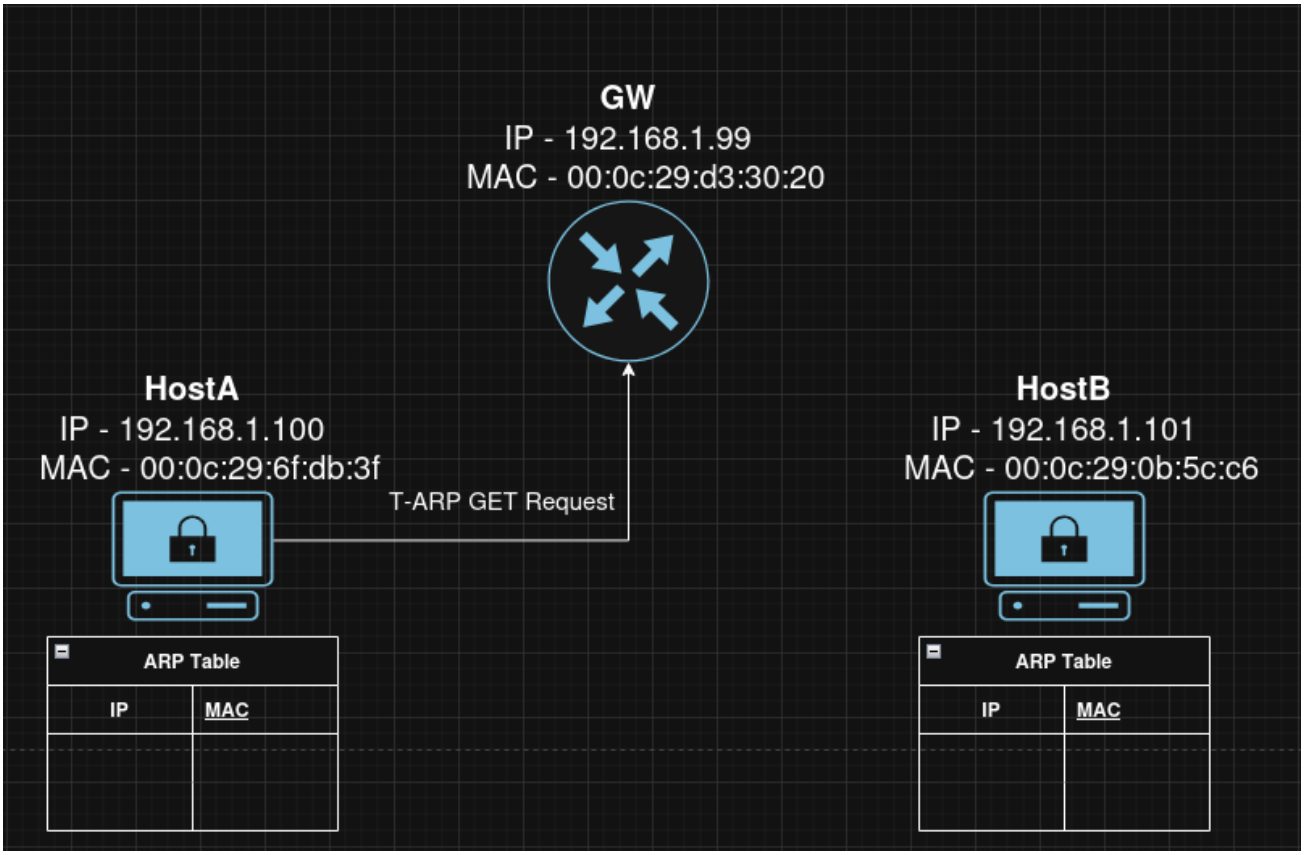
В примера по-долу се демонстрира обмяната на пакетите в среда, в която всички крайни устройства вече разполагат с адреса на шлюза и ключ за

криптирана комуникация с него. За примера се използва конфигурацията, показана на фигура 2.8.

Подробно описание на конфигурацията на крайните устройства и шлюза е дадено в точки 2.6 и 2.7.

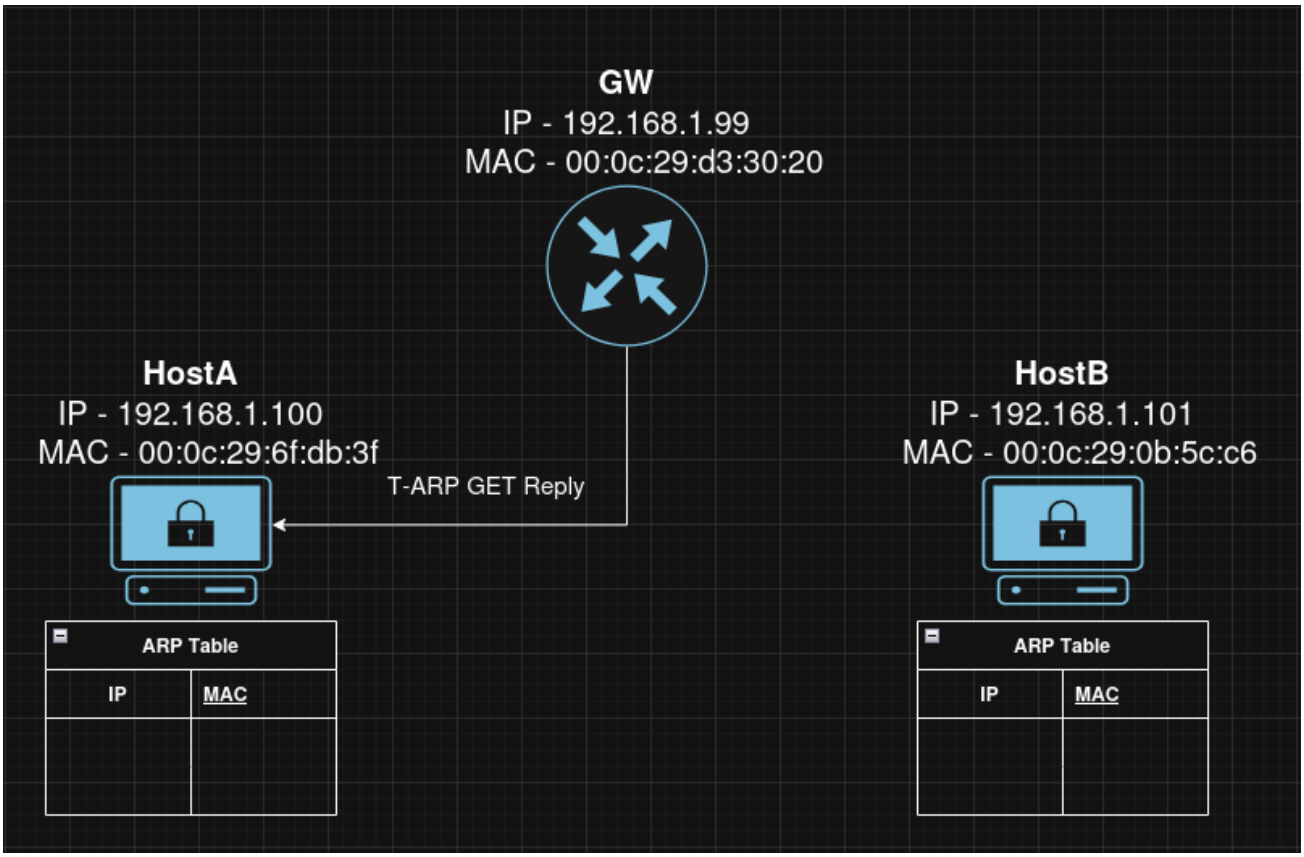
Фиг. 2.8 Начална конфигурация и статус на устройствата

HostA иска да изпрати пакет на HostB, но в ARP таблицата му няма запис, който да обвързва IP и MAC адреса на HostB. HostA изпраща пакет “Заявка за ключ” към шлюза (gateway - GW) (фиг. 2.9)



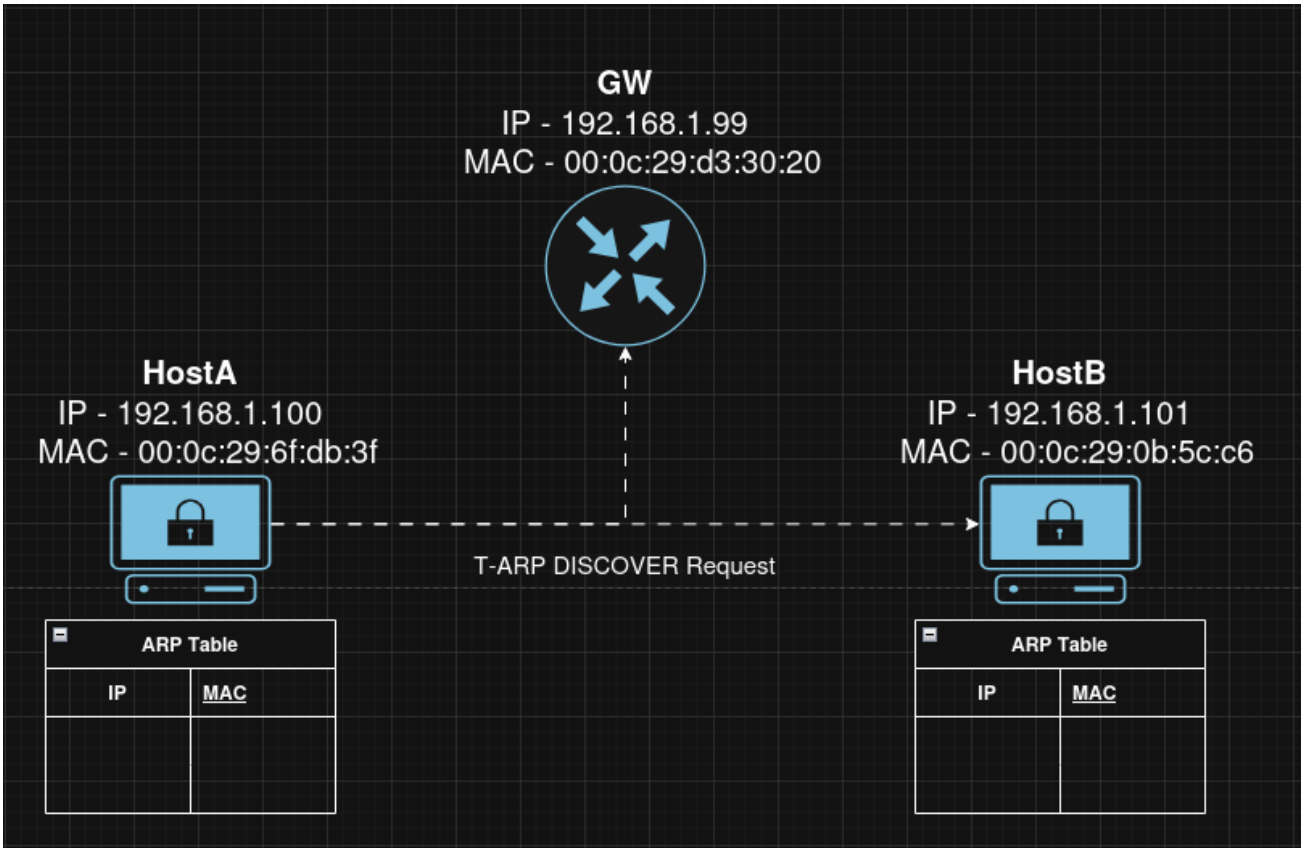
Фиг. 2.9 HostA изпраща заявка към GW

GW получава пакета, верифицира интегритета и автентичността му и ако проверките са успешни, връща отговор на HostA, съдържащ ключ, който да се използва за комуникация между HostA и HostB (фиг. 2.10). Ако някоя от проверките не е успешна, пакетът не се обработва.



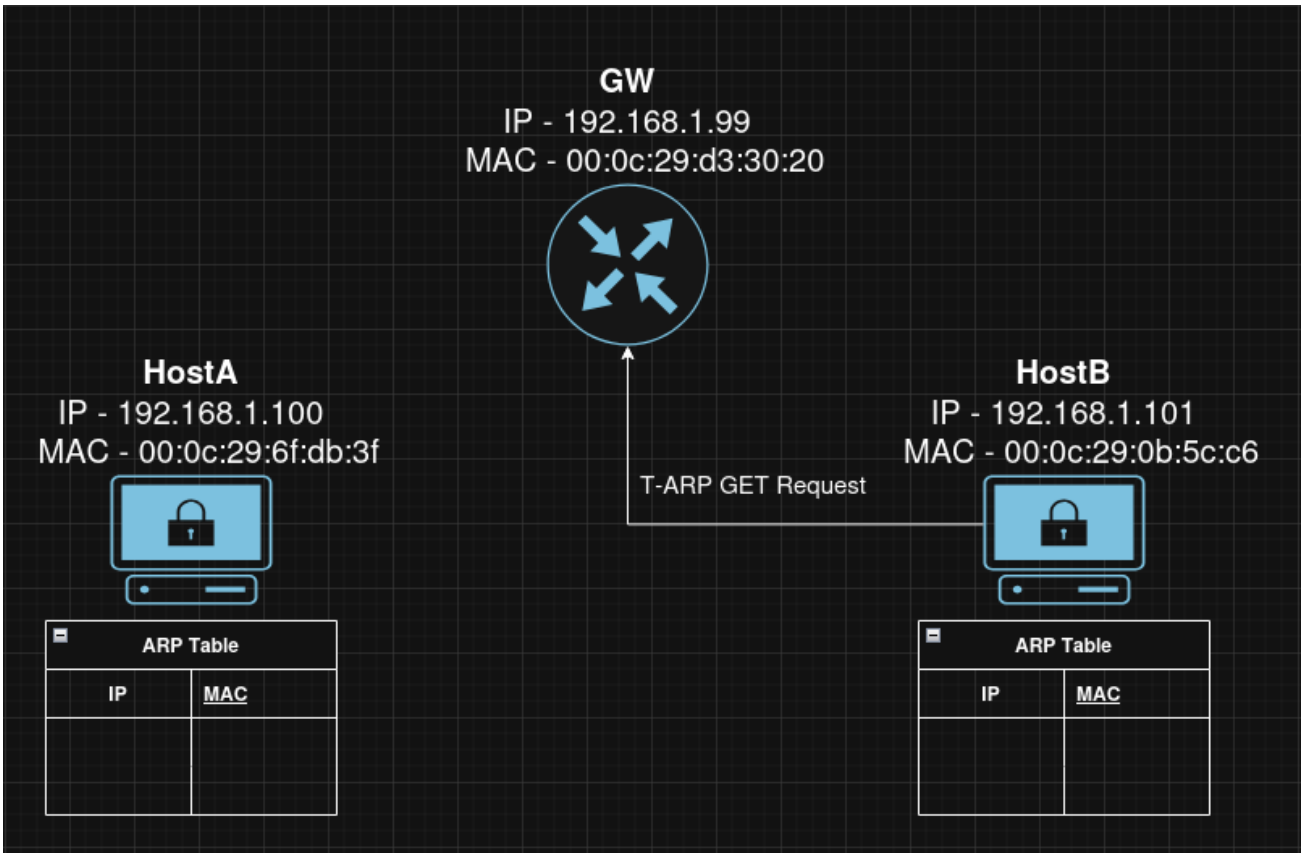
Фиг. 2.10 GW отговаря с ключа за комуникация между HostA и HostB

След като HostA се сдобие с ключа за комуникация с HostB, HostA изпраща broadcast Discover request с цел да получи MAC адреса на HostB (фиг. 2.11) .



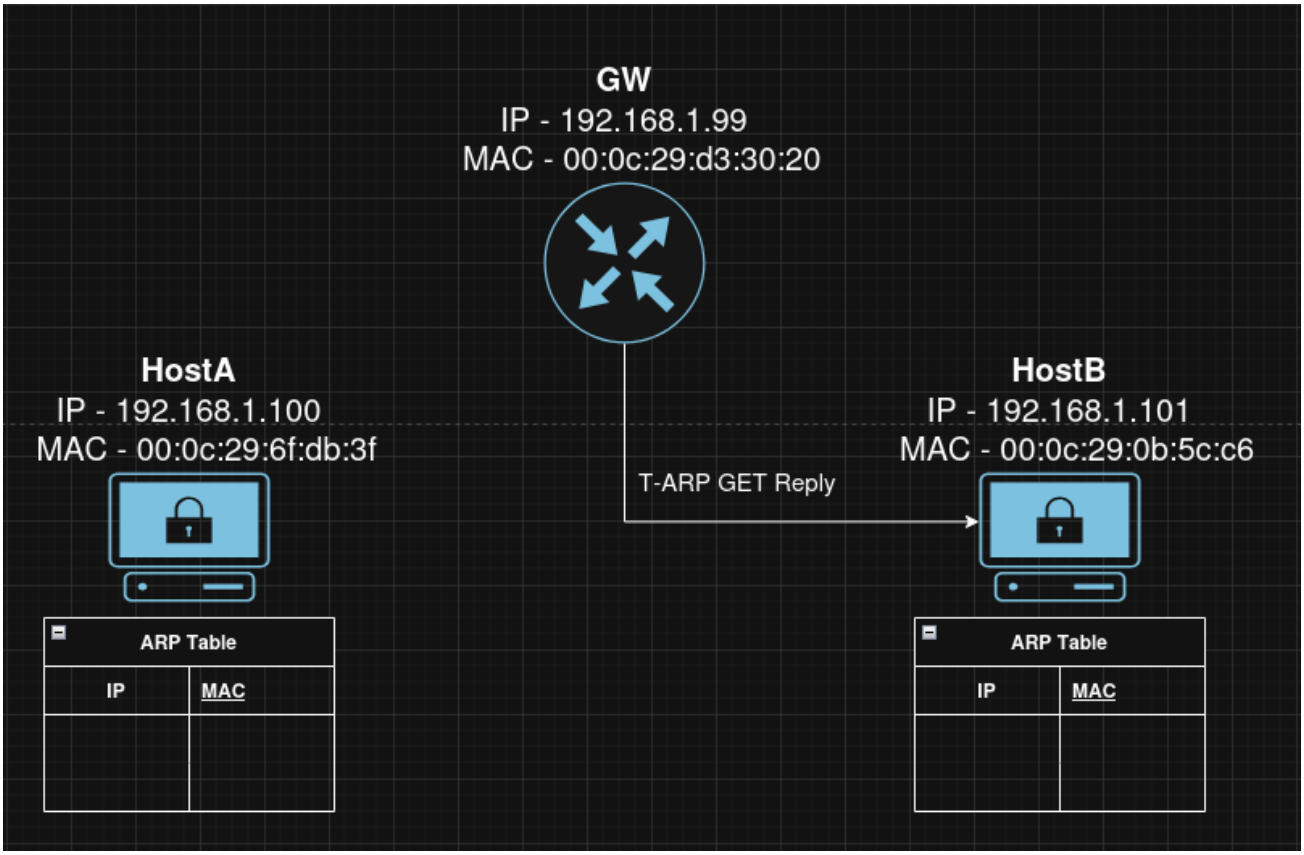
Фиг. 2.11 HostA изпраща T-ARP DISCOVER Request

Всяко устройство в сегмента получава broadcast Discover пакета и проверява дали е предназначен за него. Устройството - получател е необходимо да провери дали симетричният MAC в пакета е валиден. За тази цел се изпраща заявка до GW за получаване на симетричния ключ за комуникация между HostA и HostB (фиг. 2.12).



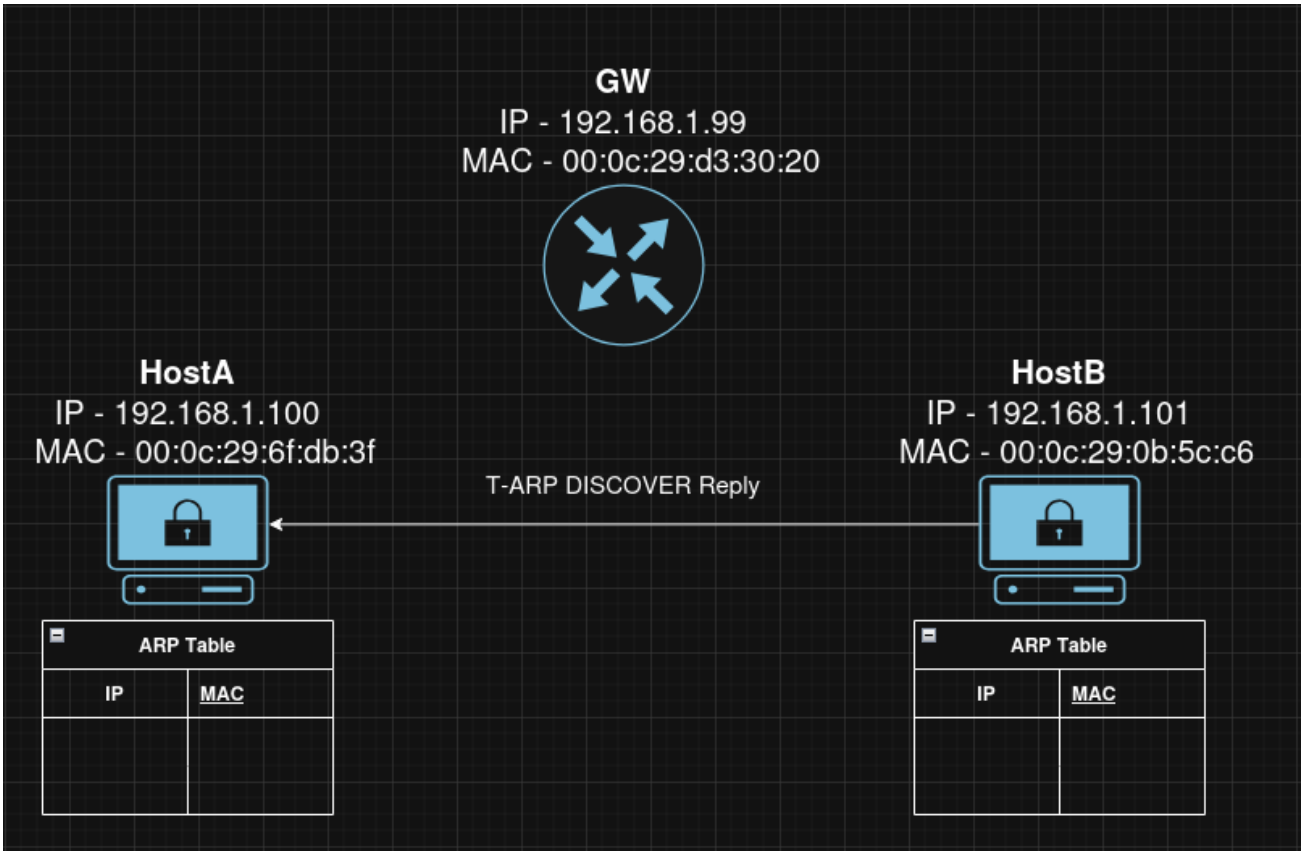
Фиг. 2.12 HostB изпраща заявка за симетричния ключ към GW

GW получава и верифицира заявката на HostB, която е криптирана със специфичния за HostB ключ за комуникация с GW, и ако всички проверки са успешни, GW изпраща на HostB симетричния ключ за комуникация между HostA и HostB (фиг. 2.13).



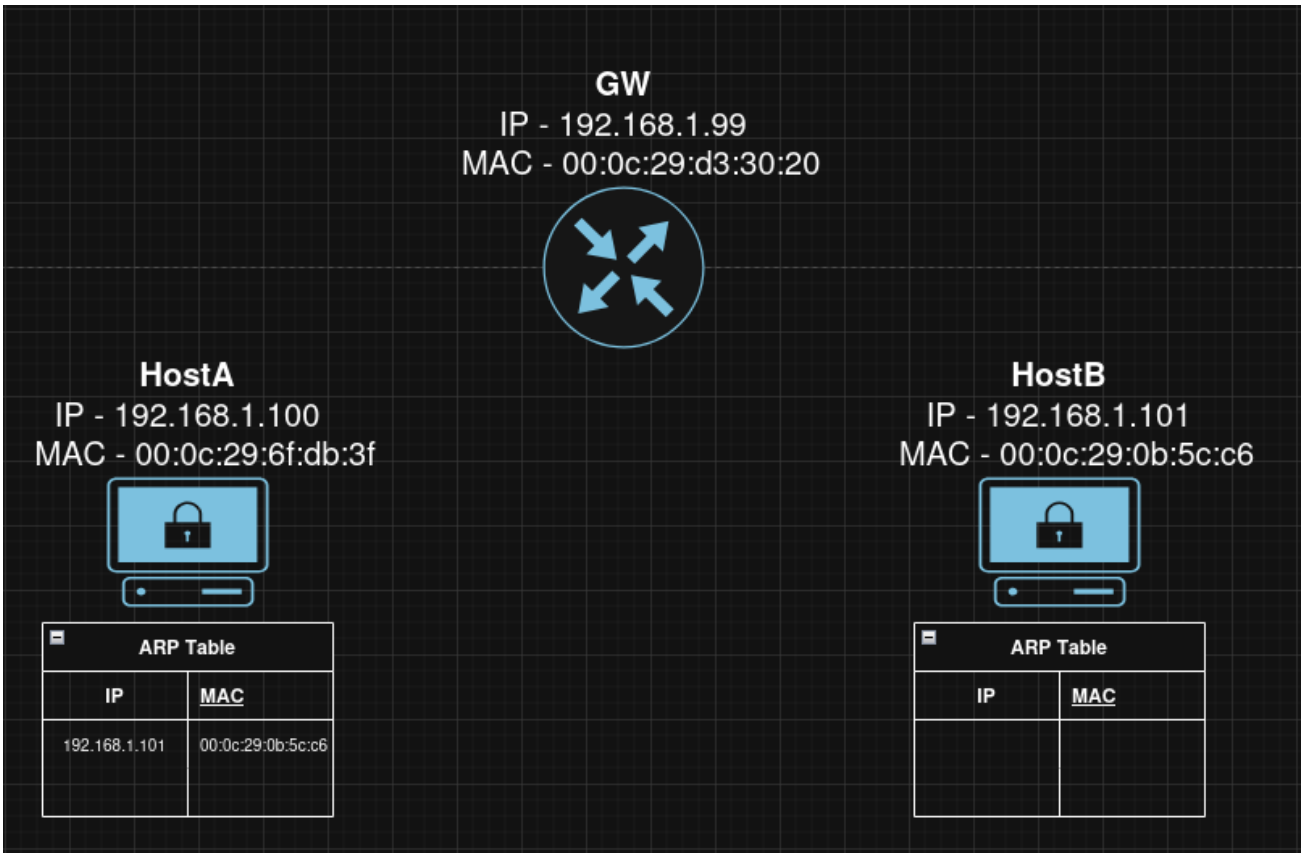
Фиг. 2.13 GW изпраща на HostB симетричния ключ за комуникация между HostA и HostB

HostB получава от GW ключа за комуникация с HostA и с него вече може да провери автентичността на симетричния MAC. Ако проверката е успешна, HostB изпраща unicast отговор на HostA (фиг. 2.14).



Фиг. 2.14 HostB изпраща unicast T-ARP DISCOVER Reply на HostA

HostA получава пакета, верифицира го чрез симетричния MAC и ако верификацията е успешна, записва IP и MAC адреса на HostB в ARP таблицата си (фиг. 2.15).



Фиг. 2.15 HostA записва IP и MAC адреса на HostB в ARP таблицата си

В резултат на осъществената комуникация HostA вече разполага с необходимата информация за изпращане на пакети до HostB, което прави възможна последващата стандартна комуникация между HostA и HostB, която е извън обхвата на настоящата дипломна работа.

2.6 Осъществяване на първоначална комуникация на устройство с централния шлюз (gateway, GW)

Илюстрираният по-горе процес изисква всяко крайно устройство да разполага с предварителна конфигурация за комуникация с шлюза, която

включва: IP адрес и MAC адрес на шлюза, както и ключ за криптиране на комуникацията между крайното устройство и шлюза. За целта на дипломната работа тази конфигурация ще бъде предварително дефинирана, но в тази точка ще бъде показано как те могат да се намерят без да трябва да се въвеждат ръчно.

2.6.1 Принцип на работа

При конфигурацията на централния шлюз се извлича “ключ за инициализация”, който се записва на външен носител, заедно с IP и MAC адреса му. Ключът за инициализация играе ролята на хардуерен ключ за включване в мрежата. При първоначалното включване на устройство към мрежата е необходим локален достъп до външния носител, откъдето новата система извлича необходимата конфигурационна информация и я съхранява локално, което позволява последваща криптирана комуникация с шлюза.

Инициализиращата комуникация е обмен на два типа пакети (T-ARP INIT Request и T-ARP INIT Reply), чрез които шлюзът бива информиран, че устройството ще използва T-ARP като комуникационен протокол и за целта шлюзът създава и изпраща на устройството ключ за криптиране на бъдещата комуникация между тях.

2.6.2 Стъпков процес

За целта на примера по-долу ще изложа стандартен процес за включване на ново устройство към локална мрежа на компания, в която мрежовият администратор отговаря за конфигурирането на всички мрежови устройства и за сигурността на мрежата.

2.6.2.1 Първоначална конфигурация на шлюза

При конфигурацията на централния шлюз, мрежовият администратор извлича на външен носител информацията, съдържаща “ключ за инициализация”, IP адрес и MAC адрес на шлюза.

2.6.2.2 Добавяне на ново устройство към мрежата

Нека разгледаме като пример процесът при наемане на нов служител на компания, на който се предоставя нов служебен компютър без предварително конфигурирани данни за ползване на T-ARP протокола.

При първото включване на компютъра към локалната мрежа на компанията, администратор е необходимо да включи външния носител с конфигурационна информация към компютъра, за да може да се извлече необходимата информация за комуникация с шлюза и тя да бъде запазена за последваща употреба. Ключът се записва в TPM, а останалата информация за шлюза се пази в конфигурационен файл на протокола.

2.6.2.3 Първоначален обмен на данни между крайното устройство и шлюза

След като устройството е извлякло и запазило необходимата му информация от външния носител, то изпраща T-ARP INIT Request към шлюза (фиг. 2.17), с който обявява готовност за ползване на T-ARP протокола и нужда от ключ за криптиране на бъдещата комуникация с шлюза. Пакетът спазва структурата, показана на фигура 2.16 с тип 5 на пакетът, като предаваните данни се криптират посредством AES-GCM с “ключа за инициализация” и AAD, идентично с използвания подход при T-ARP GET Request.

Nonce		Ciphertext		Auth Tag
	Type	Type	Timestamp	
12 bytes	1 byte	1 byte	4 bytes	16 bytes

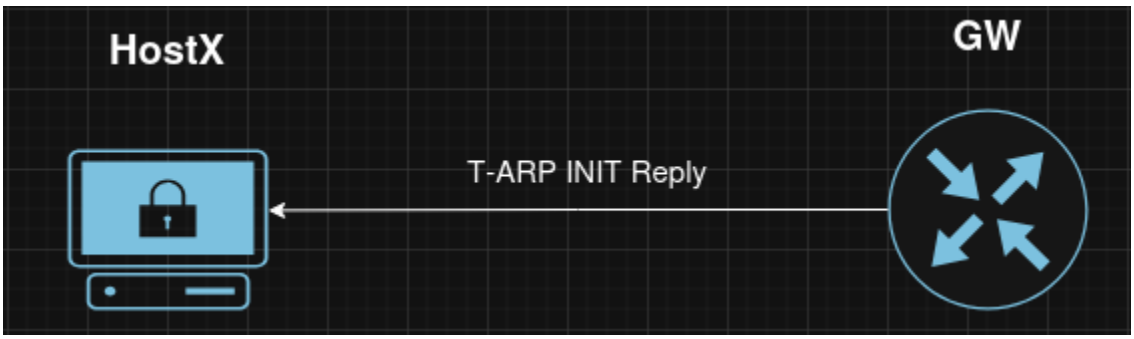
Фиг. 2.16 Формат на T-ARP INIT Request пакет

Фиг. 2.17 Крайно устройство изпраща T-ARP INIT Request към шлюза

Шлюзът GW получава пакета и го декриптира. Ако всички проверки преминат успешно, шлюзът генерира нов ключ и изпраща T-ARP INIT Reply обратно към крайното устройство (фиг. 2.19). Пакетът спазва структурата, показана на фигура 2.18, като предаваните данни се криптират посредством AES-GCM с “ключа за инициализация” и AAD, идентично с използвания подход при T-ARP GET Request.

Nonce		Ciphertext			Auth Tag
	Type	Type	Timestamp	Key	
12 bytes	1 byte	1 byte	4 bytes	16 bytes	16 bytes

Фиг. 2.18 Формат на T-ARP INIT Reply пакет



Фиг. 2.19 Шлюзът изпраща T-ARP INIT Reply към крайното устройство

Крайното устройство получава и декриптира пакета, като след неговата успешна верификация записва получения ключ за бъдеща комуникация с шлюза.

2.7 T-ARP GW

Във всяка мрежа, която е подсигурена чрез T-ARP протокола, е необходимо да има един мрежови рутер, който да бъде конфигуриран като T-ARP шлюз (gateway, GW). В повечето случаи най-оптимално е тази конфигурация да се приложи на рутера, който се намира на границата на локалната мрежа. T-ARP GW е централизирана система за управление на

ключовете, използвани за подсигуряването на комуникацията в една локална мрежа.

2.7.1 VyOS

При симулациите за GW е използвана ръчно компилирана разработка на VyOS операционната система, която е със свободен код и е лесно персонализируема, което я прави идеална за добавяне на разработената T-ARP GW структура.

2.7.2 Конфигурация на протокола

В настоящата реализация на протокола все още не е дефинирана CLI интеграция, като по-долу са показани примерни команди за конфигурация на протокола, които са планирани за реализация при бъдещото развитие на протокола. VyOS използва подобна на Juniper йерархична структура на командите.

Примерни команди:

set security t-arp enable - *Активира протокола.*

set security t-arp gateway address 192.168.1.99/24 - *Дефинира на кой адрес да е достъпен T-ARP GW и в коя мрежа да работи.*

2.7.3 Таблици с данни в T-ARP

За коректната работа на шлюза са необходими и се поддържат две таблици с информация за свързаните устройства и ключове. **За целта на демонстрацията в таблиците ще се попълнят примерни данни, които симулират извършена първоначална комуникация, както е описано в т. 2.6.**

2.7.3.1 Таблица с данни за устройствата и ключовете за комуникация с тях

hosts				
ID	**IP**	**MAC**	**KEY**	**created_at**
1	192.168.1.100	00:0c:29:6f:db:3f	<16 bytes>	…
2	192.168.1.101	00:0c:29:0b:5c:c6	<16 bytes>	…

Фиг. 2.20 Таблица hosts

В таблица hosts се пазят ключовете, които се използват от устройствата за комуникация с шлюза (фиг. 2.20). Ключовете се обвързват с IP и MAC адреса на устройството, за което са предназначени. Налично е и поле, в което се съхранява времето на създаване на съответния ключ.

2.7.3.2 Таблица със симетрични ключове за комуникация между две устройства

host_pair_keys			
ID	**Host1 IP**	**Host2 IP**	**Key**
1	192.168.1.100	192.168.1.101	<16 bytes>
…	**…**	**…**	**…**

Фиг. 2.21 Таблица host_pair_keys

В таблица host_pair_keys се пазят ключовете, които шлюзът генерира за комуникация между две обвързани устройства (фиг. 2.21).

2.7.3.3 Таблица с метаданни

В настоящата реализация на T-ARP протокола главният ключ на шлюза се съхранява в таблица metadata. В бъдещи реализации тази таблица ще бъде заменена от по-сигурно хранилище на данни, например от външен secret manager (Vault, Ansible Vault и др.), като данните биха могли да се зареждат само в паметта или данните могат да се пазят във файл в /config директорията.

2.8 Заключение

Новоразработеният с настоящата дипломна работа протокол T-ARP надгражда стандартния протокол ARP, като добавя допълнително ниво на сигурност при комуникацията между устройства в локална мрежа и така преодолява заплахите от атаки срещу ARP (ARP Spoofing, Man-in-the-Middle атаки и други), които могат да доведат до катастрофални последици и разкриване на конфиденциална информация. Чрез използването на AES-GCM и симетричен MAC се подсигурява защитата на предаваната информация чрез криптиране, което не позволява неоторизиран достъп до нея без необходимите ключ и AAD.

Трета глава: Софтуерна разработка на T-ARP

3.1 T-ARP GW

3.1.1 Изграждане на Debian пакет

3.1.1.1 Структура на Debian пакет

На фигура 3.1 е показана файловата структура на разработеният Debian пакет, който се компилира заедно с операционната система на VyOS.

На фигура 3.2 може да се види файлът control, намиращ се в DEBIAN директорията. Control файлът описва метаданните на Debian пакет, използван от пакетния мениджър за инсталиране, управление на зависимости и класификация на софтуера.

Важни редове във файла:

- **Package: vyos-tarp-gw** - Уникално име на пакета в системата. Използва се от пакетния мениджър за идентификация.
- **Depends:** - Задължителни зависимости. В този случай полето е празно, което означава, че няма твърди зависимости, необходими за инсталацията му.
- **Recommends: python3, python3-pycryptodome, sqlite3** - Препоръчителни пакети, необходими за пълна функционалност.
- **Section: net** - Категория на пакета. Показва, че пакетът е мрежов софтуер.

На фигура 3.3 е показан файлът, който дефинира systemd услуга, която управлява стартирането, рестартирането и правата на демона vyos-tarp-gw. В [Unit] секцията са дефинирани зависимостите и реда на стартиране. Чрез:

After=network-online.target

After=network.target

се дефинира, че протоколът ще се стартира след инициализация на мрежата.

В [Service] секцията се дефинира поведението и сигурността:

- **Type=simple** - Услугата се счита за стартирана веднага след изпълнение на процеса.
- **ExecStart=/usr/bin/vyos-tarp-gw** - Основният изпълним файл на демона.
- **Restart=always** - Демонът се рестартира автоматично при срив или неочаквано спиране.
- **RestartSec=5** - Изчакване от 5 секунди преди рестарт, което предотвратява restart loop.
- **User=root** - Услугата се изпълнява с root права, необходими за директен достъп до ARP и работа с “raw” сокети.
- **CapabilityBoundingSet=CAP_NET_RAW CAP_NET_ADMIN AmbientCapabilities=CAP_NET_RAW CAP_NET_ADMIN** - Ограничават правата само до CAP_NET_RAW (изпращане/приемане на raw ARP пакети) и CAP_NET_ADMIN (управление на мрежови интерфейси и настройки).
- **NoNewPrivileges=true** - Забранява ескалация на привилегии.
- **ReadWritePaths=/config** - Позволява запис само в /config (VyOS

конфигурационна директория).

- **ProtectSystem=false**

  ProtectHome=false

  PrivateTmp=false - Тези опции са изключени поради специфичните нужди на VyOS и мрежовия демон за директен достъп до системни ресурси.

В **usr/bin** се намира **vyos-tarp-gw**, който е главния изпълним файл, отговарящ за функционалността на пакета.

```
vyos-tarp-gw
├── DEBIAN
│   ├── conffiles
│   ├── control
│   ├── md5sums
│   ├── postinst
│   └── prerm
├── etc
│   ├── systemd
│   │   └── system
│   │       └── tarp-gw-init.service
│   └── vyos
│       ├── tarp-gw.conf
│       └── tarp-gw.conf.template
├── opt
│   └── vyatta
│       ├── post-config.d
│       │   └── 50-tarp-gw-reload
├── usr
│   ├── bin
│   │   └── vyos-tarp-gw
│   ├── lib
│   │   └── systemd
│   │       └── system
│   │           └── vyos-tarp-gw.service
│   ├── libexec
│   │   └── vyos
│   │       └── tarp-gw-init
│   └── share
│       ├── doc
│       │   └── vyos-tarp-gw
│       │       ├── changelog.Debian.gz
│       │       └── copyright
│       └── vyos-tarp-gw
│           └── db
│               └── schema.sql
└── var
    └── lib
        └── vyos-tarp-gw
            └── tarp.db
```

Фиг. 3.1 Файлова структура на T-ARP GW debian пакет

Package: vyos-tarp-gw

Version: 1.0-1

Architecture: all

Maintainer: Nikola Aleksov

Installed-Size: 40

Depends:

Recommends: python3, python3-pycryptodome, sqlite3

Section: net

Priority: optional

Description: T-ARP Gateway service for VyOS. A trusted ARP gateway daemon that generates host-host keys, validates T-ARP packets and provides secure key exchange.

Фиг. 3.2 Съдържание на **control**

```
[Unit]
Description=VyOS T-ARP Gateway Daemon
After=network-online.target
After=network.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/vyos-tarp-gw
Restart=always
RestartSec=5
User=root
CapabilityBoundingSet=CAP_NET_RAW CAP_NET_ADMIN
AmbientCapabilities=CAP_NET_RAW CAP_NET_ADMIN
NoNewPrivileges=true
ReadWritePaths=/config
ProtectSystem=false
ProtectHome=false
PrivateTmp=false

[Install]
WantedBy=multi-user.target
```

Фиг. 3.3 Съдържание на **vyos-tarp-gw.service**

3.1.1.2 Компилиране на Debian пакета

За да се генерира използваем .deb пакет е необходимо да се извърши компилация със следната команда:

dpkg-deb --build vyos-tarp-gw

След изпълнението на командата се създава vyos-tarp-gw.deb файл в активната директория.

3.1.1.3 Подготовка на .deb файла за компилация с VyOS

След като успешно сме компилирали пакета, необходимо е да го поставим в директорията, от която ще компилираме VyOS операционната система. За да може по време на изграждането да се обработи и инсталира пакета, необходимо е да го запишем в директорията **vyos-build/data/live-build-config/archives/**.

3.1.2 Основен изпълним файл, отговарящ за функционалността на пакета

Файлът /usr/bin/vyos-tarp-gw е скрипт, написан на python, който слуша за, получава и обработва пакети от T-ARP (с EtherType 0x88b5).

3.1.2.1 Използвани функции и библиотеки

Използват се множество стандартни python библиотеки (фиг. 3.4), от които най-важните са:

- **sys** - Предоставя достъп до вътрешни параметри и функции на Python интерпретатора. В конкретния случай:

```
sys.path.insert(0, "/usr/lib/python3/dist-packages")
```

се използва за ръчно добавяне на системната директория с Python пакети. Това е необходимо в среди като VyOS, където стандартният Python path може да бъде ограничен или модифициран.

- **socket -** Осигурява интерфейс на ниско ниво за мрежова комуникация, който се използва за работа със сурови (raw) сокети, изпращане и приемане на пакети и позволява директен достъп до мрежовия слой (Layer 2).
- **sqlite3** - sqlite3 е вградена релационна база данни в Python, която се използва за създаване на таблиците за съхранение на криптографски ключове и асоцииране на MAC/IP адреси с доверени устройства.
- **os -** Библиотеката предоставя интерфейс към операционната система, който се използва за работа с файловата система и проверка на права и наличност на файлове.
- **secrets -** Осигурява криптографски сигурен генератор на случайни стойности, който се използва за генерация на nonce.
- **cryptography.hazmat.primitives.ciphers.aead.AESGCM -** AESGCM реализира алгоритъма AES в режим Galois/Counter Mode.
- **cryptography.hazmat.primitives.kdf.hkdf.HKDF -** HKDF е функция за извеждане на ключове (Key Derivation Function), която се използва за генериране на сесийни ключове.

- **cryptography.hazmat.primitives.hashes -** Модулът hashes предоставя криптографски хеш функции (напр. SHA-256), които се използват за входен параметър за HKDF.

```
import sys
sys.path.insert(0, "/usr/lib/python3/dist-packages")
import socket, struct, time, sqlite3, json, os, signal
import logging
import logging.handlers
import secrets
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes
```

Фиг. 3.4 Използвани библиотеки

Използвани функции:

sys.path.insert(index, path) добавя директория към пътя за търсене на модули. Използва се за осигуряване на достъп до системните библиотеки в среда с ограничен Python environment, каквато е и VyOS.

Параметри:

1. index (int) - позиция в списъка с пътища.
2. path (str) - директория с Python модули.

os.path.exists(path) проверява дали даден файл или директория съществува.

Параметри:

1. path (str) - път към файл или директория.

os.makedirs(path, exist_ok=False) създава директории, необходими за конфигурация или логове.

Параметри:

1. path (str) - път към директория.
2. exist_ok (bool) - флаг дали да се игнорира грешка при вече съществуваща директория.

socket.socket(family, type, proto=0) създава суров сокет за директна работа с Ethernet пакети.

Параметри:

1. family - тип адрес (напр. AF_PACKET).
2. type - тип сокет (SOCK_RAW).
3. proto - протокол (напр. ETH_P_ARP).

socket.bind(address) свързва сокета с конкретен мрежов интерфейс.

Параметри:

1. address (tuple) - мрежов интерфейс и протокол.

socket.recvfrom(bufsize) приема входящ мрежов пакет и адрес на подателя.

Параметри:

1. bufsize (int) - максимален размер на пакета.

socket.send(data) изпраща пакет в мрежата.

Параметри:

1. data (bytes) - бинарни данни.

struct.pack(format, *values) преобразува структурирани данни в бинарен формат.

Параметри:

1. format (str) - описание на бинарния формат.
2. values - стойности за кодиране.

struct.unpack(format, buffer) декодира бинарни данни.

Параметри:

1. format (str) - формат на данните.
2. buffer (bytes) - входни бинарни данни.

sqlite3.connect(database) отваря или създава локална база данни.

Параметри:

1. database (str) - път към DB файл.

cursor.execute(sql, parameters=None) изпълнява заявки за запис или четене на ключове и устройства от таблиците.

Параметри:

1. sql (str) - SQL заявка.
2. parameters (tuple) - параметри за заявката.

cursor.fetchone() / cursor.fetchall() извлича резултати от заявка.

connection.commit() потвърждава промените в базата данни.

signal.signal(signalnum, handler) регистрира функция за контролирано спиране на демона.

Параметри:

1. signalnum - тип сигнал (SIGTERM, SIGINT).
2. handler - функция обработчик.

logging.getLogger(name) създава логер за текущия модул.

Параметри:

1. name (str).

logger.setLevel(level) определя детайлността на логовете.

Параметри:

1. level - ниво на логване (INFO, ERROR).

logger.info(msg) / logger.error(msg) записва диагностични съобщения.

Параметри:

1. msg (str).

logging.handlers.RotatingFileHandler(filename, maxBytes, backupCount) управлява лог файлове с ротация.

Параметри:

1. filename (str).
2. maxBytes (int).
3. backupCount (int).

secrets.token_bytes(n) генерира криптографски сигурни случайни данни.

Параметри:

1. n (int) - брой байтове.

AESGCM(key) инициализира AES-GCM криптографски контекст.

Параметри:

1. key (bytes) - 128/192/256-битов ключ.

AESGCM.encrypt(nonce, data, associated_data) криптира и автентикира данни.

Параметри:

1. nonce (bytes).
2. data (bytes).
3. associated_data (bytes | None).

AESGCM.decrypt(nonce, data, associated_data) проверява целостта и декриптира данни.

Параметри:

4. nonce (bytes).
5. data (bytes).
6. associated_data (bytes | None).

HKDF(algorithm, length, salt, info) инициализира механизъм за извеждане на ключове.

Параметри:

1. algorithm - хеш функция (напр. SHA256).
2. length (int) - дължина на ключа.

3. salt (bytes).
4. info (bytes).

HKDF.derive(input_key_material) генерира сесиен криптографски ключ.

Параметри:

1. input_key_material (bytes).

hashes.SHA256() криптографска хеш функция, използвана за KDF (Key Derivation Function) и защита на данните.

3.1.2.2 Слушане за пакети

Библиотеката socket се използва за слушане на мрежови трафик (фиг. 3.5) и при получаване на пакет, като пакетите с различен от необходимия EtherType не се обработват (фиг. 3.6).

```
        s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.htons(ETH_P_ALL))
        s.bind((iface, 0))
```

Фиг. 3.5 Използване на socket за слушане за мрежови трафик

```
            eth_type = pkt[12:14]
                .
                .
                .
            if eth_type != b"\x88\xb5":
                continue
```

Фиг. 3.6 Проверка на EtherType

Проверява се какъв е типа на пакета и се обработват само тези с тип “1” (фиг. 3.7).

```
            tarp_type = pkt[14]

            if tarp_type != 1:
                continue
```

Фиг. 3.7 Проверка на T-ARP типа

3.1.2.3 Декриптиране и верификация на пакета

При успешни проверки на ниво Ethernet header, от заглавната част на пакета се извлича MAC адреса на изпращача и се прави опит да се извлече ключа за комуникация със съответното устройство (фиг. 3.8).

```
            eth_src = pkt[6:12]

            mac_hex = eth_src.hex()

            cur = conn.cursor()
            cur.execute("SELECT gw_key FROM hosts WHERE mac = ?",
(mac_hex,))
            row = cur.fetchone()
            if not row:
                logger.info("Host key is not found in db. Dropping
%s", row)
                continue
```

Фиг. 3.8 Извличане на ключа за комуникация с устройството от базата данни

При успешно извлечен ключ се изгражда AAD (фиг. 3.9) и с него и ключа се опитва декриптиране на криптирания полезен товар на пакета (фиг. 3.10).

```
def build_aad(src_mac_str, dst_mac_str, ethertype=0x88B5):
    if not isinstance(src_mac_str, bytes):
        src = bytes.fromhex(src_mac_str.replace(':',''))
    else:
        src = src_mac_str
    if not isinstance(dst_mac_str, bytes):
        dst = bytes.fromhex(dst_mac_str.replace(':',''))
    else:
        dst = dst_mac_str
    et  = struct.pack('!H', ethertype)
    return min(src, dst) + max(src, dst) + et
aad = build_aad(
            src_mac_str=eth_src,
            dst_mac_str=eth_dst
        )
```

Фиг. 3.9 Изграждане на AAD

```
host_key_hex = row[0]
        host_key = bytes.fromhex(host_key_hex)
        blob = pkt[15:]

        REAL_BLOB_LEN = 41
        blob = blob[:REAL_BLOB_LEN]

        try:
            plaintext = aes_gcm_decrypt(host_key, blob, aad)
        except ValueError:
            print(f"[GW] auth failed from {eth_src}")
            continue
```

Фиг. 3.10 Декриптиране на криптирания пакет с ключ и AAD

При успешно декриптиране на пакета се извличат sender_ip, target_ip и timestamp (фиг. 3.11).

```
            sender_ip = socket.inet_ntoa(plaintext[1:5])
            target_ip = socket.inet_ntoa(plaintext[5:9])
            timestamp = struct.unpack('!I', plaintext[9:13])[0]
```

Фиг. 3.11 Извличане на данни от декриптирания пакет

3.1.2.4 Обработка на декриптирания пакет и изпращане на отговор

След като се извлекат данните се проверява дали в таблицата за ключове между устройствата вече има създаден валиден ключ за комуникация между двете устройства (фиг. 3.12). Ако има такъв се изпраща обратно пакет, съдържащ ключа (фиг. 3.13), а ако няма се генерира нов ключ (фиг. 3.14) и се изпраща отговор.

```
                a, b = sorted([sender_ip, target_ip])
                cur.execute("SELECT key FROM host_pair_keys WHERE
host_a=? AND host_b=?", (a, b))
                row = cur.fetchone()
                logger.info("Key fetched successfully")
                if row:
                    hhkey = row[0]
                else:
                      .
                      .
```

Фиг. 3.12 Проверка дали съществува ключ

```
    def create_arpt_packet(pkt_type, sender_ip, target_ip, key_hex,
host_key, gw_mac, host_mac):
        key = normalize_key(host_key)
        ts = struct.pack("!I", int(time.time()))
        plaintext = (
            b'\x02' +
            socket.inet_aton(sender_ip) +
            socket.inet_aton(target_ip) +
            ts +
            key
       )
        aad = build_aad(
            src_mac_str=gw_mac,
            dst_mac_str=host_mac
       )
        payload = aes_gcm_encrypt(host_key, plaintext, aad)
        return b'\x02' + payload

    def send_packet(interface, dst_mac, payload):
        s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW)
        s.bind((interface, 0))
        src_mac = s.getsockname()[4]
        eth = dst_mac + src_mac + b"\x88\xb5"
        s.send(eth + payload)
                . . .
    reply = create_arpt_packet(2, sender_ip, target_ip, hhkey,
host_key, eth_dst, eth_src)
    send_packet(iface, eth_src, reply)
```

Фиг. 3.13 Изпращане на пакет с ключа за комуникация обратно към устройството

```
    def derive_hhkey_from_master_key(
        master_key: bytes,
        context: bytes,
        length: int = 16,
    ) -> bytes:
        if not isinstance(master_key, (bytes, bytearray)):
            raise TypeError("master_key must be bytes")
        if not isinstance(context, (bytes, bytearray)):
            raise TypeError("context must be bytes")

        hkdf = HKDF(
            algorithm=hashes.SHA256(),
            length=length,
            salt=None,
            info=context,
       )

        return hkdf.derive(master_key)
                .
                .
                .
                if row:
                    hhkey = row[0]
                else:
                    cur.execute("SELECT value FROM metadata WHERE
key='master_key'")
                    master_key_hex = cur.fetchone()[0]
                    master_key = bytes.fromhex(master_key_hex)
                    hhkey_bin =
derive_hhkey_from_master_key(normalize_key(master_key),
```

```
sender_ip.encode()+target_ip.encode()+struct.pack("!I",
int(time.time())))[:16]
                    hhkey = hhkey_bin.hex()

                    cur.execute("INSERT INTO
host_pair_keys(host_a,host_b,key) VALUES(?,?,?)",
                                (a, b, hhkey))
                    conn.commit()
```

Фиг. 3.14 Генериране на нов ключ

3.1.3 Компилиране на VyOS [7]

За да може да се компилира VyOS операционната система, устройството, от което се компилира, е необходимо да работи на Debian Bookworm операционна система.

Първо е необходимо да се клонира GitHub хранилището на локалната машина с командата:

git clone -b current --single-branch https://github.com/vyos/vyos-build

След като се свали, влизаме в новосвалената директория:

cd vyos-build

Има две възможности: да се компилира с docker контейнер или локално. За целите на настоящата дипломна работа е използвана локална компилация.

Необходимо е vyos-tarp-gw пакетът да се постави в директория **data/live-build-config/archives/,** за да може да се компилира в операционната система.

За да компилираме самата операционна система и да получим .iso файл, първо е добре да се изчистят минали компилационни файлове с:

sudo make clean

След това се изпълнява командата, която стартира процеса на компилация:

sudo ./build-vyos-image generic --architecture amd64 --build-type release --version <име на версията>

След като завърши процеса на компилация, който обикновено отнема около 5 до 10 минути, в директорията builds може да се намери новосъздадения .iso файл.

3.2 T-ARP симулационен скрипт на крайните устройства

Файлът, намиращ се в /opt/arp-t/arpt_host.py, представлява основен изпълним Python скрипт за T-ARP устройство. Той реализира пълната логика от страната на крайното устройство, включваща постоянно слушане за Ethernet

трафик (sniffer), заявяване на криптографски ключове от шлюза, изпращане на T-ARP заявки (включително и broadcast) и обработка и отговор на входящи T-ARP заявки.

3.2.1 Използвани библиотеки и зависимости

В настоящата разработка са използвани стандартни Python библиотеки, както и криптографски модули от PyCryptodome (фиг. 3.15).

```
import os
import socket
import struct
import select
import json
import time
import threading
import queue
import fcntl
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
```

Фиг. 3.15 Използвани библиотеки в скрипта на крайните устройства

Основните използвани библиотеки са:

os - Използва се за работа с файловата система, включващо проверка за наличие на конфигурационни и ключови файлове и извикване на системни команди (promiscuous режим).

socket - Осигурява нисконивòв достъп до Ethernet слоя чрез AF_PACKET и SOCK_RAW, позволявайки слушане на всички Ethernet пакети и изпращане на собствени T-ARP пакети.

threading - Използва се за стартиране на двете паралелни нишки: sniffer нишка и handler нишка за заявки.

queue - Осигурява thread-safe комуникация между нишките.

fcntl - Използва се за извличане на хардуерния MAC адрес на интерфейса чрез ioctl.

Crypto.Cipher.AES - Включва в себе си:

- **AES-ECB** - за подписване и проверка на T-ARP пакети;
- **AES-GCM** - за криптиране и автентикация на комуникацията с шлюза.

Crypto.Random.get_random_bytes - използва се за генериране на nonce стойности и ключове.

3.2.3 Конфигурационен файл

В конфигурационния файл /opt/arp-t/config.json се съхраняват IP и MAC адресите на шлюза, както и ключа за сигурна комуникация с него.

3.2.4 Помощни функции

В настоящата разработка са използвани и помощни функции за работа с адреси, които се използват за конвертиране между текстови и бинарни формати (фиг. 3.16).

```
     def mac_str_to_bytes(mac): return
bytes.fromhex(mac.replace(':',''))
     def mac_bytes_to_str(b): return ':'.join(f"{x:02x}" for x in b)
     def ip_str_to_bytes(ip): import socket as _s; return
_s.inet_aton(ip)
     def ip_bytes_to_str(b): import socket as _s; return
_s.inet_ntoa(b)
```

Фиг. 3.16 Помощни функции

Прави се и извличане на собствения MAC адрес чрез използването на ioctl за директно четене на хардуерния MAC адрес от мрежовия интерфейс (фиг. 3.17).

```
     def get_hw_mac(iface=IFACE):
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        info = fcntl.ioctl(s.fileno(), 0x8927, struct.pack('256s',
iface.encode('utf-8')))
        return ':'.join('%02x' % b for b in info[18:24])
```

Фиг. 3.17 Функция за извличане на собствения MAC адрес

Имплементирана е и функция, която изключва стандартния ARP протокол от съответния интерфейс и включва promiscuous режим (фиг. 3.18).

```
def set_promisc(iface=IFACE):
    os.system(f'ip link set {iface} arp off')
    os.system(f'ip link set {iface} promisc on')
```

Фиг. 3.18 Функция за изключване на ARP протокола и включване на promiscuous режим на зададения интерфейс

3.2.5 Криптографска логика

AES-ECB се използва за подписване на T-ARP Broadcast заявки и проверка на автентичността на T-ARP отговори (фиг. 3.19).

```
def aes_ecb_encrypt(key, data):
    cipher = AES.new(key, AES.MODE_ECB)
    pad = 16 - (len(data) % 16)
    return cipher.encrypt(data + bytes([pad])*pad)

def aes_ecb_decrypt(key, ct):
    cipher = AES.new(key, AES.MODE_ECB)
    pt = cipher.decrypt(ct)
    pad = pt[-1]
    return pt[:-pad]
```

Фиг. 3.19 Помощни функции за AES-ECB криптиране и декриптиране

AES-GCM се използва за комуникацията с шлюза с цел да се осигури конфиденциалност и защита от фалшифициране на данните (фиг. 3.20).

```python
def aes_gcm_encrypt(key, plaintext, aad):
    nonce = get_random_bytes(12)
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    cipher.update(aad)
    ciphertext, tag = cipher.encrypt_and_digest(plaintext)
    return nonce + ciphertext + tag

def aes_gcm_decrypt(key, blob, aad):
    nonce = blob[:12]
    tag = blob[-16:]
    ciphertext = blob[12:-16]
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    cipher.update(aad)
    return cipher.decrypt_and_verify(ciphertext, tag)
```

Фиг. 3.20 Помощни функции за AES-GCM криптиране и декриптиране

AES-GCM използва AAD, базирано на MAC адресите и EtherType, за допълнително ниво на сигурност (фиг. 3.21).

```
def build_aad(src_mac_str, dst_mac_str, ethertype=0x88B5):
    if not isinstance(src_mac_str, bytes):
        src = bytes.fromhex(src_mac_str.replace(':',''))
    else:
        src = src_mac_str
    if not isinstance(dst_mac_str, bytes):
        dst = bytes.fromhex(dst_mac_str.replace(':',''))
    else:
        dst = dst_mac_str
    et  = struct.pack('!H', ethertype)
    return min(src, dst) + max(src, dst) + et
```

Фиг. 3.21 Помощна функция за изграждане на AAD

На фигура 3.22 е показана функцията, използвана за изпращане на пакетите посредством socket.

```
def send_eth_raw(dst_mac_str, payload):
    dst = mac_str_to_bytes(dst_mac_str)
    src = bytes.fromhex(HW_MAC.replace(':',''))
    eth_frame = dst + src + struct.pack('!H', ETH_P_ARPT) +
payload
    s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW)
    s.bind((IFACE, 0))
    s.send(eth_frame)
    s.close()
```

Фиг. 3.22 Функция за изпращане на мрежови пакети

3.2.6 Sniffer нишка

Функцията sniffer() създава raw socket с ETH_P_ALL, който постоянно приема Ethernet пакети. Ако пакетът е с EtherType 0x88b5 (EtherType на T-ARP), то той се разпределя спрямо типа му в две опашки: request_queue за T-ARP заявки (за пакети от тип 3) и reply_queue за T-ARP отговори (за пакети от тип 4) (фиг. 3.23).

```
    def sniffer():
        s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.htons(ETH_P_ALL))
        s.bind((IFACE, 0))
        while True:
            pkt = s.recv(4096)
            if len(pkt) < 14:
                continue
            eth_type = struct.unpack('!H', pkt[12:14])[0]
            src_mac = mac_bytes_to_str(pkt[6:12])
            dst_mac = mac_bytes_to_str(pkt[0:6])
            payload = pkt[14:]
            if eth_type != ETH_P_ARPT:
                continue

            if len(payload) >= 33
                try:
                    pkt_type = payload[0]
                except Exception:
                    pkt_type = None
                if pkt_type == 3:
                    request_queue.put((src_mac, payload, pkt))
                elif pkt_type == 4:
                    reply_queue.put((src_mac, payload, pkt))
                else:
                    request_queue.put((src_mac, payload, pkt))
            else:
                print("[SNIF] unknown T-ARP payload (too short)")
```

Фиг. 3.23 Sniffer функция

3.2.7 Заявяване на ключ от шлюза

Функцията request_key_from_gw() отговаря за получаването на host-to-host ключ за комуникация от GW.

На фигура 3.24 може да се види как се изпраща заявка към GW за получаване на този комуникационен ключ.

```
        plaintext = (
            b'\x01' +
            ip_str_to_bytes(config['ip']) +
            ip_str_to_bytes(target_ip) +
            struct.pack('!I', int(time.time()))
        )
        aad = build_aad(
            src_mac_str=hw_mac,
            dst_mac_str=gw_mac_str
        )
        payload = b'\x01' + aes_gcm_encrypt(host_key, plaintext, aad)
        ss = socket.socket(socket.AF_PACKET, socket.SOCK_RAW)
        ss.bind((IFACE, 0))
        eth_frame = gw_mac + bytes.fromhex(hw_mac) + struct.pack('!H',
ETH_P_ARPT) + payload
        send_time = time.time()
        ss.send(eth_frame)
        ss.close()
```

Фиг. 3.24 Изпращане на T-ARP GET Request

След като се изпрати заявка, се изчаква за отговор (пакет от тип 2). На фигура 3.25 може да се види как се осъществява GCM проверка на

достоверността, времева проверка, предпазваща от използване на стари пакети, проверка на самия пакет и как, след успешна валидация, се връща получения ключ.

```
            pkt = rs.recv(4096)
            if len(pkt) < 14:
                continue
            eth_type = struct.unpack('!H', pkt[12:14])[0]
            if eth_type != ETH_P_ARPT:
                continue
            if pkt[14] != 2:
                continue
            src_mac = mac_bytes_to_str(pkt[6:12])
            dst_mac = mac_bytes_to_str(pkt[0:6])
            blob = pkt[15:]

            aad = build_aad(
                src_mac_str=src_mac,
                dst_mac_str=dst_mac
           )

            try:
                plaintext = aes_gcm_decrypt(host_key, blob, aad)
            except ValueError:
                print("[HOST] GCM auth failed - drop")
                return None

            pkt_type  = plaintext[0]
            sender_ip = ip_bytes_to_str(plaintext[1:5])
            target_ip = ip_bytes_to_str(plaintext[5:9])
```

```
            timestamp = struct.unpack('!I', plaintext[9:13])[0]
            key       = plaintext[13:29]
            now = int(time.time())
            if abs(now - timestamp) > 30:
                print("[HOST] stale GW packet - replay drop")
                return None
            print(f"[REQ]    recv    payload={payload[:24].hex()}    at
{time.time():.6f}")
            if pkt_type == 2 and len(payload) >= 19:
                print(f"[REQ] got KEY for {target_ip} key={key.hex()}")
                rs.close()
                return key
```

Фиг. 3.25 Получаване и проверка на пакета

3.2.8 Изпращане на T-ARP заявки (broadcast)

Функцията broadcast_request() отговаря за изпращането на broadcast заявки (фиг. 3.26), като преди това заявява комуникационен ключ от шлюза.

```
     def create_arpt_packet(pkt_type, sender_ip, target_ip, key):
        nonce = get_random_bytes(4)
        ts = struct.pack('!I', int(time.time()))
        core = struct.pack('!B', pkt_type) +
ip_str_to_bytes(sender_ip) + ip_str_to_bytes(target_ip) + nonce + ts
        sig = aes_ecb_encrypt(key, core)[:16]
        return core + sig
                ...
     payload = create_arpt_packet(3, config['ip'], target_ip, key)
     send_eth_raw('ff:ff:ff:ff:ff:ff', payload)
```

Фиг. 3.26 Генериране и изпращане на T-ARP Broadcast пакет

Изчаква се unicast T-ARP отговор (тип 4) и след успешна верификация на данните, функцията връща получения MAC адрес (фиг. 3.27).

```
                try:
                    src_mac, payload, raw =
reply_queue.get(timeout=wait_left)
                except queue.Empty:
                    break
                pkt_type = payload[0]
                sender_ip = ip_bytes_to_str(payload[1:5])
                sig = payload[17:33]
                if pkt_type != 4 or sender_ip != target_ip:
                    continue
                calc_input = payload[:17]
                calc_sig = aes_ecb_encrypt(key, calc_input)[:16]
                if calc_sig != sig:
                    print(f"[T-ARP] signature mismatch from
{sender_ip} (dropping).")
                    continue
                print(f"[T-ARP] got valid reply from {sender_ip}
mac={src_mac} (recv at {time.time():.6f})")
                return src_mac
```

3.27 Верификация на T-ARP DISCOVER Reply

3.2.9 Обработка на входящи T-ARP заявки

Във функция arpt_request_handler() се приемат и обработват пакети от тип 3. В нея се проверява достоверността на пакета, като се изпраща заявка до шлюза за ключа, с който да се верифицира пакета. Отговор се изпраща само ако проверката се извърши успешно (фиг. 3.28).

```
        rc_mac, payload, raw = request_queue.get()
            try:
                pkt_type = payload[0]
                sender_ip = ip_bytes_to_str(payload[1:5])
                target_ip = ip_bytes_to_str(payload[5:9])
                sig = payload[17:33]
            except Exception as e:
                print("[HANDLER] malformed arpt payload", e)
                continue
            if pkt_type != 3:
                continue
            if target_ip != config['ip']:
                continue
            print(f"[HANDLER] Received T-ARP request from {sender_ip}
({src_mac}) targeting us")

            key_for_pair = request_key_from_gw(sender_ip, timeout=6)
            if not key_for_pair:
                print("[HANDLER] no key from GW for sender, cannot
verify -> still will send reply (policy)")
            calc_input = payload[:17]
            calc_sig = aes_ecb_encrypt(key_for_pair, calc_input)[:16]
            if calc_sig != sig:
                continue

            reply_payload = create_arpt_packet(4, config['ip'],
sender_ip, key_for_pair)
            send_eth_raw(src_mac, reply_payload)
```

Фиг. 3.28 Функция arpt_request_handler

3.2.10 Основна функция main()

В основната функция main() се активира promiscuous режим и се стартират нишките на приложението (фиг. 3.29).

```
def main():
    set_promisc(IFACE)
    t = threading.Thread(target=sniffer, daemon=True)
    t.start()
    # handler thread - processes incoming T-ARP requests and
replies
    th = threading.Thread(target=arpt_request_handler,
daemon=True)
    th.start()
```

Фиг. 3.29 Активиране на promiscuous режим и стартиране на нишките

След като се извърши инициализацията, се приема избор на устройство за симулиране на протокола и се изпълнява T-ARP комуникация с него (фиг. 3.30).

```
        while True:
            try:
                target = input("Target IP> ").strip()

                if not target:
                    continue

                mac = broadcast_request(target, wait_timeout=8)

                if mac:
                    print(f"[MAIN] Success: {target} -> {mac}")
                else:
                    print(f"[MAIN] Failed to get T-ARP reply for
{target}")

            except KeyboardInterrupt:
                print("\n[MAIN] Shutting down")
                break
            except Exception as e:
                print(f"[MAIN] Error: {e}")
```

Фиг. 3.30 Тестов цикъл

Четвърта глава: Използвани ресурси за верификация на T-ARP в емулирана мрежова инфраструктура

4.1 Цел и обхват на верификацията

Целта на настоящата глава е да представи методологията, инструментите и средата, използвани за верификация на разработения Trusted ARP (T-ARP) механизъм. Основният фокус е поставен върху доказване на функционалната коректност, стабилността и практическата приложимост на решението в контролирана, но реалистична мрежова среда.

Верификацията има за задача да отговори на следните ключови въпроси:

1. Работи ли T-ARP коректно в мрежа с реален мрежов стек?
2. Успешно ли предотвратява класически ARP spoofing атаки?
3. Може ли решението да бъде интегрирано в съществуваща мрежова операционна система?
4. Подходящо ли е решението за реална употреба, а не само за теоретичен модел?

За да се постигне това, е използвана емулирана мрежова инфраструктура, която позволява възпроизвеждане на реални сценарии без необходимост от физически мрежови устройства.

4.2 Избор на подход за тестване

4.2.1 Предимства на емулираната инфраструктура

Емулираната среда е предпочетена пред физическа инфраструктура поради наличието на пълен контрол върху мрежовата топология и възможността за бързо създаване и модифициране на сценарии. Друго голямо предимство е възможността за безопасно симулиране на злонамерени атаки и лесното възпроизвеждане на експерименти. Тези характеристики правят емулацията особено подходяща за тестване на мрежови протоколи за сигурност, какъвто е T-ARP.

4.2.2 GNS3

GNS3 е използван като основна платформа за изграждане на тестовата мрежова инфраструктура. Той позволява изпълнението на реални операционни системи и мрежови услуги в изолирана виртуална среда.

Основните предимства на GNS3 в контекста на настоящата дипломна работа са:

1. Поддръжка на Linux-базирани мрежови операционни системи.
2. Възможност за Layer 2 и Layer 3 емулация.
3. Интеграция с виртуални машини, симулиращи крайни устройства.
4. Наблюдение на трафика в реално време.

Чрез GNS3 се осигурява среда, която максимално се доближава до реална локална мрежа, без компромис с гъвкавостта и контрола.

4.2.3 Избор на VyOS като платформа за интеграция

VyOS е избрана като платформа за интеграция на T-ARP поради своята архитектура и предназначение. Тя представлява Linux-базирана мрежова операционна система с отворен код, която предоставя директен достъп до мрежовия стек, възможност за работа с raw sockets, systemd-базирано управление на услуги и централизирана конфигурация. Тези характеристики я правят подходяща не само за лабораторни цели, но и за реална мрежова експлоатация, което е ключово за доказване на практическата стойност на разработката.

4.2.4 Използване на VMware за емулация на крайни устройства

VMware е платформа за виртуализация, която позволява създаването и изпълнението на виртуални машини върху физическо устройство. В контекста на настоящата разработка VMware се използва за емулация на крайните устройства и на шлюза, които участват в T-ARP комуникацията в тестовата инфраструктура.

Основното предимство на VMware е, че виртуалните машини използват реален Linux мрежов стек, което осигурява автентично поведение на ARP протокола и свързаните с него атаки.

Чрез VMware всяко виртуално устройство разполага със собствен:

- IP адрес;
- MAC адрес;
- Мрежов интерфейс.

VMware виртуалните машини са свързани към емулираната мрежова топология чрез GNS3, което позволява включване на VMware виртуални устройства като пълноправни участници в локалната мрежа и директна Layer 2 комуникация с VyOS и другите възли.

4.2.5 Използване на Wireshark за анализ на мрежовия трафик

Wireshark е широко използван анализатор на мрежов трафик, предназначен за прихващане, визуализация и анализ на пакети на различни мрежови нива. В настоящата дипломна работа Wireshark се използва като основен инструмент за наблюдение и верификация на T-ARP трафика.

Инструментът позволява детайлен преглед на всички полета в T-ARP кадъра, което се използва за доказване на сигурността на пакетите и комуникацията.

4.3 Архитектура на тестовата инфраструктура

4.3.1 Основни компоненти

Тестовата инфраструктура включва следните основни компоненти:

4.3.1.1 T-ARP Gateway (VyOS)

Централен възел (шлюз), на който е инсталиран VyOS с разработеният T-ARP демон. Той отговаря за разпределението на ключове в локалната мрежа.

4.3.1.2 Легитимни устройства

Представляват стандартни Linux клиенти, работещи на alpine linux, които участват в нормална T-ARP комуникация.

4.3.1.3 Виртуален суич

Осигурява Layer 2 свързаност между всички възли в инфраструктурата.

4.3.2 Логическа топология

Всички възли са разположени в една локална мрежа (192.168.1.0/24), което позволява директно ARP взаимодействие. T-ARP шлюзът функционира като доверена точка за валидиране, без да нарушава стандартния мрежов поток.

Пета глава: Верифициране на работоспособността

5.1 Инициализация и конфигурация на GNS3

5.1.1 Инсталация на GNS3 [8]

GNS3 има различен метод на инсталация за различните операционни системи. Разработката в настоящата дипломна работа е извършена на устройство с операционна система Linux Mint, като по-долу е описан специфичния процес за инсталация и конфигурация в тази среда.

Тъй като Linux Mint е базиран на Ubuntu и използва apt package manager, инсталацията му се извършва посредством следните команди:

sudo add-apt-repository ppa:gns3/ppa

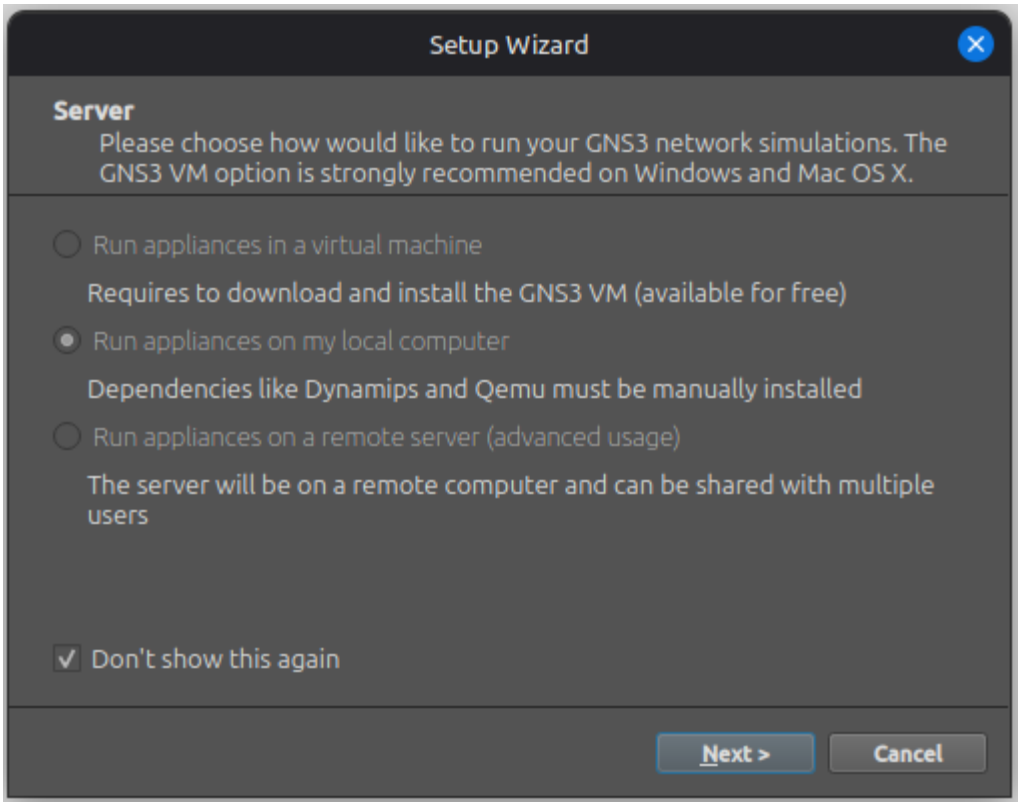
sudo apt update

sudo apt install gns3-gui gns3-server

След като инсталацията приключи, стартирането се извършва със следната команда:

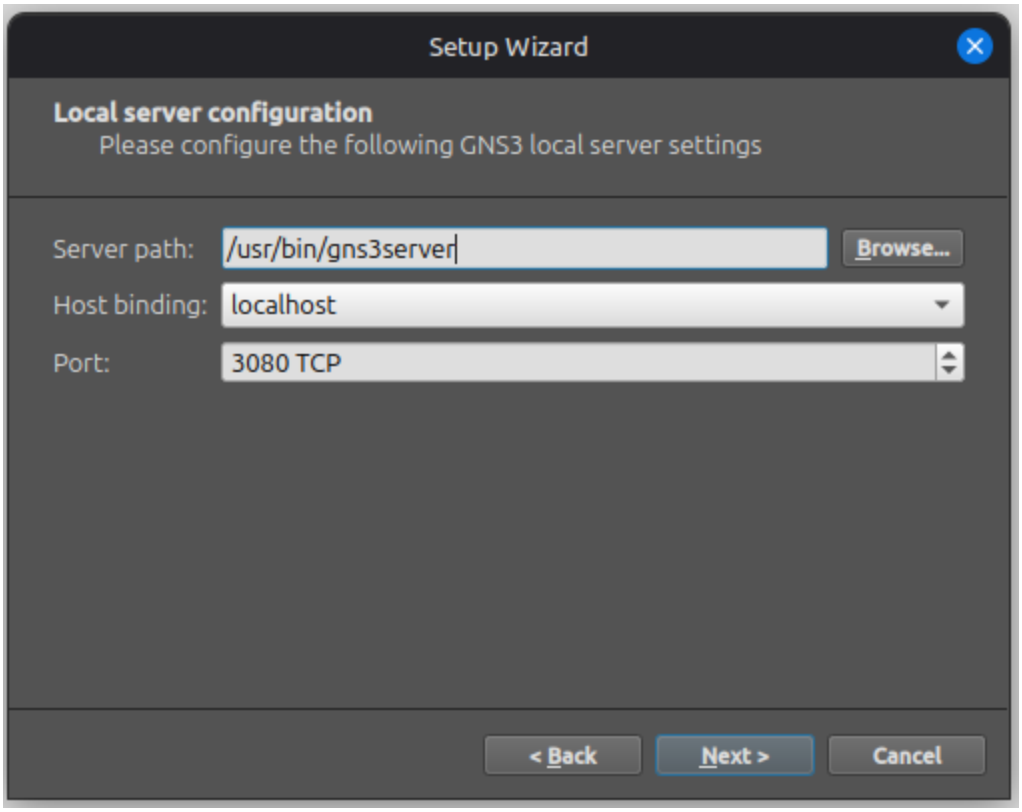
gns3

При първоначално влизане в програмата излиза прозорец за конфигурация (фиг. 5.1), в който има избор къде да се изпълни GNS3 сървъра, който отговаря за изпълнението на симулациите.



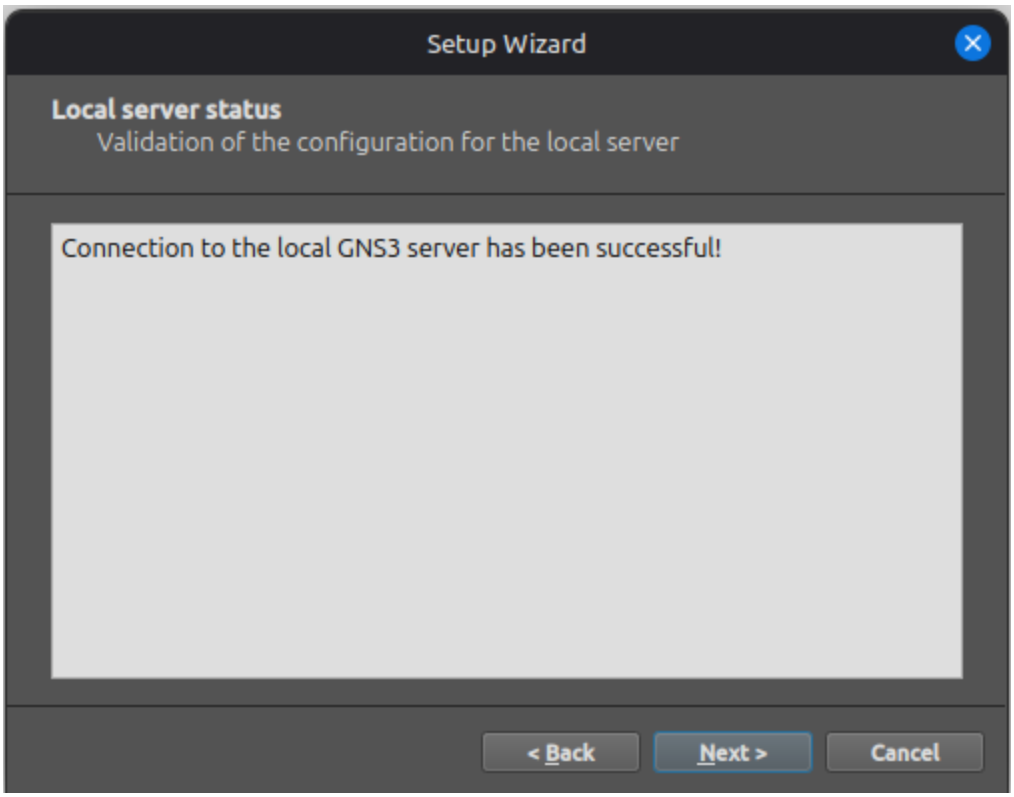
Фиг. 5.1 Конфигурационен прозорец на GNS3

Ако не се очаква да се симулират много големи мрежови инфраструктури, най-оптимално е да се избере сървърът да е локален, тъй като локалният сървър разполага с пълните функционалности само когато системата е базирана на linux дистрибуция, а при MacOS и Windows има ограничени възможности. След натискане на бутона Next има възможност за конфигурация на самия сървър (IP, Port и път към файла) (фиг. 5.2).



Фиг. 5.2 Конфигурация на сървъра

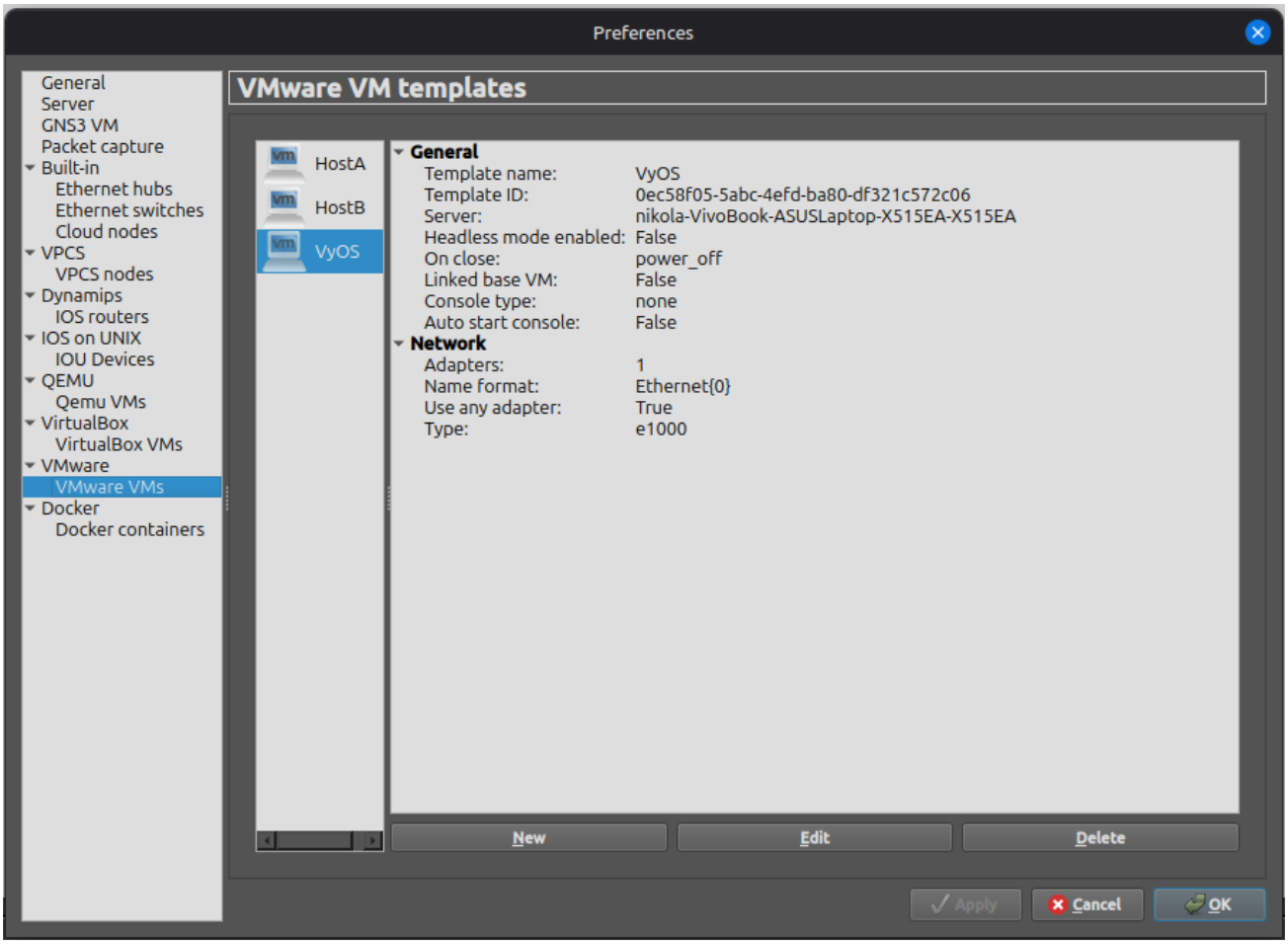
След това се показва прозорец, който потвърждава успешното свързване към сървъра (фиг. 5.3).



Фиг. 5.3 Успешна инициализация и свързване към GNS3 сървъра

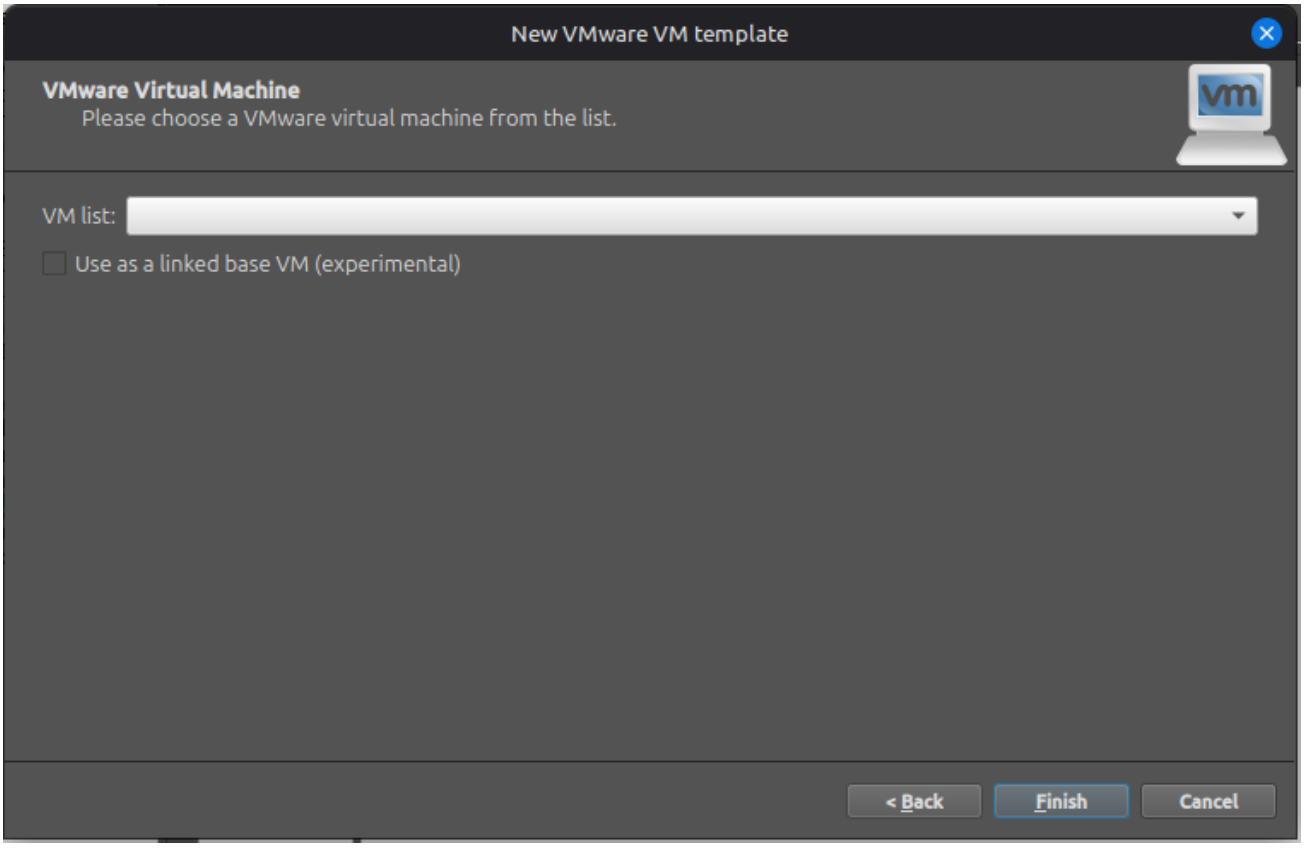
5.1.2 Свързване на VMware виртуални машини към GNS3

GNS3 позволява свързване с VMware, за да използва вече създадени виртуални машини като устройства в симулираната мрежа. Менюто за добавяне на нови виртуални машини се намира под **Edit -> Preferences -> VMware -> VMware VMs**. В менюто се показват свързаните виртуални машини, а под тях има опции за добавяне на нова, редактиране на вече съществуваща и изтриване на избрана (фиг. 5.4).



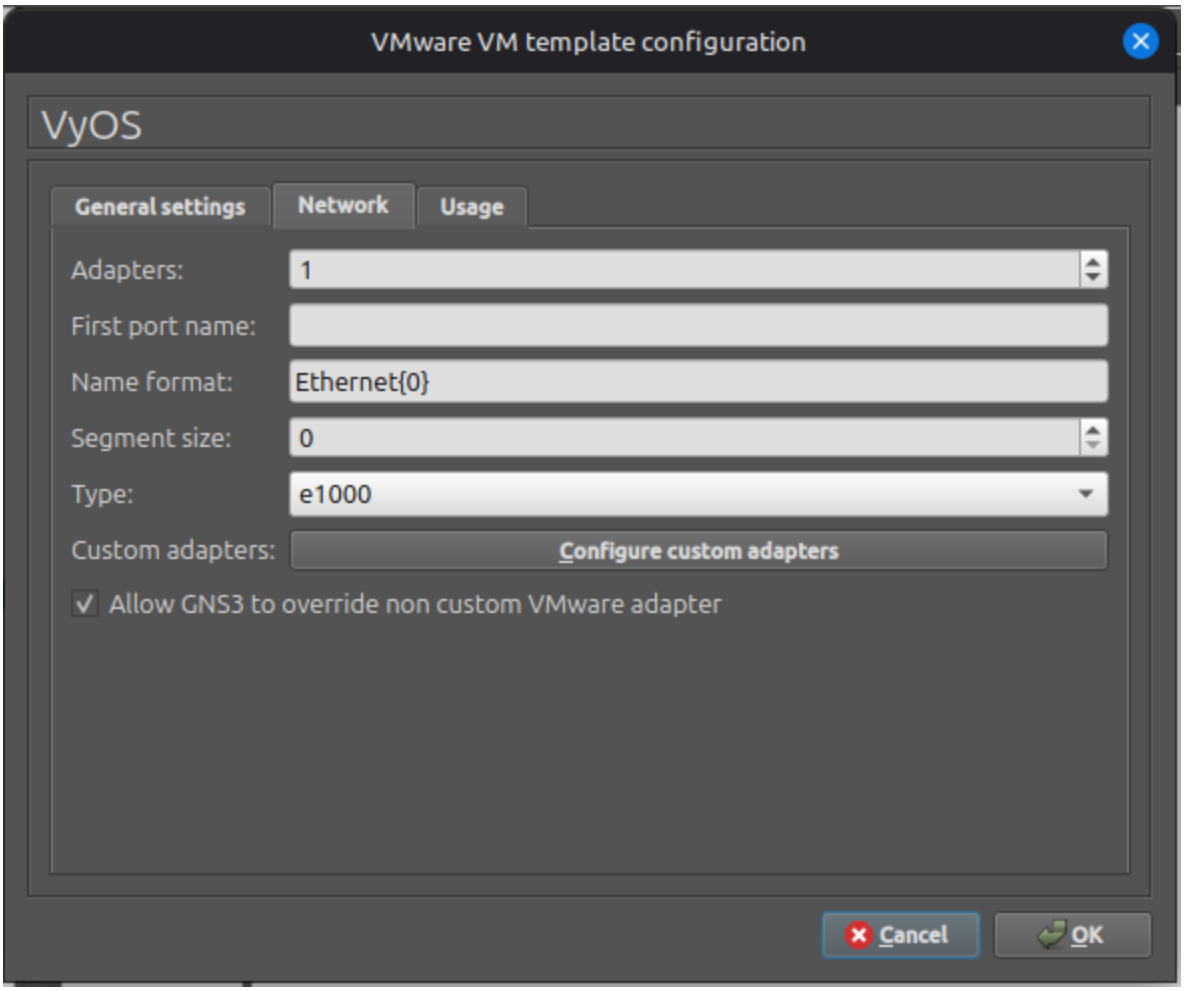
Фиг. 5.4 Меню за конфигурация и добавяне на нови VMware виртуални машини

При свързване на нова виртуална машина се визуализира диалога, показан на фигура 5.5, където се избира от списъка с несвързани виртуални машини.



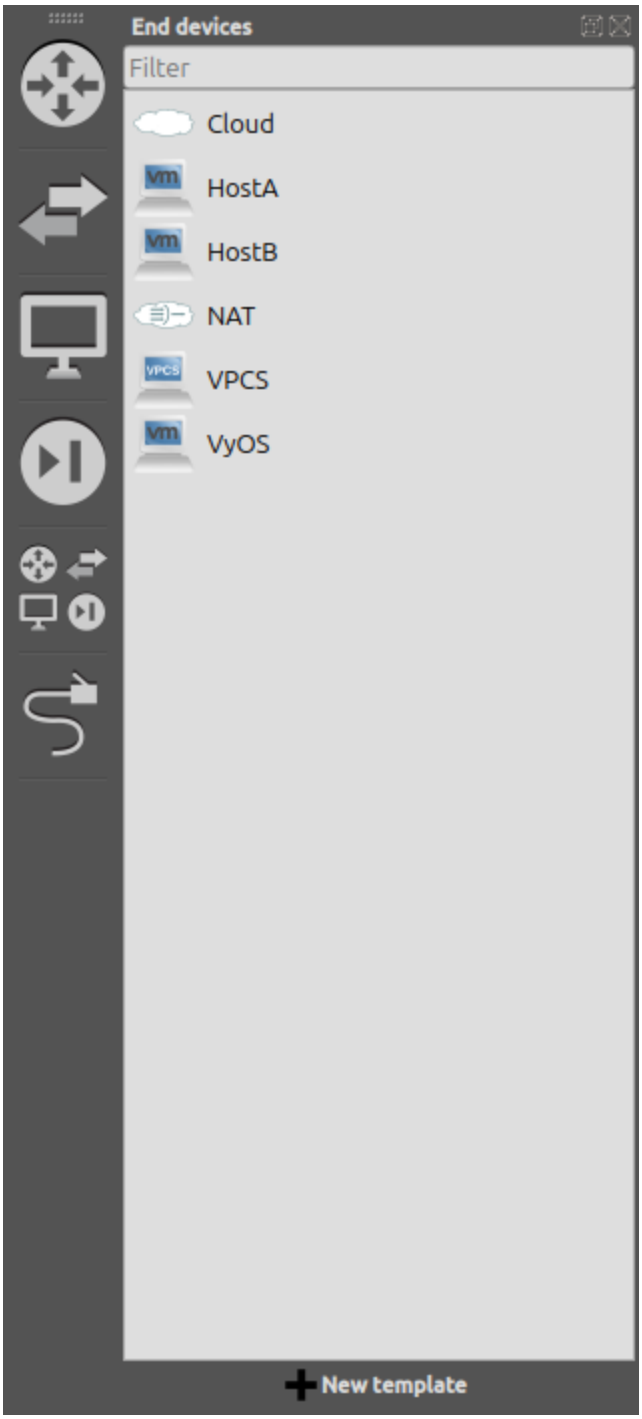
Фиг. 5.5 Прозорец за свързване на нова виртуална машина

След като се свърже нова виртуална машина, трябва да се позволи на GNS3 да променя настройките за мрежовия адаптер, като се избере опцията “Allow GNS3 to override non custom VMware adapter” и се приложи конфигурацията (фиг. 5.6).



Фиг. 5.6 Меню за промяна на мрежовите настройки на виртуалната машина

След това виртуалната машина вече може да бъде включена към мрежовата симулация и ще се покаже в полето, от където се добавят крайни устройства (фиг. 5.7).



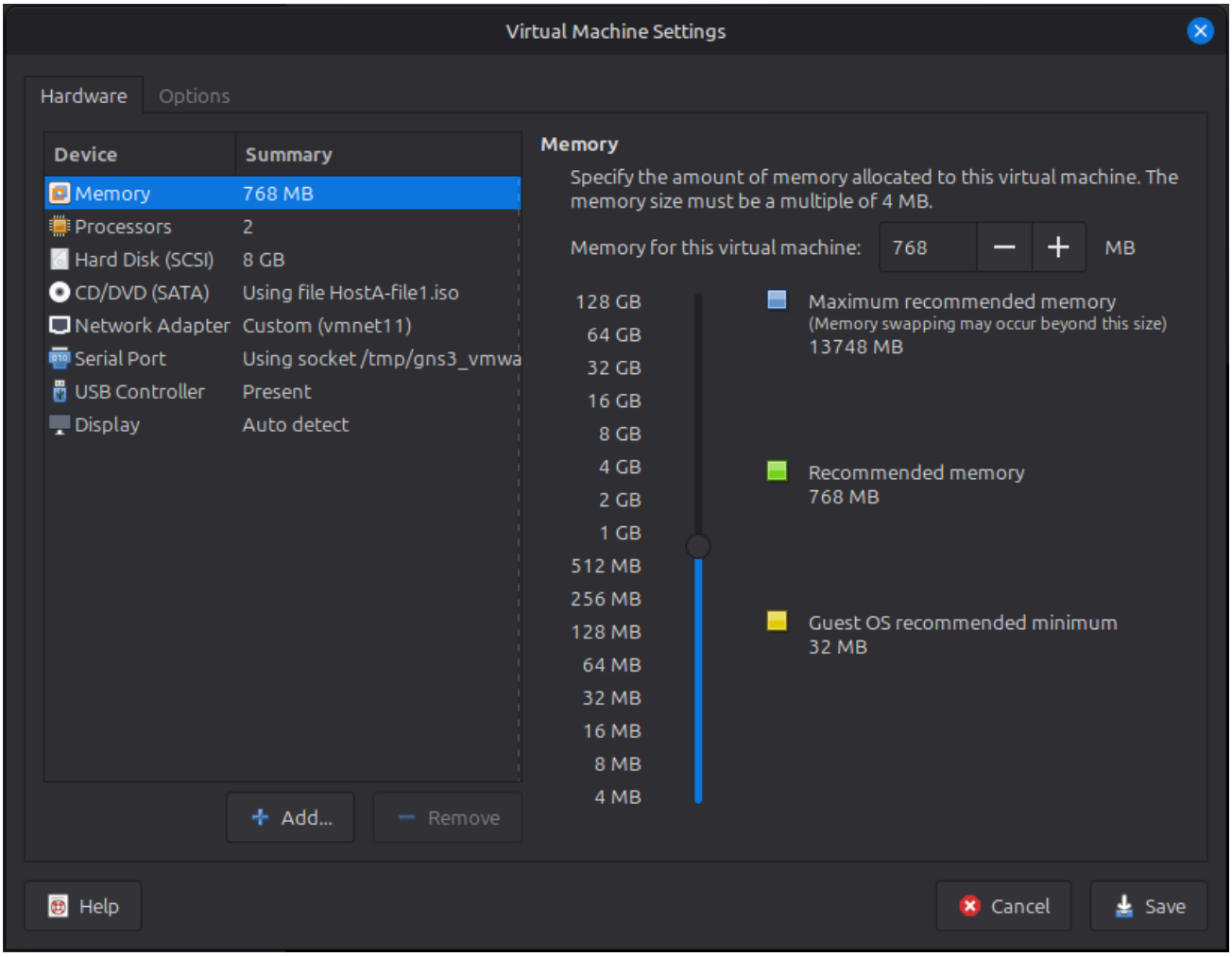
Фиг. 5.7 Меню за добавяне на крайни устройства в симулацията

5.2 Създаване и конфигурация на VMware виртуалните машини

5.2.1 Виртуални машини, използвани за крайни устройства в симулацията

Крайните устройства работят на alpine linux операционна система, която може да се свали от официалния сайт на разработката [9].

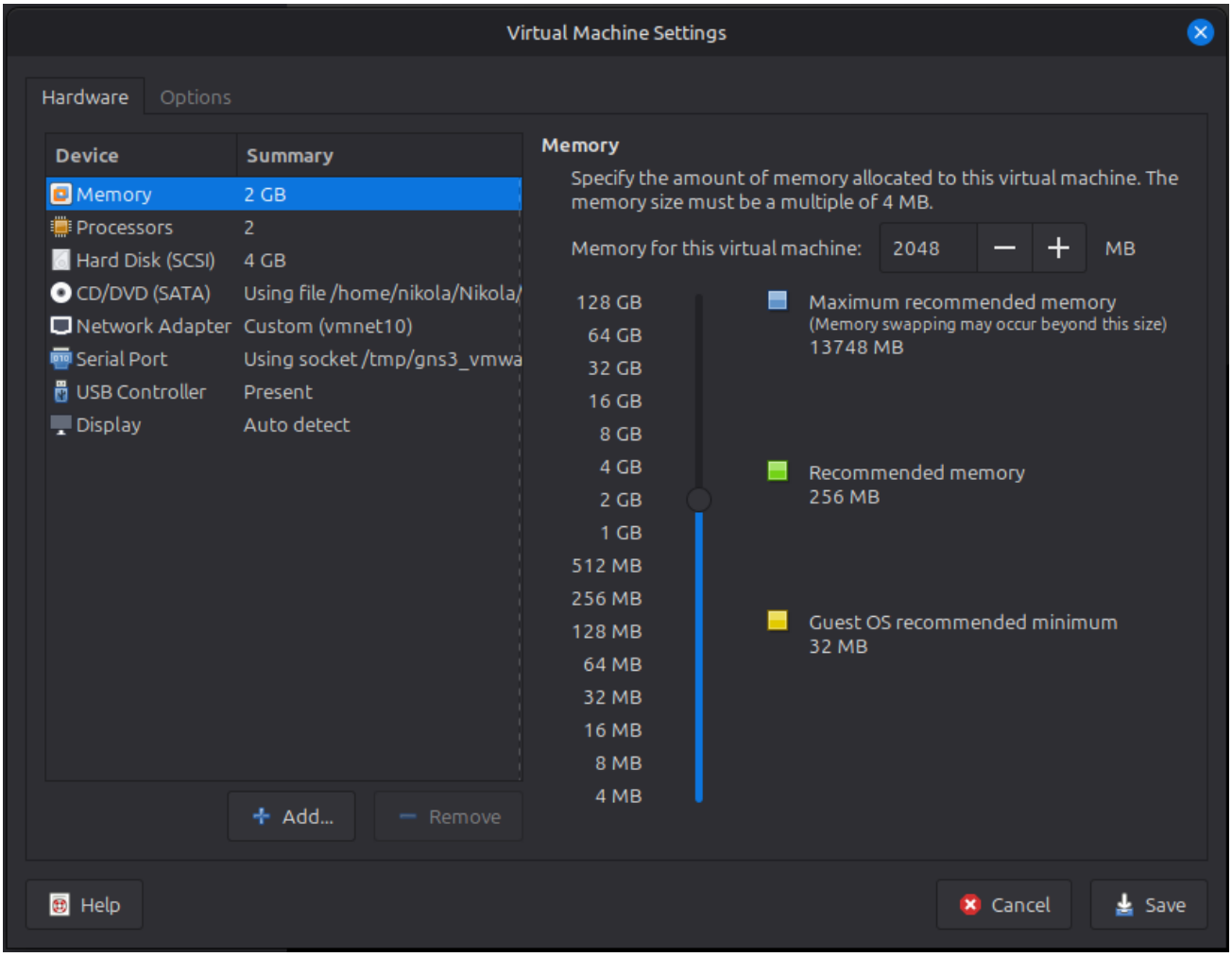
Тъй като alpine linux е операционна система, която не изисква много изпълнителни ресурси, при създаването на виртуалните машини е достатъчно да се заделят ограничени ресурси - като пример при симулацията за настоящата дипломна работа са конфигурирани максимално по 768 MB оперативна памет, 2 виртуални процесора и по 8 GB виртуален хард диск (фиг. 5.8). При избирането на ISO се селектира сваления alpine linux .iso файл.



5.8 Заделени ресурси за виртуалните машини на крайните устройства

5.2.2 Виртуална машина на VyOS

За виртуалната машина, която симулира VyOS рутера, са конфигурирани 2GB оперативна памет, 2 виртуални процесора и 4GB виртуален хард диск (фиг. 5.9).

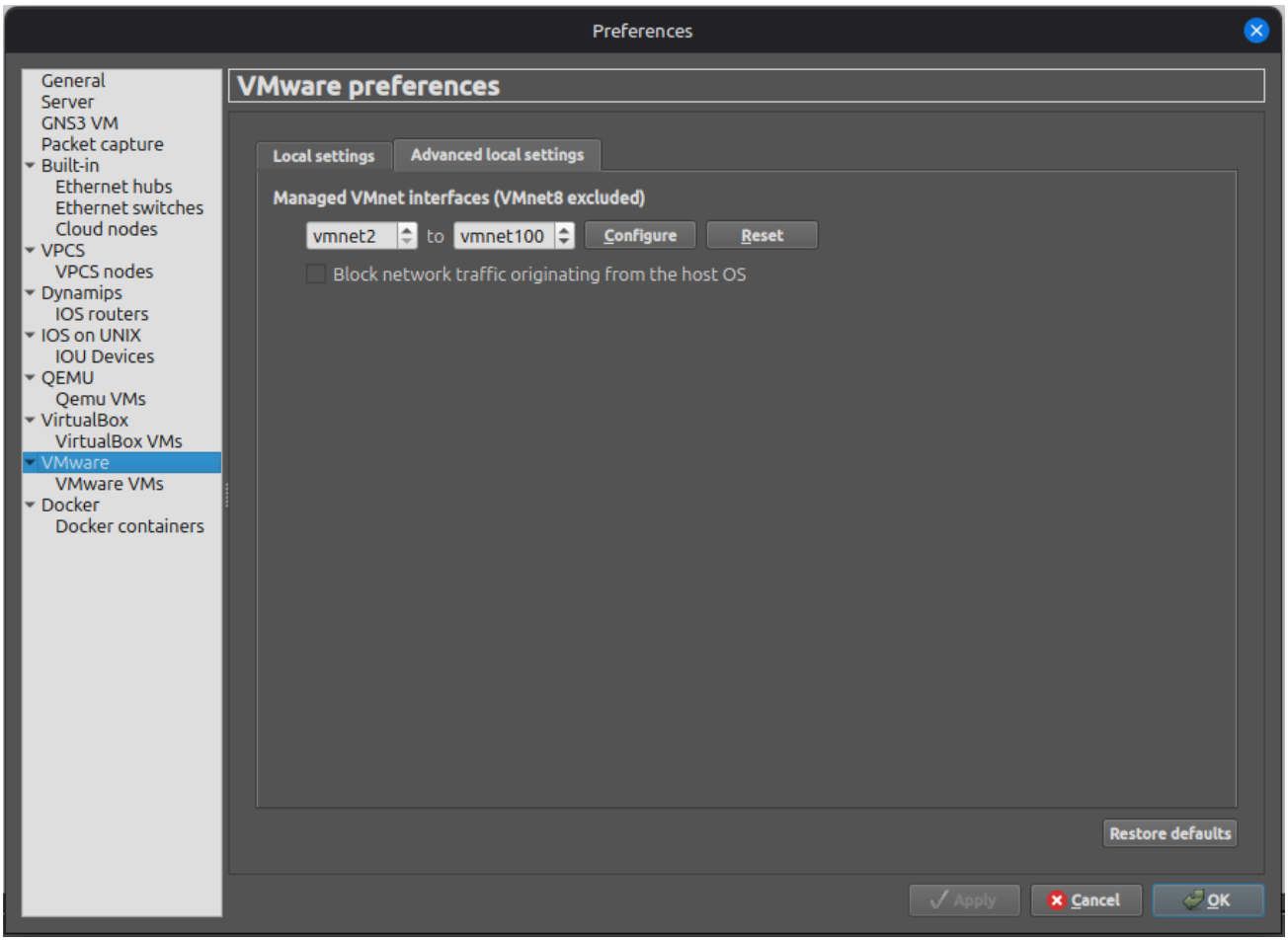


Фиг. 5.9 Заделени ресурси за VyOS виртуалната машина

5.2.3 Конфигурация на vmnets (виртуални мрежи) във VMware

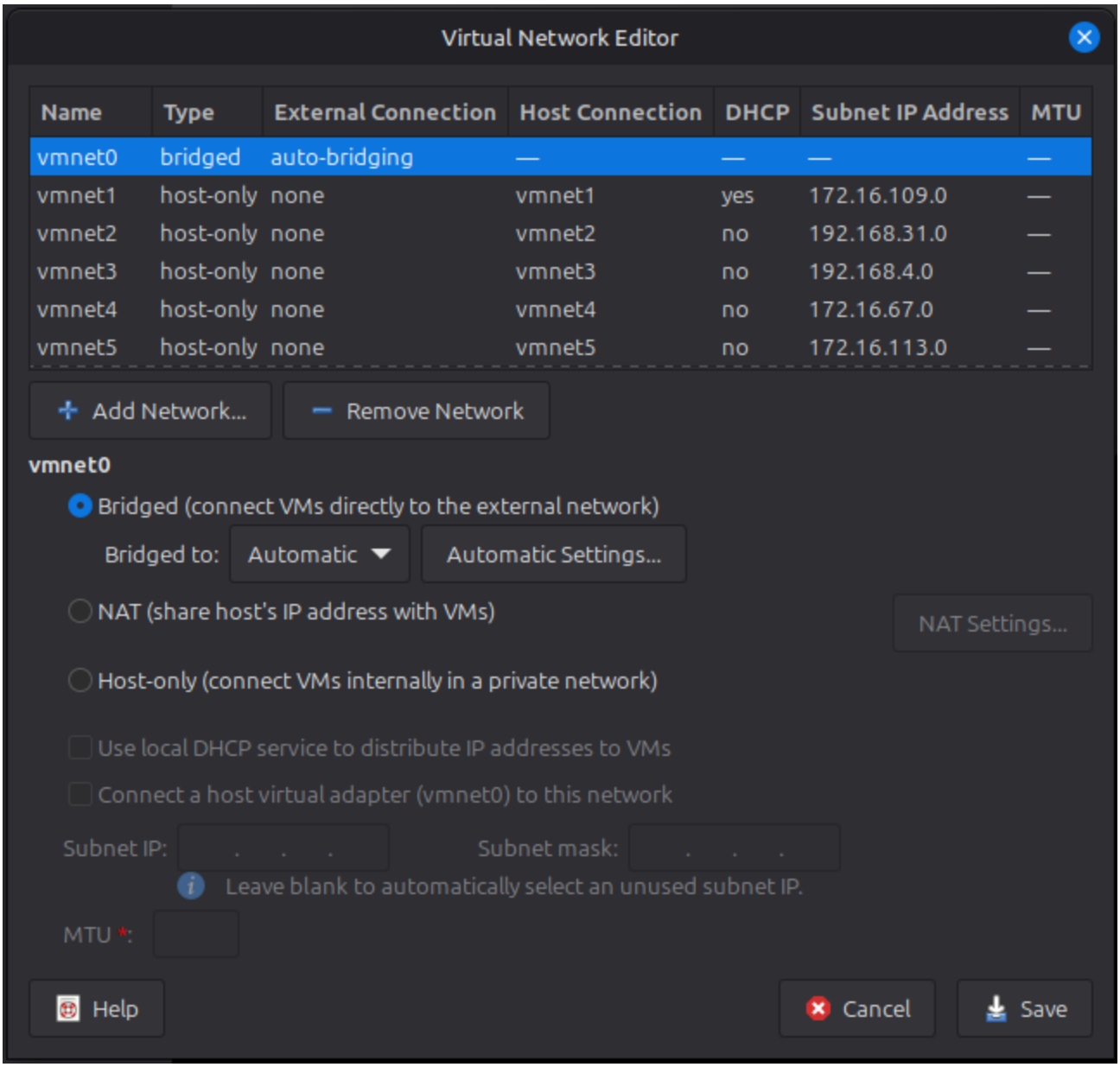
За да може устройствата в GNS3 симулацията да си комуникират, необходимо е да се настроят виртуалните мрежи на VMware.

GNS3 има функционалност, която го извършва автоматично и може да бъде намерена в **Edit -> Preferences -> VMware -> Advanced Local Settings** (фиг. 5.10), където има опция “configure”, която автоматично създава и конфигурира 100 виртуални мрежи.



Фиг. 5.10 Прозорец за конфигурация на виртуални мрежи

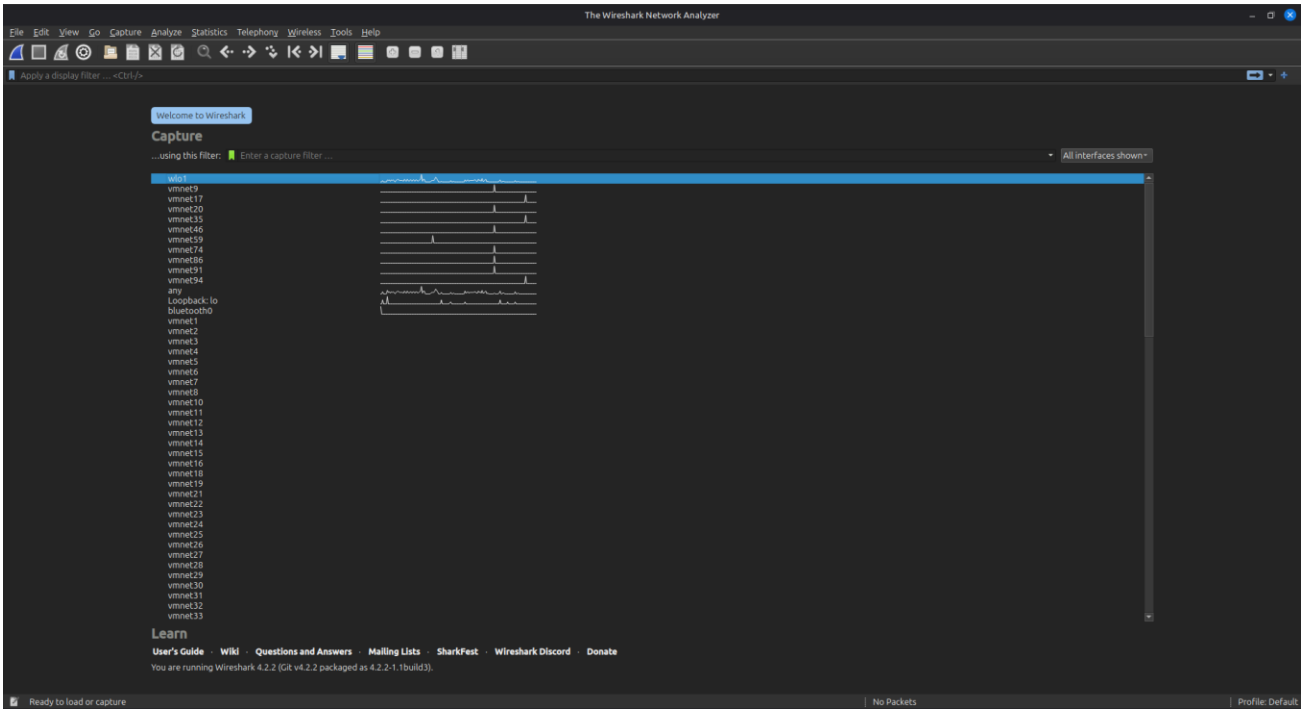
Всички виртуални мрежи във VMware могат да се видят в меню **Edit -> Virtual Network Editor** (фиг. 5.11).



Фиг. 5.11 Прозорец за показване, създаване и промяна на виртуалните мрежи на VMware

5.3 Наблюдаване на мрежовия трафик с Wireshark

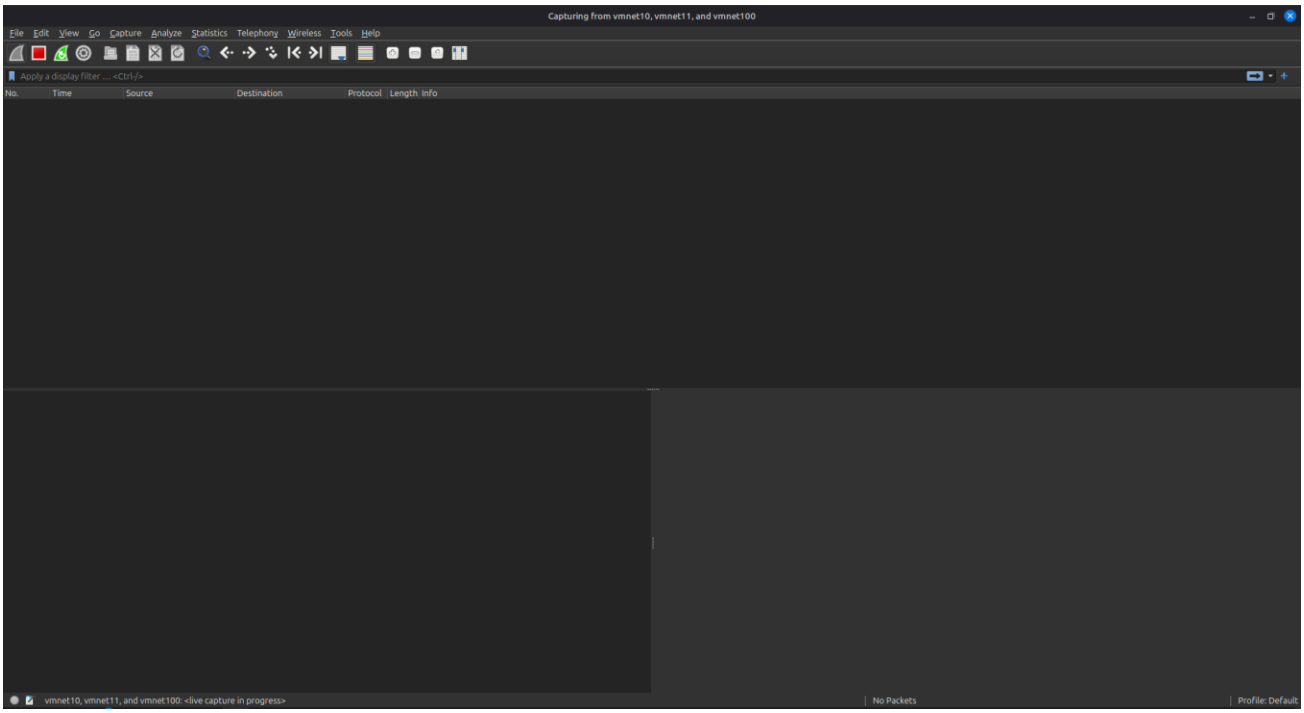
За наблюдение на мрежовия трафик в дадена реална или виртуална мрежа се използва Wireshark. При стартиране на Wireshark се показва меню с всички активни мрежови интерфейси и виртуални мрежи, които могат да бъдат избрани за наблюдение (фиг. 5.12).



Фиг. 5.12 Начална страница на Wireshark

След като се изберат мрежите за наблюдение, се отваря нов диалог (фиг. 5.13), в който се показват заснетите пакети в съответните мрежи. Има поле за филтриране на пакетите по различни параметри, като вид на протокола, изпращач, получател и други.

GNS3 поставя трите виртуални машини в три виртуални мрежи, в които имаме интерес да следим трафика (vmnet 10, vmnet 11 и vmnet 100).

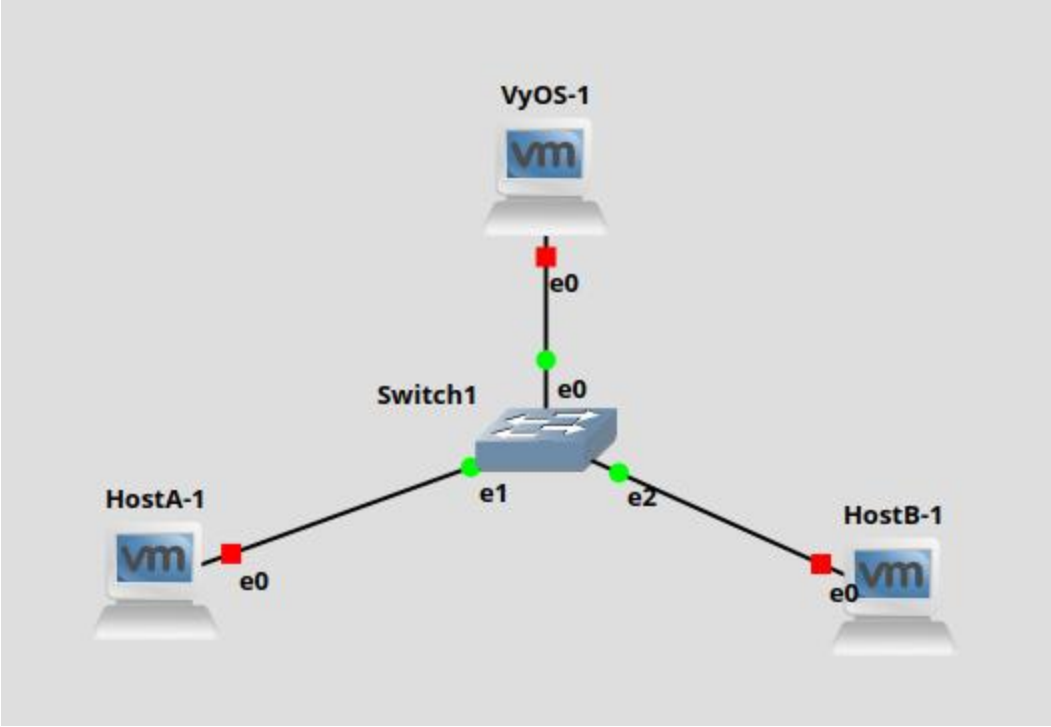


Фиг. 5.13 Страница за наблюдение на трафик

5.4 Симулация на T-ARP

5.4.1 Създаване на GNS3 симулацията

За да можем да симулираме работата на T-ARP, необходимо е да се създаде нов GNS3 проект, в който да се поставят и свържат избраните VMware виртуални машини, както е показано на фигура 5.14.



Фиг. 5.14 Подредба на GNS3 симулацията

След това симулацията може да бъде стартирана със зеления бутон в менюто с функции (фиг. 5.15). При стартиране се отваря VMware и всичките използвани виртуални машини също се стартират.

Фиг. 5.15 Меню с функции в GNS3

5.4.2 Конфигурация на виртуалните машини след включване

5.4.2.1 Конфигурация на крайните устройства

След стартиране на операционните системи на крайните устройства се изисква въвеждане на потребител и парола. За целта на симулацията е използван root потребителя, който е конфигуриран без парола (Това не е добра практика, но в контролирана тестова среда е приемливо).

За да сложим статични IP адреси на устройствата, можем директно да променим **/etc/network/interfaces** файла (фиг. 5.16).

```
auto lo
iface lo inet loopback

auto eth0
#iface eth0 inet dhcp
iface eth0 inet static
        address 192.168.1.100
        netmask 255.255.255.0
        gateway 192.168.1.1
```

Фиг. 5.16 Конфигурационен файл, отговарящ за настройките на мрежовите интерфейси

Файлът, симулиращ T-ARP, се намира в **/opt/tarp/tarp_host.py**, а конфигурацията - в **/opt/tarp/config.json**. За да се стартира файлът се използва:

python3 /opt/tarp/tarp_host.py

Тази команда изпълнява скрипта и се показва поле за въвеждане на желания IP адрес, с който да се симулира комуникацията (фиг. 5.17).

```
HostA:~# python3 /opt/tarp/tarp_host.py
[STARTUP] Host IP=192.168.1.100 hw_mac=00:0c:29:6f:db:3f configured-mac=00:0c:29:6f:db:3f
[MAIN] T-ARP node started
[MAIN] Enter a target IP to send an T-ARP request (Ctrl+C to quit)
Target IP> _
```

Фиг. 5.17 Изход при изпълняване на скрипта за симулация на крайно устройство

5.4.2.2 Конфигурация на VyOS рутера

След стартиране на VyOS операционната система се изисква въвеждането на потребител и парола. За целта на симулацията съм оставил потребителя и паролата по подразбиране, като и двете са “vyos” (Това не е добра практика, но в контролирана тестова среда е приемливо).

За да може рутерът да комуникира с останалите устройства в мрежата, необходимо е да се конфигурира неговия мрежови адрес. Тъй като VyOS следва подобна на Juniper структура на командите, за конфигуриране на адреса се изпълняват следните команди:

vyos@vyos:~$ config

С “config” се влиза в конфигурационен режим.

vyos@vyos# set interfaces ethernet eth0 address 192.168.1.99/24

Задаваме на интерфейсът “eth0” на рутера IP адреса 192.168.1.99 от мрежата 192.168.1.0/24

vyos@vyos# commit

Използва се, за да се запише конфигурацията и след това можем с

vyos@vyos# exit

да излезем от конфигурационния режим. С командата

vyos@vyos:~$ show interfaces

можем да проверим дали конфигурацията се е запазила коректно.

След конфигурирането на интерфейса, трябва да се стартира T-ARP Gateway процеса. Използват се следните команди:

vyos@vyos:~$ sudo systemctl enable vyos-tarp-gw

По този начин се създава symbolic link, така че услугата да се стартира автоматично при boot.

vyos@vyos:~$ sudo systemctl restart vyos-tarp-gw

стартира процеса. Използвайки командата

vyos@vyos:~$ sudo systemctl status vyos-tarp-gw

можем да видим статуса на процеса (фиг. 5.18).

```
vyos@vyos:~$ sudo systemctl status vyos-tarp-gw
● vyos-tarp-gw.service - VyOS T-ARP Gateway Daemon
     Loaded: loaded (/lib/systemd/system/vyos-tarp-gw.service; enabled; preset:>
     Active: active (running) since Tue 2026-02-17 18:03:40 UTC; 7s ago
   Main PID: 2742 (vyos-tarp-gw)
      Tasks: 1 (limit: 2279)
     Memory: 13.7M
        CPU: 89ms
     CGroup: /system.slice/vyos-tarp-gw.service
             └─2742 /usr/bin/python3 /usr/bin/vyos-tarp-gw

Feb 17 18:03:40 vyos systemd[1]: Started VyOS T-ARP Gateway Daemon.
Feb 17 18:03:40 vyos vyos-tarp-gw[2742]: 2026-02-17 18:03:40,631 [INFO] T-ARP G>
lines 1-12/12 (END)
```

Фиг. 5.18 Изход на командата sudo systemctl status vyos-tarp-gw

За да се види журналния файл на процеса се използва:

vyos@vyos:~$ journalctl -u vyos-tarp-gw -f

5.4.3 Тестване на работоспособността

За да се покаже че протоколът работи, както е предвидено, от HostA ще симулираме T-ARP комуникация към HostB.

Адресите на устройствата в симулацията са:

HostA eth0:

IP - 192.168.1.100

MAC - 00:0c:29:6f:db:3f

HostB eth0:

IP - 192.168.1.101

MAC - 00:0c:29:0b:5c:c6

VyOS Router eth0:

IP - 192.168.1.99

MAC - 00:0c:29:4a:a3:aa

Въвеждаме адреса на HostB от HostA и в терминала на HostA получаваме изхода, показан на фигура 5.19, а в терминала на HostB получаваме изхода, показан на фигура 5.20.

```
HostA:~# python3 /opt/tarp/tarp_host.py
[STARTUP] Host IP=192.168.1.100 hw_mac=00:0c:29:6f:db:3f configured-mac=00:0c:29:6f:db:3f
[MAIN] T-ARP node started
[MAIN] Enter a target IP to send an T-ARP request (Ctrl+C to quit)
Target IP> 192.168.1.101
[T-ARP] beginning broadcast_request for 192.168.1.101: retries=3, wait=8s
[REQ] sent REQ for 192.168.1.101 to GW 00:0c:29:4a:a3:aa at 1771353006.284332
[T-ARP] attempt 1: broadcast sent at 1771353006.308970 for 192.168.1.101
----- VERIFICATION DEBUG START -----
  expected sender_ip: 192.168.1.101, target_ip: 192.168.1.101
  core (len 17): 04c0a80165c0a80164a008a48a6994b3b3
  received signature: dd9ac4771c68ec6d0abc27bea8ad809d
  calculated signature: dd9ac4771c68ec6d0abc27bea8ad809d
  nonce: a008a48a ts: 1771353011 (Tue Feb 17 18:30:11 2026)
----- VERIFICATION DEBUG END -----
[T-ARP] got valid reply from 192.168.1.101 mac=00:0c:29:0b:5c:c6 (recv at 1771353006.397448)
[MAIN] Success: 192.168.1.101 -> 00:0c:29:0b:5c:c6
Target IP> _
```

Фиг. 5.19 Изход в терминала на HostA

```
HostB:~# python3 /opt/tarp/tarp_host.py
[STARTUP] Host IP=192.168.1.101 hw_mac=00:0c:29:0b:5c:c6 configured-mac=00:0c:29:0b:5c:c6
[MAIN] T-ARP node started
[MAIN] Enter a target IP to send an T-ARP request (Ctrl+C to quit)
Target IP> [HANDLER] Received T-ARP request from 192.168.1.100 (00:0c:29:6f:db:3f) targeting us
[REQ] sent REQ for 192.168.1.100 to GW 00:0c:29:4a:a3:aa at 1771353011.448119
[HANDLER] Sent T-ARP reply to 192.168.1.100 (00:0c:29:6f:db:3f)
```

Фиг. 5.20 Изход в терминала на HostB

От изхода можем да видим, че HostA изпраща заявка към шлюза, за да получи ключ за комуникация с HostB и после изпраща broadcast request насочен към HostB. HostB получава T-ARP Broadcast пакета и изпраща заявка към шлюза, за да получи ключа за комуникация с HostA. След като го получава, верифицира пакета чрез симетричния MAC и изпраща unicast отговор, който HostA получава, обработва и успешно свързва IP и MAC адреса на HostB.

В Wireshark може да се наблюдава обмена на пакети (фиг. 5.21).

Забележка: В Wireshark при следене на три виртуални мрежи наведнъж пакетите се показват по два пъти, защото се следят и трите виртуални мрежи на веднъж и всеки пакет се показва веднъж в мрежата на изпращача и веднъж в мрежата на получателя.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	VMware_6f:db:3f	VMware_4a:a3:aa	0x88b5	60	Local Experimental Ethertype 1
2	0.003064873	VMware_4a:a3:aa	VMware_6f:db:3f	0x88b5	72	Local Experimental Ethertype 1
3	0.023682948	VMware_6f:db:3f	Broadcast	0x88b5	60	Local Experimental Ethertype 1
4	0.043694572	VMware_0b:5c:c6	VMware_6f:db:3f	0x88b5	60	Local Experimental Ethertype 1

Фиг. 5.21 Обмен на пакети при T-ARP

Пакет 1:

На фигура 5.22 могат да се видят данните на първия пакет от обмена, а на фигура 5.23 е показан хекс-дъмп на пакета. Ясно могат да се видят

некриптираните данни: Destination Mac (6 bytes), Source MAC (6 bytes) и EtherType (2 bytes) на пакета. Следващият байт показва типа на пакета, който в този случай е 01 (T-ARP GET Request). Следват данните от прихванатия пакет, при които си личи, че са криптирани и е невъзможно да се декриптират без съответния ключ.

```
▾ Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface vmnet100, id 0
    Section number: 1
  ▸ Interface id: 0 (vmnet100)
    Encapsulation type: Ethernet (1)
    Arrival Time: Feb 17, 2026 18:40:55.701944494 EET
    UTC Arrival Time: Feb 17, 2026 16:40:55.701944494 UTC
    Epoch Arrival Time: 1771346455.701944494
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.000000000 seconds]
    [Time delta from previous displayed frame: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 60 bytes (480 bits)
    Capture Length: 60 bytes (480 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:data]
▾ Ethernet II, Src: VMware_6f:db:3f (00:0c:29:6f:db:3f), Dst: VMware_4a:a3:aa (00:0c:29:4a:a3:aa)
  ▸ Destination: VMware_4a:a3:aa (00:0c:29:4a:a3:aa)
  ▸ Source: VMware_6f:db:3f (00:0c:29:6f:db:3f)
    Type: Local Experimental Ethertype 1 (0x88b5)
▾ Data (46 bytes)
    Data: 01cad5b127689d8ad5b41625fe05fda2282dcc4f781561b9c0aef1684752e4ebf330b6016147a4dbf2ab00000000
    [Length: 46]
```

Фиг. 5.22 Данни от първия пакет

```
0000  00 0c 29 4a a3 aa 00 0c  29 6f db 3f 88 b5 01 ca   ··)J···· )o·?····
0010  d5 b1 27 68 9d 8a d5 b4  16 25 fe 05 fd a2 28 2d   ··'h···· ·%····(-
0020  cc 4f 78 15 61 b9 c0 ae  f1 68 47 52 e4 eb f3 30   ·Ox·a··· ·hGR···0
0030  b6 01 61 47 a4 db f2 ab  00 00 00 00               ··aG···· ····
```

Фиг. 5.23 Хекс-дъмп на първия пакет

Пакет 2:

На фигура 5.24 могат да се видят данните на първия пакет от обмена, а на фигура 5.25 е показан хекс-дъмп на пакета. Ясно могат да се видят некриптираните данни: Destination Mac (6 bytes), Source MAC (6 bytes) и

EtherType (2 bytes) на пакета. Следващият байт показва типа на пакета, който в този случай е 02 (T-ARP GET Reply). Следват данните от прихванатия пакет, при които си личи, че са криптирани и е невъзможно да се декриптират без съответния ключ. Данните са по-дълги, тъй като съдържат и ключ в тях.

```
▾ Frame 2: 72 bytes on wire (576 bits), 72 bytes captured (576 bits) on interface vmnet100, id 0
    Section number: 1
  ▸ Interface id: 0 (vmnet100)
    Encapsulation type: Ethernet (1)
    Arrival Time: Feb 17, 2026 18:40:55.705009367 EET
    UTC Arrival Time: Feb 17, 2026 16:40:55.705009367 UTC
    Epoch Arrival Time: 1771346455.705009367
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.003064873 seconds]
    [Time delta from previous displayed frame: 0.003064873 seconds]
    [Time since reference or first frame: 0.003064873 seconds]
    Frame Number: 2
    Frame Length: 72 bytes (576 bits)
    Capture Length: 72 bytes (576 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:data]
▾ Ethernet II, Src: VMware_4a:a3:aa (00:0c:29:4a:a3:aa), Dst: VMware_6f:db:3f (00:0c:29:6f:db:3f)
  ▸ Destination: VMware_6f:db:3f (00:0c:29:6f:db:3f)
  ▸ Source: VMware_4a:a3:aa (00:0c:29:4a:a3:aa)
    Type: Local Experimental Ethertype 1 (0x88b5)
▾ Data (58 bytes)
    Data: 0208918a1ece792ca55bac02c9bf9a3ab34728ddbe90d7301c4a17b48b7e0185c76d9b6f6d805b3e50b00966d9e19fc654a74917686ee44
    [Length: 58]
```

Фиг. 5.24 Данни от втория пакет

```
0000  00 0c 29 6f db 3f 00 0c  29 4a a3 aa 88 b5 02 08   ··)o·?·· )J·····
0010  91 8a 1e ce 79 2c a5 5b  ac 02 c9 bf 9a 3a b3 47   ····y,·[ ·····:·G
0020  28 dd be 90 d7 30 1c 4a  17 b4 8b 7e 01 85 c7 6d   (····0·J ···~···m
0030  9b 6f 6d 80 5b 3e 50 b0  09 66 d9 e1 9f c6 54 a7   ·om·[>P· ·f····T·
0040  49 17 68 6e e4 4e 93 9c                            I·hn·N··
```

Фиг. 5.25 Хекс-дъмп на втория пакет

Пакет 3:

На фигура 5.26 могат да се видят данните на първия пакет от обмена, а на фигура 5.27 е показан хекс-дъмп на пакета. Ясно могат да се видят некриптираните данни: Destination Mac (6 bytes) (В този случай “ff ff ff ff ff ff”,

което показва, че е broadcast пакет), Source MAC (6 bytes) и EtherType (2 bytes) на пакета. Следващият байт показва типа на пакета, който в този случай е 03 (T-ARP DISCOVER Request). Следват данните от прихванатия пакет, от които могат да се прочетат plaintext данните (17 байта), а следващите 16 байта съдържат симетричния MAC. Пакетът завършва с нули, които са падинг, тъй като минималната дължина на пакета е 60 байта.

```
▾ Frame 3: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface vmnet100, id 0
    Section number: 1
  ▸ Interface id: 0 (vmnet100)
    Encapsulation type: Ethernet (1)
    Arrival Time: Feb 17, 2026 18:40:55.725627442 EET
    UTC Arrival Time: Feb 17, 2026 16:40:55.725627442 UTC
    Epoch Arrival Time: 1771346455.725627442
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.020618075 seconds]
    [Time delta from previous displayed frame: 0.020618075 seconds]
    [Time since reference or first frame: 0.023682948 seconds]
    Frame Number: 3
    Frame Length: 60 bytes (480 bits)
    Capture Length: 60 bytes (480 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:data]
    [Coloring Rule Name: Broadcast]
    [Coloring Rule String: eth[0] & 1]
▾ Ethernet II, Src: VMware_6f:db:3f (00:0c:29:6f:db:3f), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  ▸ Destination: Broadcast (ff:ff:ff:ff:ff:ff)
  ▸ Source: VMware_6f:db:3f (00:0c:29:6f:db:3f)
    Type: Local Experimental Ethertype 1 (0x88b5)
▾ Data (46 bytes)
    Data: 03c0a80164c0a801659246e5f06994b62f563d5b60789a5a64c2c95b6c50902fb6000000000000000000000000000
    [Length: 46]
```

Фиг. 5.26 Данни от третия пакет

```
0000  ff ff ff ff ff ff 00 0c  29 6f db 3f 88 b5 03 c0   ........ )o·?····
0010  a8 01 64 c0 a8 01 65 92  46 e5 f0 69 94 b6 2f 56   ··d···e· F··i··/V
0020  3d 5b 60 78 9a 5a 64 c2  c9 5b 6c 50 90 2f b6 00   =[`x·Zd· ·[lP·/··
0030  00 00 00 00 00 00 00 00  00 00 00 00              ........ ····
```

Фиг. 5.27 Хекс-дъмп на третия пакет

Пакет 4:

На фигура 5.28 могат да се видят данните на първия пакет от обмена, а на фигура 5.29 е показан хекс-дъмп на пакета. Ясно могат да се видят некриптираните данни: Destination Mac (6 bytes), Source MAC (6 bytes) и EtherType (2 bytes) на пакета. Следващият байт показва типа на пакета, който в този случай е 04 (T-ARP DISCOVER Reply). Следват данните от прихванатия пакет, от които могат да се прочетат plaintext данните (17 байта), а следващите 16 байта съдържат симетричния MAC. Пакетът завършва с нули, които са падинг, тъй като минималната дължина на пакета е 60 байта.

```
▾ Frame 4: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface vmnet100, id 0
    Section number: 1
  ▸ Interface id: 0 (vmnet100)
    Encapsulation type: Ethernet (1)
    Arrival Time: Feb 17, 2026 18:40:55.745639066 EET
    UTC Arrival Time: Feb 17, 2026 16:40:55.745639066 UTC
    Epoch Arrival Time: 1771346455.745639066
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.020011624 seconds]
    [Time delta from previous displayed frame: 0.020011624 seconds]
    [Time since reference or first frame: 0.043694572 seconds]
    Frame Number: 4
    Frame Length: 60 bytes (480 bits)
    Capture Length: 60 bytes (480 bits)
    [Frame is marked: True]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:data]
▾ Ethernet II, Src: VMware_0b:5c:c6 (00:0c:29:0b:5c:c6), Dst: VMware_6f:db:3f (00:0c:29:6f:db:3f)
  ▸ Destination: VMware_6f:db:3f (00:0c:29:6f:db:3f)
  ▸ Source: VMware_0b:5c:c6 (00:0c:29:0b:5c:c6)
    Type: Local Experimental Ethertype 1 (0x88b5)
▾ Data (46 bytes)
    Data: 04c0a80165c0a801641e3630116994b63494dab9e54c1b202d785a7571c618b8850000000000000000000000000000
    [Length: 46]
```

Фиг. 5.28 Данни от четвъртия пакет

```
0000  00 0c 29 6f db 3f 00 0c  29 0b 5c c6 88 b5 04 c0   ··)o·?·· )·\·····
0010  a8 01 65 c0 a8 01 64 1e  36 30 11 69 94 b6 34 94   ··e···d· 60·i··4·
0020  da b9 e5 4c 1b 20 2d 78  5a 75 71 c6 18 b8 85 00   ···L· -x Zuq·····
0030  00 00 00 00 00 00 00 00  00 00 00 00               ........ ····
```

Фиг. 5.29 Хекс-дъмп на четвъртия пакет

5.4.4 Обобщение на получените резултати

От наблюдавания обмен на пакети и от структурата на данните в тях може да се заключи че съвпадат с описания метод на работа, което доказва възможността за имплементация на T-ARP в реална мрежова обстановка.

Шеста глава: Заключение

Настоящата дипломна работа е насочена към изследване и практическо решаване на проблема със сигурността при динамично обвързване на MAC и IP адреси в IPv4 мрежи. Анализът на стандартния ARP протокол показа, че неговата архитектура е изградена върху доверителен модел, който не включва механизми за автентикация, криптографска защита или проверка на целостта на обменяната информация. Именно тази фундаментална слабост позволява реализирането на атаки от тип подмяна на ARP кеша (ARP spoofing), "човек по средата" (Man-in-the-Middle), отказ от услуга (Denial of Service) и превземане на сесии (Session hijacking), които могат да компрометират конфиденциалността, целостта и наличността на мрежовите услуги.

Поставената цел на дипломната работа е да се проектира, разработи и верифицира подсигурен протокол за обвързване MAC-IP и IP-MAC, който да елиминира описаните уязвимости чрез интегриране на съвременни криптографски механизми. В хода на разработката беше създаден протоколът T-ARP (Trusted Address Resolution Protocol), който представлява защитено разширение на стандартния ARP протокол и функционира на второ ниво от OSI модела, използвайки сурови Ethernet кадри. Основната концепция на T-ARP се базира на централизирано управление на ключове за защита на комуникацията чрез доверен шлюз и използване на алгоритъма AES-GCM за осигуряване на конфиденциалност, интегритет и автентичност на комуникацията между крайните устройства и шлюза, както и симетричен MAC за защита на директната комуникация между крайните устройства.

Разработката обхваща не само теоретично моделиране на протокола, но и неговата практическа реализация под формата на софтуерен модул за операционната система VyOS. Беше създаден Debian пакет, реализиращ T-ARP Gateway демон, който обработва сурови Ethernet пакети, валидира криптографските параметри, управлява база данни с ключове и осигурява сигурен механизъм за разпределение на симетрични ключове между крайните устройства. Чрез използването на AES-GCM и допълнително автентикирани данни (AAD) беше постигнато криптографско обвързване между съдържанието на пакета и неговия Ethernet хедър, което елиминира възможността за злонамерена подмяна на MAC адреси без притежание на валиден ключ.

Проведените експериментални тестове в емулирана мрежова среда потвърдиха работоспособността и ефективността на разработеното решение. Използването на nonce и времеви печати осигурява защита срещу преизползване на стари пакети, а централизираната архитектура позволява надеждно управление на криптографските ключове. По този начин беше демонстрирано, че е възможно значително повишаване на сигурността на комуникация в локалните мрежи чрез модифициране на механизма за адресна резолюция, без да се засягат по-горните слоеве на мрежовия стек.

Въпреки постигнатите резултати, разработката има определени ограничения. Тя е фокусирана единствено върху IPv4 среда и не разглежда аналогични механизми за IPv6 и Neighbor Discovery Protocol. Освен това използваният централен шлюз представлява критична точка в архитектурата, което налага висока степен на надеждност и защита на самата система. Тези аспекти очертават възможни направления за бъдещо развитие, включително

разширяване към IPv6, интеграция с хардуерни модули за защита на ключове и оптимизация на управлението на криптографските материали.

В заключение може да се обобщи, че поставената цел на дипломната работа е изпълнена успешно. Проектираният и реализиран T-ARP протокол представлява функционално и криптографски обосновано решение на проблема със сигурността при динамичното обвързване на MAC и IP адреси. Работата демонстрира, че чрез внимателно инженерно проектиране и прилагане на съвременни криптографски методи е възможно да се преодолеят дългогодишни архитектурни слабости в основополагащи мрежови протоколи и да се създаде основа за по-сигурна локална мрежова инфраструктура.

Седма глава: Бъдещо развитие

В тази глава са разгледани някои от възможностите за бъдещо развитие и подобряване на новосъздадения T-ARP протокол.

7.1 Разширяване към IPv6 среда

Настоящата реализация на T-ARP е фокусирана върху IPv4 мрежи и разширява функционалността на ARP с криптографски защитен механизъм. В бъдеще протоколът може да бъде разширен и към IPv6, където аналогичната роля се изпълнява от Neighbor Discovery Protocol (NDP). Макар че IPv6 предвижда Secure Neighbor Discovery (SEND), неговата сложност и зависимост от PKI инфраструктура ограничават широкото му приложение.

Възможно направление за развитие е създаването на T-NDP - защитена версия на NDP, базирана на симетричен криптографски модел и AEAD механизми. Това би позволило унифицирана архитектура за сигурна адресна резолюция както в IPv4, така и в IPv6 среди, без необходимост от тежка асиметрична инфраструктура.

7.2 Усъвършенстване на управлението на криптографските ключове

В текущата реализация ключовете се генерират и съхраняват локално, без дефинирана политика за изтичане и ротация. В реална продукционна среда е необходимо въвеждане на по-строг модел за управление на криптографския материал.

Бъдещи версии на протокола могат да включват автоматична ротация на ключовете, дефиниране на срок на валидност (key lifetime), механизъм за отмяна при компрометиране и използване на forward secrecy чрез периодично извеждане на нови сесийни ключове. Интеграцията със системи за управление на тайни данни би позволила по-висока степен на защита и централизирано администриране.

7.3 Имплементиране на механизъм за осъществяване на първоначална връзка с шлюза (GW)

Както е описано в **т. 2.6** е необходимо да се имплементира механизъм за първоначална комуникация с централизирания шлюз.

7.4 Интеграция с хардуерни механизми за защита

Криптографските ключове в настоящата версия се съхраняват в софтуерна база данни, което при компрометиране на системата може да доведе до изтичане на чувствителна информация.

Бъдещо развитие е необходимо да включва съхранение на основните ключове в Trusted Platform Module (TPM) или Hardware Security Module (HSM), както и криптиране на локалната база данни. Интеграцията със secure boot механизми би гарантирала целостта на изпълнимия код и би повишила устойчивостта срещу локални атаки.

7.5 Оптимизация на производителността

Настоящата имплементация е реализирана на Python и използва raw sockets, което е подходящо за прототипна и изследователска среда, но за среди с висока плътност на трафика се налагат оптимизации.

Възможно е пренаписване на критичните части на по-ниско ниво, например на C или Rust, както и използване на хардуерно ускорение чрез AES-NI. Допълнителни подобрения могат да бъдат постигнати и чрез паралелна обработка на пакети или използване на kernel-level механизми като eBPF.

7.6 Разширяване на функционалността и интеграция в цялостна стратегия за сигурност

T-ARP може да бъде надграден като част от по-широка архитектура за мрежова сигурност. Интеграцията със системи за централизирано логване и SIEM би позволила анализ на събитията и откриване на аномалии на канално ниво.

Допълнително протоколът може да бъде обвързан със системи за контрол на достъпа до мрежата (NAC), което би позволило ограничаване на устройства със съмнително поведение. По този начин T-ARP може да се превърне в елемент от Zero Trust мрежова архитектура.

7.7 Потенциал за стандартизация

При по-нататъшно развитие новосъздаденият протокол T-ARP може да бъде формализиран като пълна техническа спецификация в стил RFC документ. За целта е необходимо детайлно описание на форматите на пакетите, състоянията на протокола и процедурите за обработка.

Подобна стъпка би отворила възможност за експериментално внедряване и евентуална стандартизация, което би разширило приложимостта на T-ARP извън рамките на изследователската среда.

7.8 Обобщение

Предложените направления за бъдещо развитие показват, че протоколът T-ARP има потенциал да еволюира от експериментален протокол към индустриално приложима архитектура. Чрез разширяване към IPv6, повишаване на устойчивостта, оптимизация на производителността и формална верификация, протоколът T-ARP може да се превърне в основа за изграждане на сигурни механизми за адресна резолюция в съвременните мрежови инфраструктури.

Източници

[1]. What is ARP?

https://www.fortinet.com/resources/cyberglossary/what-is-arp

[2]. Address Resolution Protocol (ARP)

https://ipcisco.com/lesson/address-resolution-protocol-arp/

[3]. ARP Spoofing

https://www.imperva.com/learn/application-security/arp-spoofing/

[4]. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC

https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

[5]. The Use of Galois/Counter Mode (GCM) in IPsec Encapsulating Security Payload (ESP)

https://datatracker.ietf.org/doc/html/rfc4106

[6]. An Interface and Algorithms for Authenticated Encryption

https://datatracker.ietf.org/doc/html/rfc5116

[7]. Build VyOS

https://docs.vyos.io/en/latest/contributing/build-vyos.html

[8]. GNS3 Linux Install

https://docs.gns3.com/docs/getting-started/installation/linux/

[9]. Alpine Linux

https://www.alpinelinux.org/downloads/

[10]. Adding VMware VMs to GNS3 Topologies

https://docs.gns3.com/docs/emulators/adding-vmware-vms-to-gns3-topologies/